

Module 3

Module III

Gradient Descent (GD), Momentum Based GD, Nesterov Accelerated GD, Stochastic GD, AdaGrad, RMSProp, Adam, Eigenvalues and eigenvectors, Eigenvalue Decomposition, Basis, Principal Component Analysis and its interpretations, Singular Value Decomposition. Autoencoders and relation to PCA, Regularization in autoencoders, Denoising autoencoders, Sparse autoencoders, Contractive autoencoders

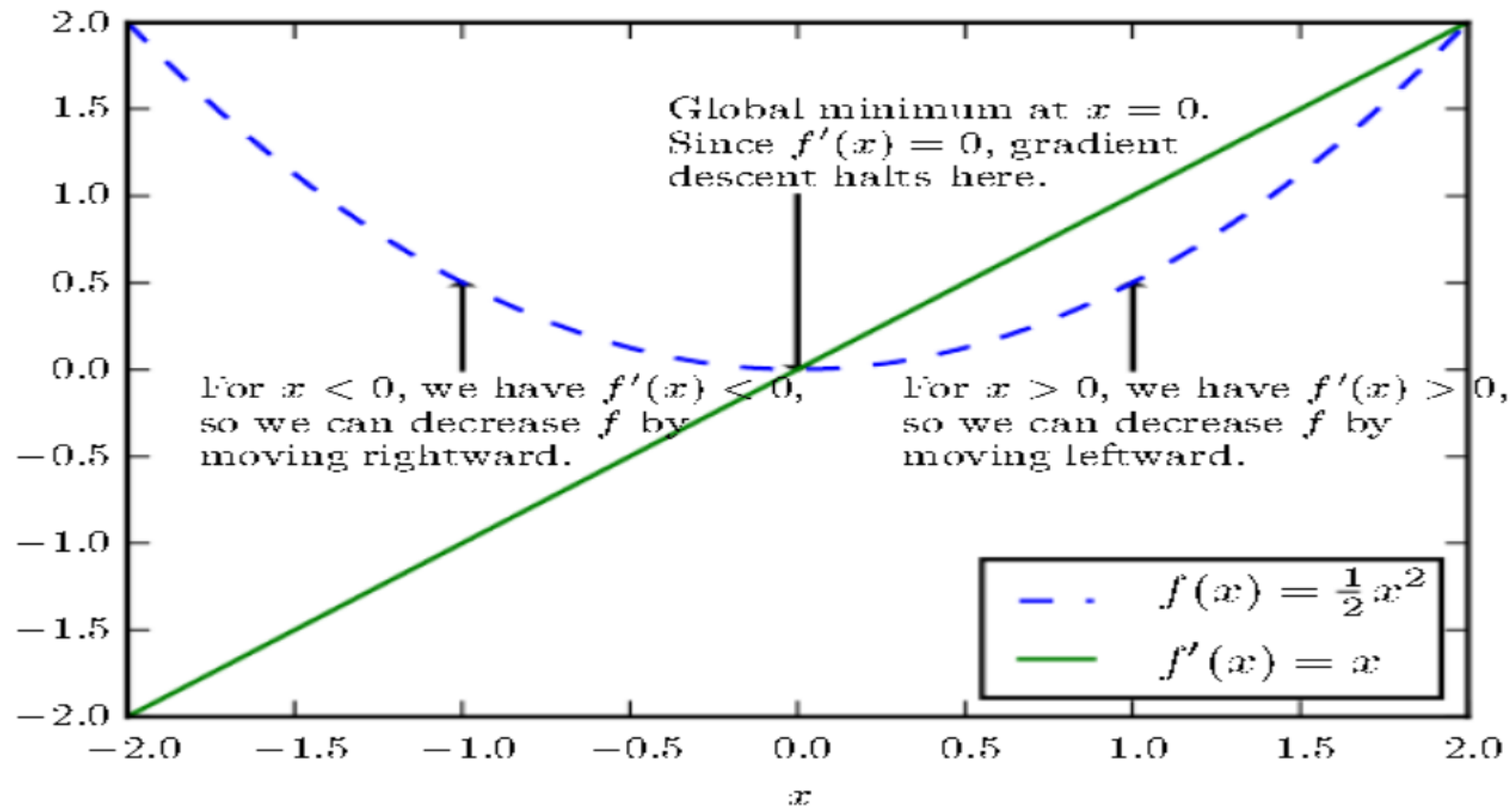
Eigen Values

- 1 An **eigenvector** x lies along the same line as Ax : $Ax = \lambda x$. The **eigenvalue** is λ .
- 2 If $Ax = \lambda x$ then $A^2x = \lambda^2x$ and $A^{-1}x = \lambda^{-1}x$ and $(A + cI)x = (\lambda + c)x$: the same x .
- 3 If $Ax = \lambda x$ then $(A - \lambda I)x = 0$ and $A - \lambda I$ is singular and $\det(A - \lambda I) = 0$. n eigenvalues.
- 4 Check λ 's by $\det A = (\lambda_1)(\lambda_2) \cdots (\lambda_n)$ and diagonal sum $a_{11} + a_{22} + \cdots + a_{nn} = \text{sum of } \lambda\text{'s}$.
- 5 Projections have $\lambda = 1$ and 0 . Reflections have 1 and -1 . Rotations have $e^{i\theta}$ and $e^{-i\theta}$: *complex!*

Gradient Descent:

- Optimization refers to the task of either minimizing or maximizing some function $f(x)$ by altering x .
- most optimization problems in terms of minimizing $f(x)$.
- The function we want to minimize or maximize is called the **objective function or criterion**.
- When we are minimizing it, we may also call it the **cost function, loss function, or error function**.
- the value that minimizes or maximizes a function with a superscript $*$. For example, we might say $x^* = \text{argmin } f(x)$.

Gradient Descent:

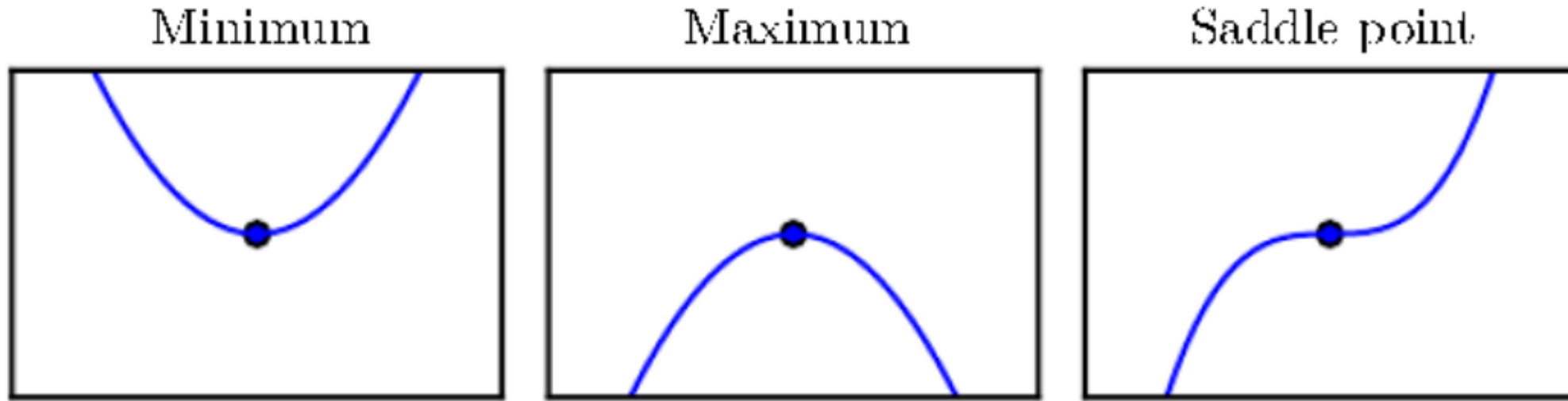


An illustration of how the gradient descent algorithm uses the derivatives of a function can be used to follow the function downhill to a minimum

Gradient Descent

- When $f'(x) = 0$, the derivative provides no information about which direction to move.
- Points where $f'(x) = 0$ are known as critical points or stationary points.
- A local minimum is a point where $f(x)$ is **lower than at all neighboring**
- points, so it is no longer possible to decrease $f(x)$ by making infinitesimal steps.
- A local maximum is a point where $f(x)$ is higher than at all neighboring points.
- Some critical points are neither maxima nor minima. These are known as saddle points

Gradient Descent



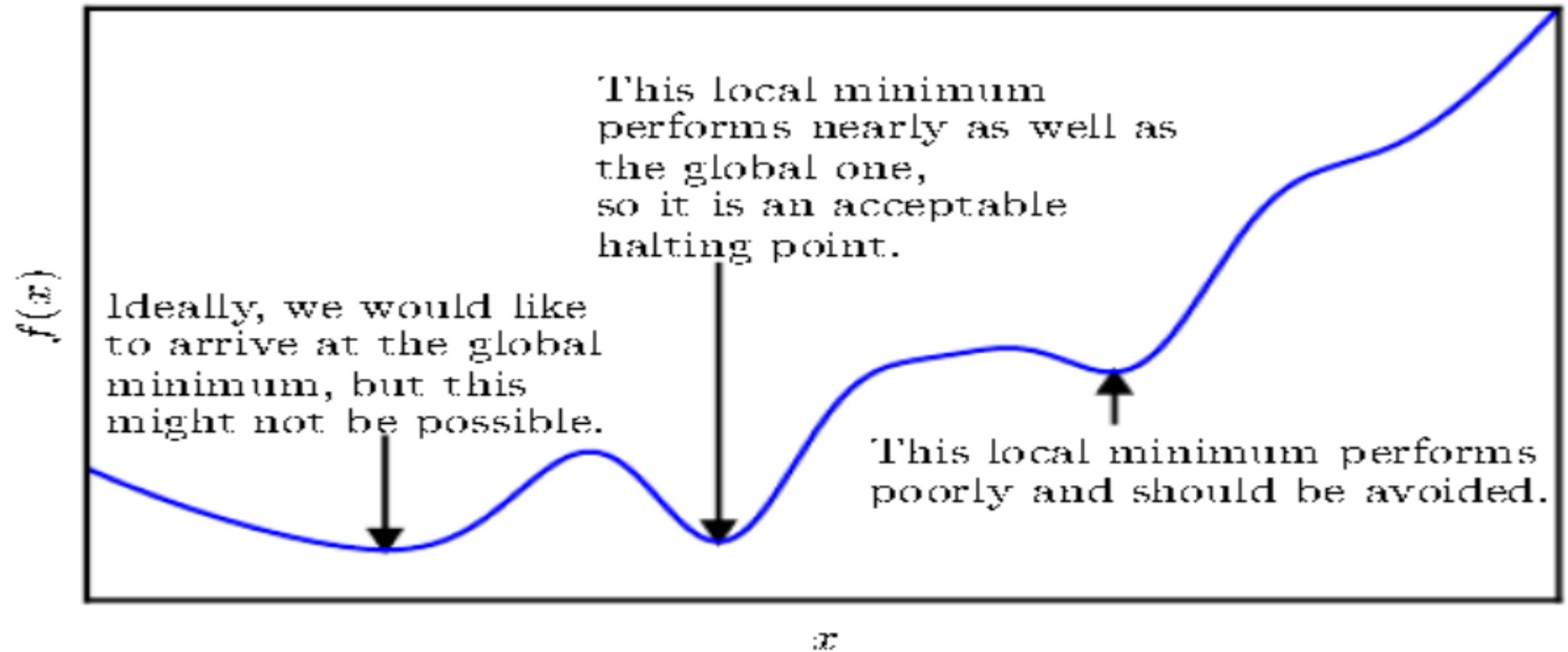
Examples of each of the three types of critical points in 1-D.

A critical point is a point with zero slope. Such a point can either be a local minimum, which is lower than the neighboring points, a local maximum, which is higher than the neighboring points, or a saddle point, which has neighbors that are both higher and lower than the point it

Gradient Descent

- A point that obtains absolute lowest value of $f(x)$ is a global min.
- It is possible for there to be only one global min or multiple global min of the function.
- It is also possible for there to be local min that are not globally Optimal.
- In the context of deep learning, we optimize functions that may have many local minima that are not optimal, and many saddle points surrounded by very flat regions.
- All of this makes optimization very difficult, especially when the input to the function is multidimensional.

Gradient Descent



Gradient Descent:

- **Gradient Descent** is an optimization algorithm used to minimize a function by iteratively moving towards the **steepest descent** direction, which is indicated by the **negative of the gradient**.
- **Objective Function**: The function you want to minimize (e.g., loss function in machine learning).
- **Gradient**: The vector of **partial derivatives of the function**, indicating the direction of **steepest ascent**.
- **Learning Rate (α)**: A hyperparameter that determines the step size for each iteration.

Gradient Descent:

- 1) initialize the parameters (e.g., θ).
- 2) Compute the Gradient of the function at the current parameter values.
- 3) Update the Parameters
 - $\theta = \theta - \alpha \nabla J(\theta)$
- **4) Repeat** steps 2-3 until convergence
- (i.e., when the change in the function value is smaller than a threshold).

Gradient Descent:

- $\theta = \theta - \alpha \nabla J(\theta)$
- **Example**
- example of minimizing the quadratic function:

$$\begin{aligned} j(\theta) &= \theta^2 + 4\theta + 4 \\ &= (\theta + 2)^2 \end{aligned}$$

The minimum occurs at $\theta = -2$.

Compute the Gradient

$$\nabla J(\theta) = \frac{d}{d(\theta)} (\theta^2 + 4\theta + 4) = 2\theta + 4$$

Gradient Descent:

- **2. Set Initial Parameters**

- **Initial parameter:** $\theta=5$

- **Learning rate (α):** 0.1

- Perform Iterations

- **Iteration 1**

- **Compute Gradient:**

- $\nabla J(5)=2 \cdot 5+4=14$

- **Update Parameter:**

- $\theta=5-0.1 \cdot 14=5-1.4=3.6$

- **Iteration 2**

- **Compute Gradient:**

- $\nabla J(3.6)=2 \cdot 3.6+4=11.2$

- **Update Parameter:**

- $\theta=3.6-0.1 \cdot 11.2=2.48$

- **Iteration 3**

- **Compute Gradient:**

- $\nabla J(2.48)=2 \cdot 2.48+4=8.96$

- **Update Parameter:**

- $\theta=2.48-0.1 \cdot 8.96=1.584$

- **Iteration 4**

- **Compute Gradient:**

- $\nabla J(1.584)=2 \cdot 1.584+4=7.168$

- **Update Parameter:**

- $\theta=1.584-0.1 \cdot 7.1768=0.8672$

- **Iteration 5**

- **Compute Gradient:**

- $\nabla J(0.8672)=2 \cdot 0.8672+4=5.7344$

- **Update Parameter:**

- $\theta=0.8672-0.1 \cdot 5.7344=0.29376$

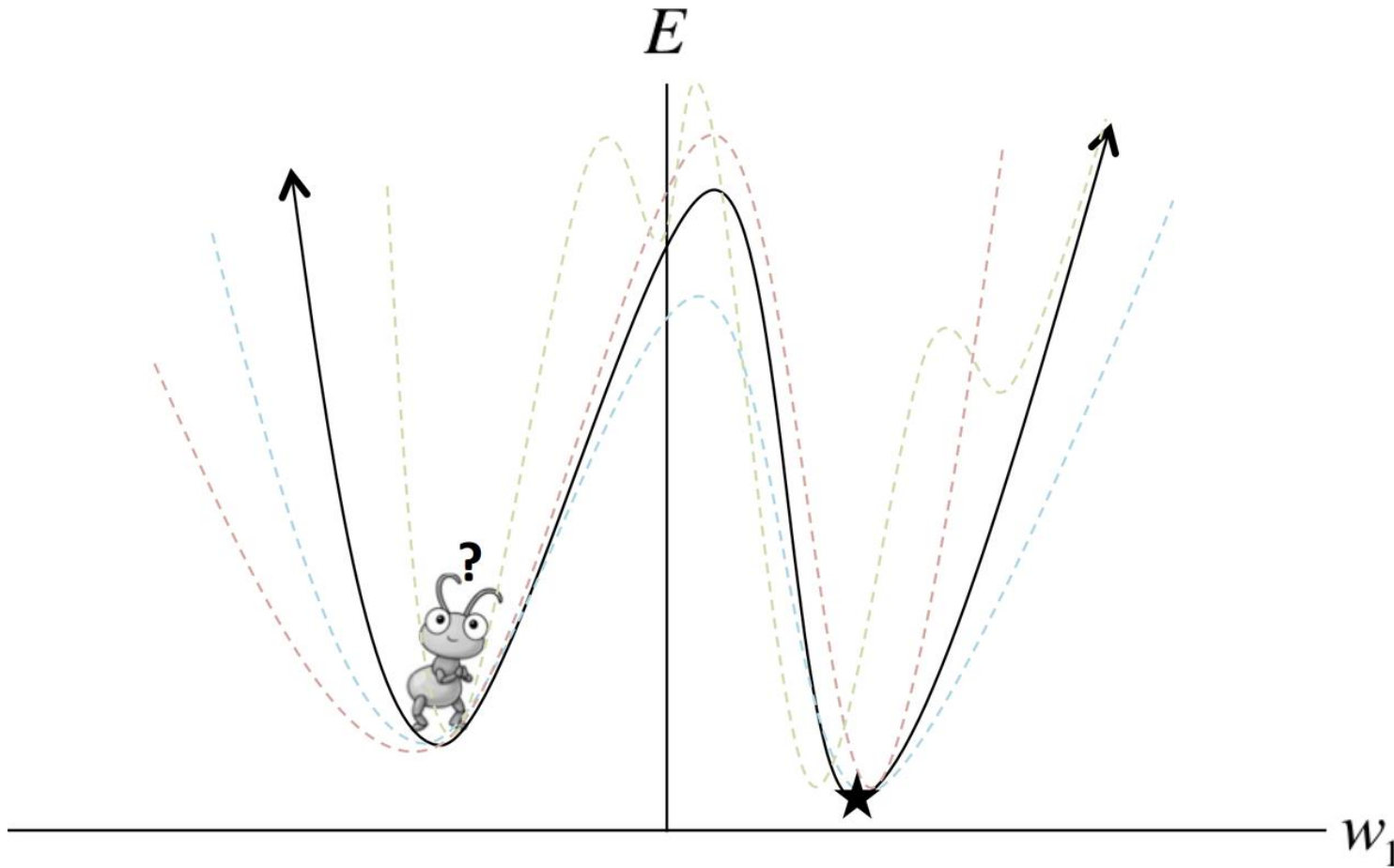
It iteratively updates parameters to minimize a cost function,

$$\theta = 3.6 \rightarrow 2.48 \rightarrow 1.584 \rightarrow 0.8672 \rightarrow 0.29376$$

The Challenges with Gradient Descent

- The primary challenge in optimizing deep learning models is that we are forced to use **minimal local information** to infer the global structure of the error surface.
- This is a hard problem because there is usually very little correspondence between local and global structure.
- **For nonconvex surface**, which means that even if we find a valley (a local minimum), we have no idea if it's the lowest valley on the map (the global minimum).

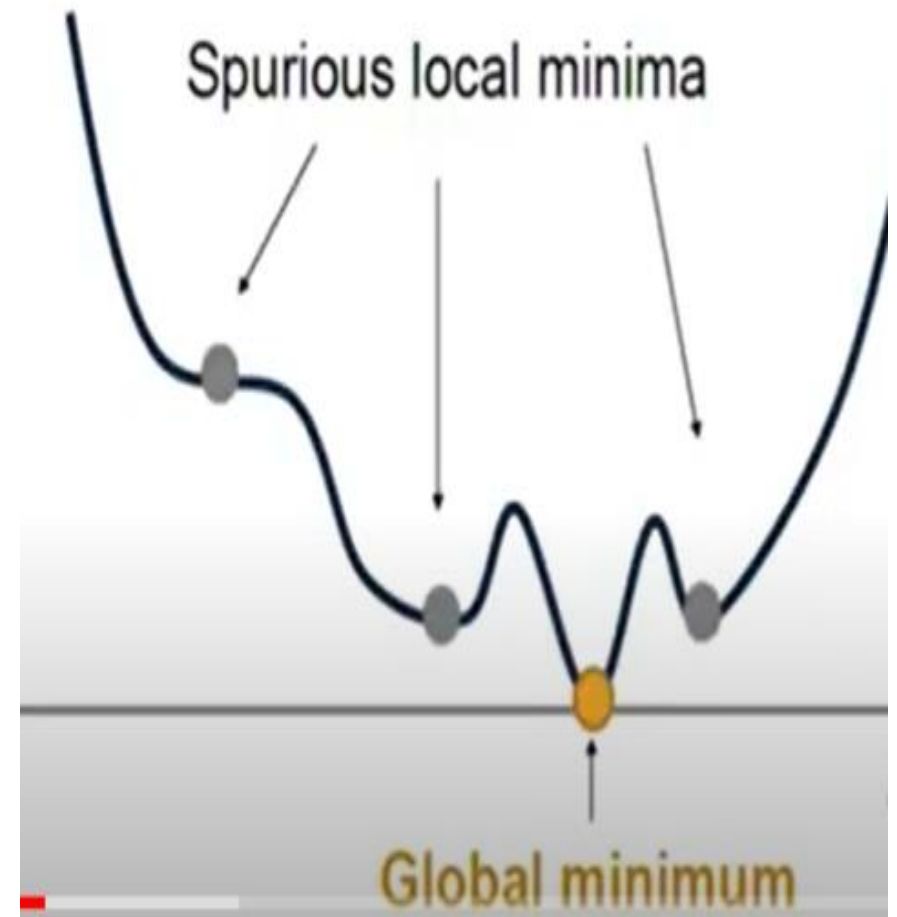
The Challenges with Gradient Descent



Mini-batch gradient descent may aid in escaping shallow local minima, but often fails when dealing with deep local minima, as shown

The Challenges with Gradient Descent

- One observation about deep neural networks is that their error surfaces are guaranteed to have a large
- —and in some cases, **an infinite—number of local minima.**
- local minima are only problematic when **they are *spurious*.**
- A spurious local minimum **incurs a higher error** than the configuration at the global minimum.
- If these kinds of local minima are common, we quickly run into significant problems while using gradient based optimization methods because we can only take into account local structure



The Challenges with Gradient Descent

- **Loss Landscape in Deep Networks:**
- Deep neural networks **have complex loss landscapes** due to **their high dimensionality and non-linearity**.
- This complexity can result in **many local minima**.
- In high-dimensional spaces, **the chance of encountering a local minimum is higher**, and many of these minima can be spurious.
- **Characteristics:**
- Spurious local minima are not necessarily the worst possible minima, but they can be worse than the global minimum or other more optimal minima.
- These minima can trap optimization algorithms, **preventing them from finding the best solution**.

- **Impact on Training:**
- Getting stuck in a spurious local minimum can affect the performance of the model, **leading to poorer generalization on unseen data.**
- The model might converge to a suboptimal set of parameters, reducing its effectiveness.
- **Strategies to Mitigate Spurious Local Minima**
- **Advanced Optimization Algorithms:**
 - **Stochastic Gradient Descent (SGD)** and its variants (e.g., Adam, RMSprop) introduce noise into the gradient updates, which can help escape local minima.
 - **Momentum-based methods** can help by providing a form of inertia that might push the optimizer out of local minima.

- **Regularization Techniques:**

- Techniques like Dropout or weight decay can help generalize better and avoid overfitting to local minima.

- **Network Architecture:**

- Batch Normalization (normalizes the outputs of each layer by adjusting and scaling the activations.)
- and Residual Networks (ResNets) can help by smoothing the optimization landscape.
- Skip connections and deeper architectures can also help in reducing the chances of getting stuck in local minima.

- **Learning Rate Strategies:**

- Using a learning rate schedule that gradually reduces the learning rate can help in fine-tuning the model and avoiding local minima.

- **Initialization Strategies:**

- Proper initialization of weights can help start the optimization process in a region that is less likely to trap the model in poor local minima.

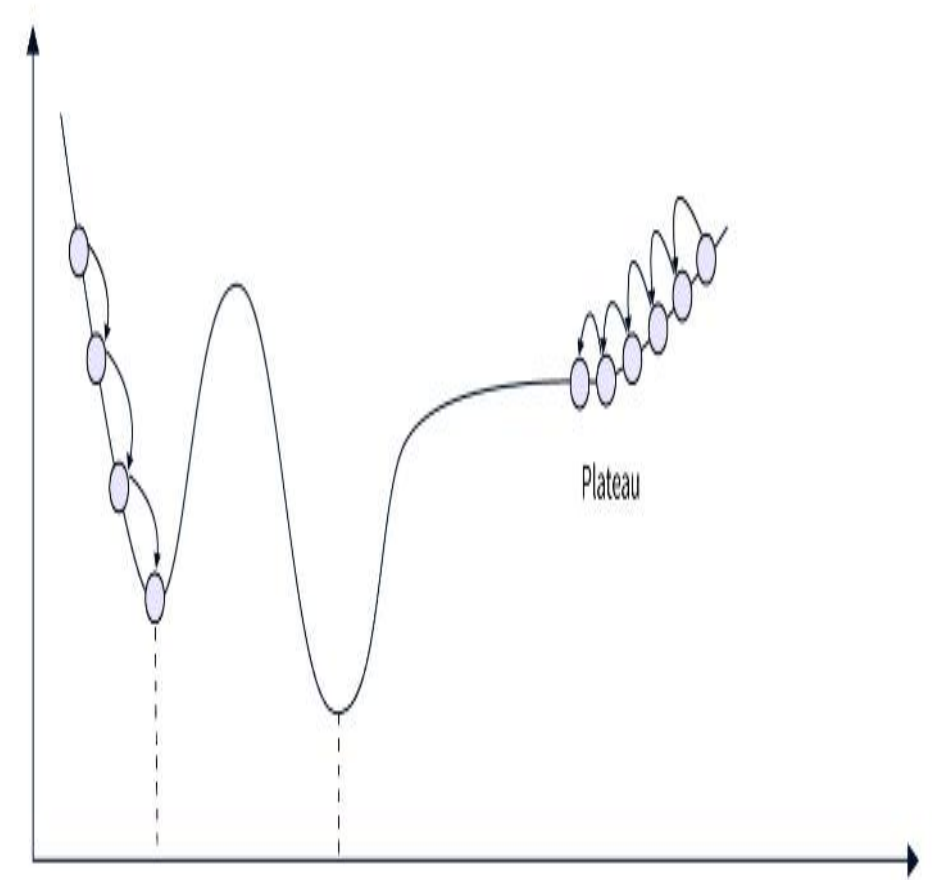
- **Ensemble Methods:**

- Training multiple models and combining their predictions can sometimes mitigate the impact of spurious local minima, as different models might end up in different minima.

- **Hyperparameter Tuning:**

- Careful tuning of hyperparameters (e.g., learning rate, batch size) can impact the optimization process and potentially help in avoiding spurious minima.

- **Flat Regions:** These are areas in **the loss landscape** where the gradient (or change in the loss function) is **very small**.
- This means the surface is relatively flat in these regions.
- **Impact of Flat Regions:**
- **Gradient Magnitude:** This makes it difficult for optimization algorithms, gradient-based methods like **SGD**, to make **significant progress**.
- **Slow Convergence:** Because the gradient is small, leading to slow convergence or stagnation.

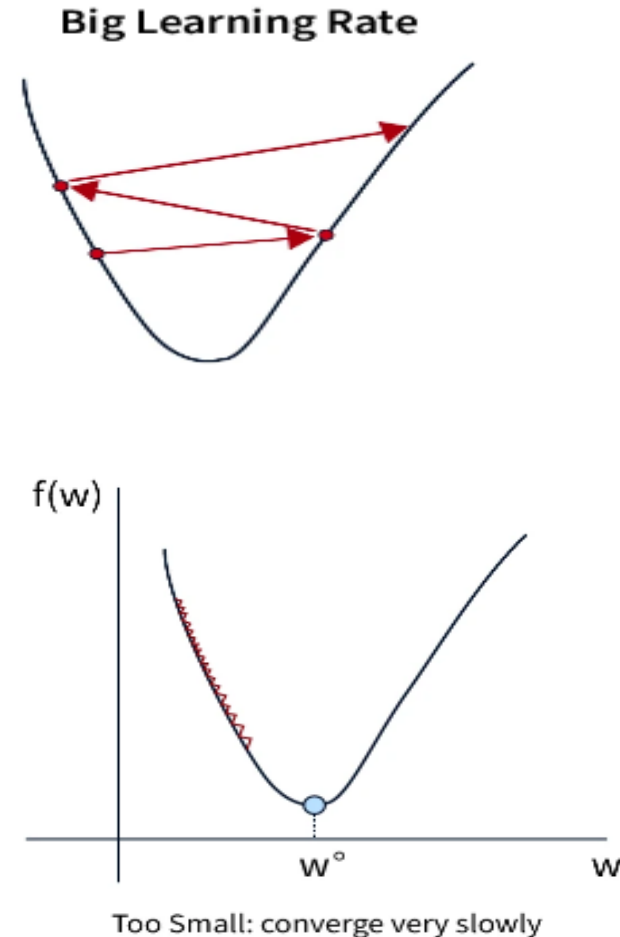


When Gradient Points in the Wrong Direction

- **Local Minima or Maxima:**
- **with non-convex functions**, the gradient might point in a direction that leads to a local minimum or maximum rather than the global one.
- This means that even if the gradient is pointing in a direction of increasing value locally, it might not be the direction to find the global optimum.

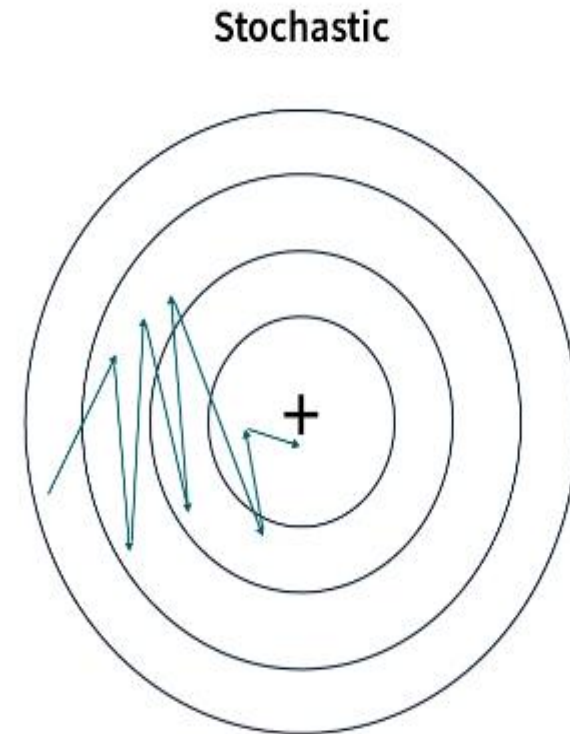
When Gradient Points in the Wrong Direction

- **Gradient Descent Algorithm Issues:**
- A too-large step size might cause **overshooting (diverge)** the min, **Oscillate back and forth**
- and a too-small step size might lead to very **slow convergence**.
- This can sometimes make it seem like the gradient is pointing in the wrong direction.



When Gradient Points in the Wrong Direction

- **Scaling and Normalization:**
- Make sure that the input features are appropriately scaled or normalized.
- Sometimes gradients can behave unexpectedly if the feature scales are very different from each other.
- **Stochasticity:**
- In stochastic gradient descent, the cost function is evaluated at random samples from the data set.
- This introduces randomness into the algorithm, making converging to a global minimum more difficult.



Circular Contour

- $J(x, y) = x^2 + y^2$
- **Gradient Direction:** At any point on a circular contour, the gradient will always point radially outward from the center
- $\nabla J(x, y) = (2x, 2y)$
- The gradient **points away from the origin** (the local minimum at (0,0))
- **Moving Towards the Minimum:** If the contours are circular, the gradient does **not** point towards the local minimum;
- instead, it points away from it.
- To find the local minimum, you would need to move **in the opposite direction** of the gradient

Circular Contour

- **Gradient Descent:** Therefore, in the context of gradient descent, you would update your parameters by moving against the gradient (i.e., towards the local minimum),
- $\theta = \theta - \alpha \nabla J(\theta)$

Example

- $J(x, y) = x^2 + y^2$
- $\nabla J(x, y) = (2x, 2y)$

Circular Contour

- **iteration 1**

- **Compute Gradient:**

$$\nabla J(3,4)=(2\cdot 3,2\cdot 4)=(6,8)$$

- Update Parameters:**

$$x_{\text{new}}=3-0.1\cdot 6=3-0.6=2.4$$

$$y_{\text{new}}=4-0.1\cdot 8=4-0.8=3.2$$

New point: **(2.4,3.2)**

- **iteration 2**

- **Compute Gradient:**

$$\nabla J(2.4,3.2)=(2\cdot 2.4,2\cdot 3.2)=(4.8,6.4)$$

- Update Parameters:**

$$x_{\text{new}}=2.4-0.1\cdot 4.8=2.4-0.48=1.92$$

$$y_{\text{new}}=3.2-0.1\cdot 6.4=3.2-0.64=2.56$$

New point: **(1.92,2.56)**

Circular Contour

- iteration 3

- **Compute Gradient:**

$$\nabla J(\mathbf{1.92, 2.56}) = (2 \cdot 1.92, 2 \cdot 2.56) = (3.84, 5.12)$$

- Update Parameters:**

$$\begin{aligned} x_{\text{new}} &= 1.92 - 0.1 \cdot 3.84 = 1.92 - 0.384 \\ &= 1.536 \end{aligned}$$

$$\begin{aligned} y_{\text{new}} &= 2.56 - 0.1 \cdot 5.12 = 2.56 - 0.512 \\ &= 2.048 \end{aligned}$$

New point: (1.536, 2.048)

- iteration 4

- **Compute Gradient:**

$$\nabla J(\mathbf{1.536, 2.048}) = (2 \cdot 1.536, 2 \cdot 2.048) = (3.072, 4.096)$$

- Update Parameters:**

$$\begin{aligned} x_{\text{new}} &= 1.536 - 0.1 \cdot 3.072 = 1.536 - 0.3072 \\ &= 1.2288 \end{aligned}$$

$$\begin{aligned} y_{\text{new}} &= 2.048 - 0.1 \cdot 4.096 = 2.048 - 0.4096 \\ &= 1.6384 \end{aligned}$$

New point: (1.2288, 1.6384)

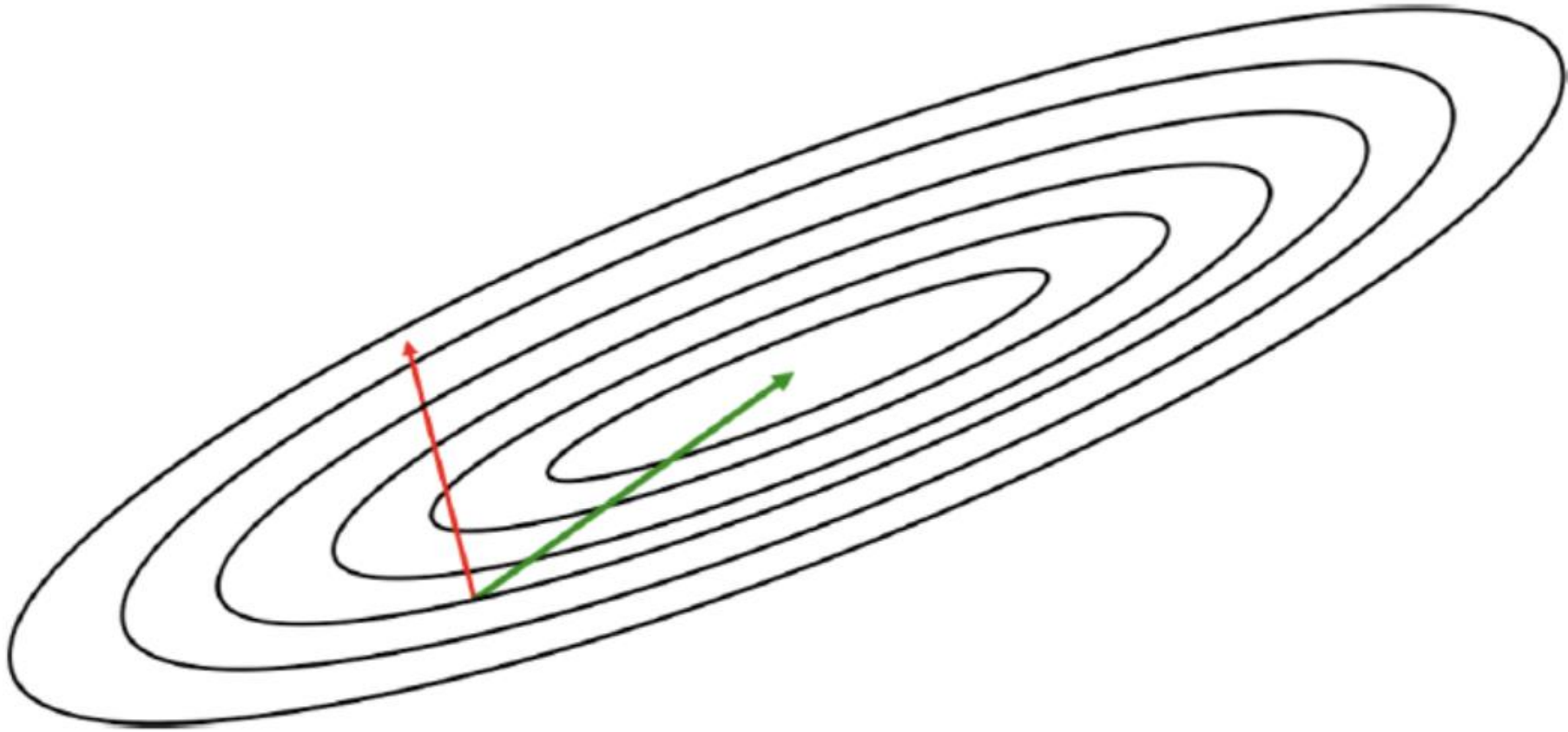
Circular Contour

- **Gradient Direction:** In each iteration, the gradient points away from the origin, indicating the direction of steepest ascent. We moved in the opposite direction (toward the local minimum).
- **Convergence:** With each iteration, the coordinates (x,y) move closer to the origin $(0, 0)$, where the function has its minimum.
- **Circular Contours:** Since the contours are circular, the updates effectively navigate towards the minimum efficiently without oscillating.
- The gradient provides the direction of steepest ascent, and by moving in the opposite direction, we effectively converge toward the local minimum.

Circular Contour

- **No Overshooting or Misalignment:** There is no risk of the gradient leading you away from the minimum or causing oscillation due to misalignment, as is possible with highly elliptical contours.
- There are no issues of directional inaccuracies like those that can occur with elliptical contours, ensuring efficient and effective optimization.

When Gradient Points in Wrong Direction



Local information encoded by the gradient usually does not corroborate the global structure of the error surface

Ellipse

- $J(x, y) = 4x^2 + y^2$
- The global minimum occurs at the origin (0,0).
- **Compute the Gradient**
- $\nabla J(x, y) = (8x, 2y)$
- **Initial point:** Let's start at $(x, y) = (2, 2)$
- **Learning rate (α):** 0.1

Iteration 1

Compute Gradient:

$$\nabla J(2, 2) = (8 \cdot 2, 2 \cdot 2) = (16, 4)$$

Update Parameters:

$$x_{\text{new}} = 2 - 0.1 \cdot 16 = 2 - 1.6 = 0.4$$

$$y_{\text{new}} = 2 - 0.1 \cdot 4 = 2 - 0.4 = 1.6$$

New point: (0.4, 1.6)

Ellipse

- Iteration 1: $(0.4, 1.6)$
- Iteration 2: $(0.08, 1.28)$
- Iteration 3: $(0.016, 1.024)$
- Iteration 4: $(0.0032, 0.8192)$

At each iteration, the gradient points towards the steepest ascent, which means we move in the opposite direction to find the minimum.

Convergence: Each update brings us closer to the minimum at $(0,0)$.

The gradient updates might cause larger movements in one direction than the other due to the aspect ratio of the ellipse

When dealing with functions that have extremely elliptical contours

- **Elliptical Contours:** When contours are highly elongated (i.e., very elliptical), the gradient at a point may not provide a good direction toward the minimum.
- This is because the steepness of the gradient can vary significantly along different axes.
- **Gradient Direction:** The gradient vector points in the direction of steepest ascent.
- In cases of extreme ellipticity, the steepest ascent could be misaligned with the shortest path to the minimum.
- This can make the update direction seem like it's pointing away from the local minimum.

When dealing with functions that have extremely elliptical contours

1. **Point of Interest:** Let's say we start at $(2, 1)$.

2. **Compute Gradient:**

$$\nabla J(2, 1) = (8 \cdot 2, 2 \cdot 1) = (16, 2)$$

3. **Update Parameters** (assuming a learning rate of 0.1):

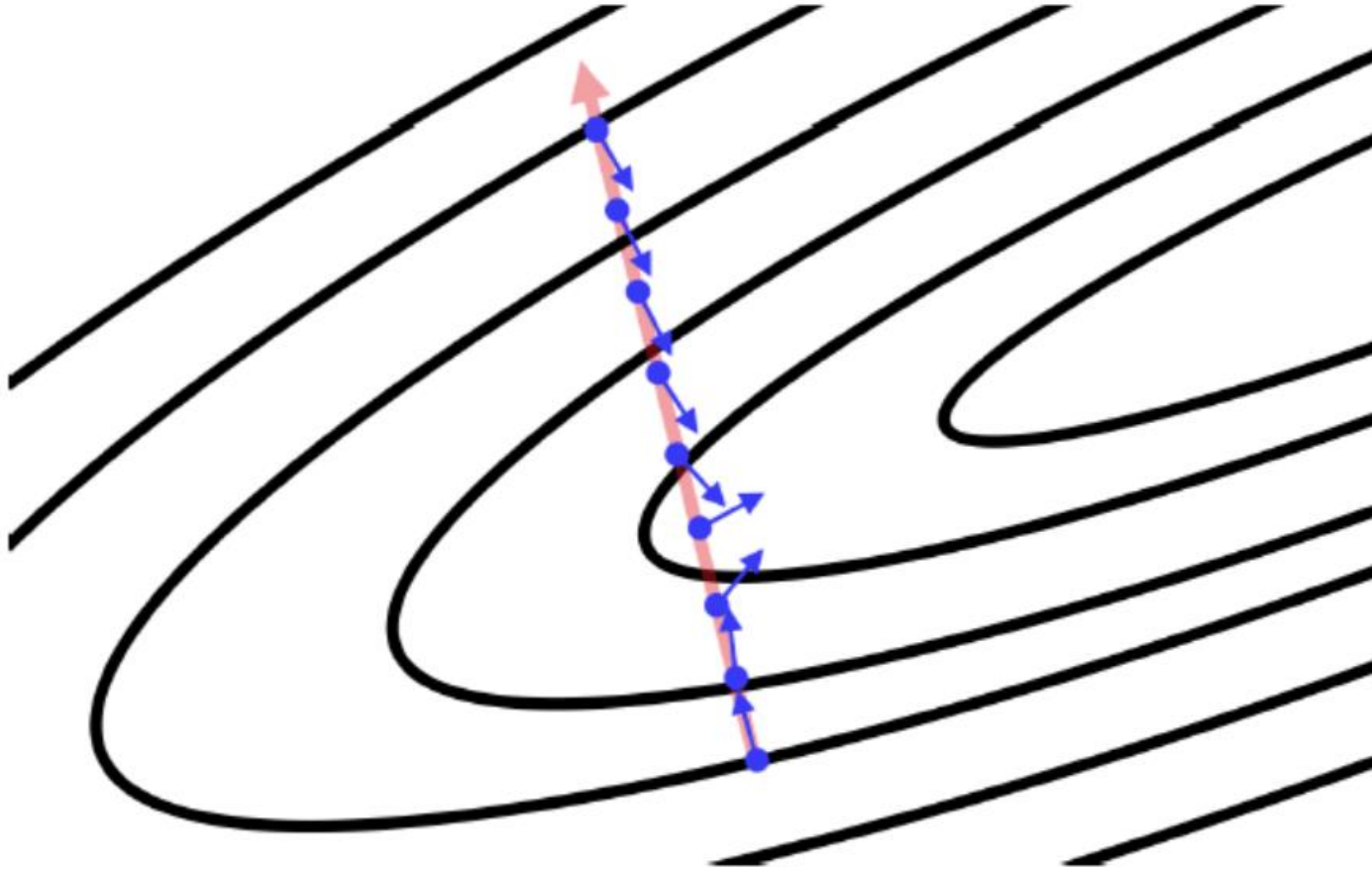
$$x_{\text{new}} = 2 - 0.1 \cdot 16 = 2 - 1.6 = 0.4$$

$$y_{\text{new}} = 1 - 0.1 \cdot 2 = 1 - 0.2 = 0.8$$

When dealing with functions that have extremely elliptical contours

- **Actual Direction:** The update moves from $(2,1)$ to $(0.4,0.8)$
- **Contour Analysis:** The actual minimum is at $(0,0)$.
- The update direction based on the gradient may point more towards $(0.4,0.8)$ rather than directly toward the minimum,
- **Overshooting:** If you continue to iterate, the gradient could potentially lead you to oscillate or move in directions that seem correct based on local gradient information but take you further away from the minimum.
- **Gradient as 90 Degrees Off:** In cases where the contours are extremely elliptical, you might experience scenarios where the effective movement toward the minimum is significantly misaligned.
- This could make the gradient seem like it's pointing almost perpendicular (or 90 degrees off) from the correct path toward the local minimum.

When Gradient Points in Wrong Direction



We show how the direction of the gradient changes as we move along the direction of steepest descent

When Gradient Points in Wrong Direction

- The **elliptical nature of the contours** can be related to the **condition number of the Hessian matrix of the function**.
- An elliptical contour indicates that the function's Hessian matrix has eigenvalues of varying magnitudes.
- This often leads to slower convergence rates in **gradient-based optimization** methods unless the problem is well-scaled.

When Gradient Points in Wrong Direction

- The Hessian matrix of a function $f(x,y)$ is a square matrix of second-order partial derivatives:

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

$$f(x, y) = \frac{x^2}{10} + y^2$$

the Hessian is:

$$H = \begin{bmatrix} \frac{1}{5} & 0 \\ 0 & 2 \end{bmatrix}$$

When Gradient Points in Wrong Direction

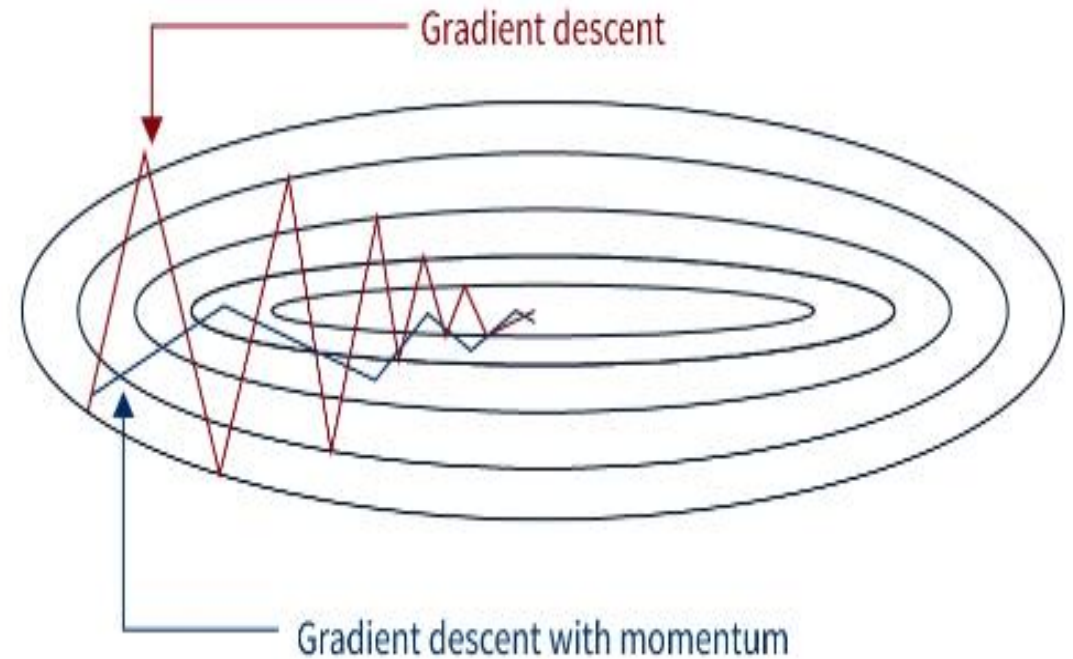
- The condition number of the Hessian matrix is defined as the ratio of the largest eigenvalue to the smallest eigenvalue:
- Condition Number = $\lambda_{\max}/\lambda_{\min}$
- Condition Number = **$2/1/5 = 10$**
- A high condition number (much greater than 1) indicates that the Hessian has a significant difference between its maximum and minimum eigenvalues.
- This leads to **highly elliptical contours**, as the steepest descent direction is much more pronounced in one direction than the other.

When Gradient Points in Wrong Direction

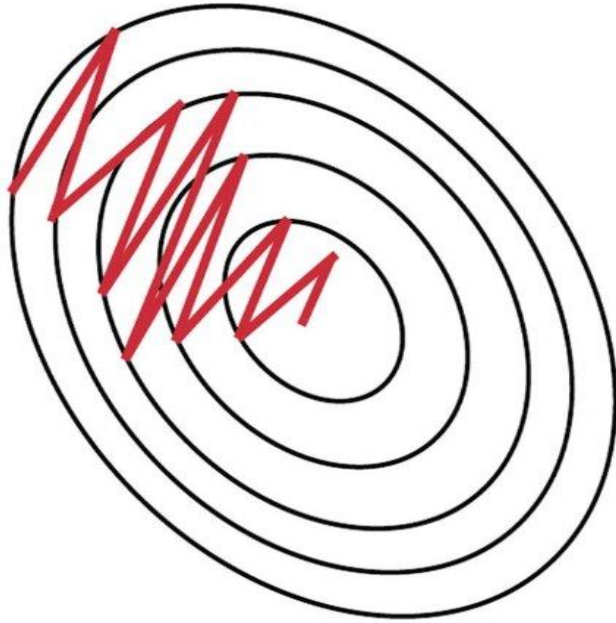
- if the condition number is close to 1 (indicating that the Hessian is well-conditioned),
- the contours will be more circular, which implies that the function is easier to optimize, as all directions have similar steepness.

Momentum-Based Optimization

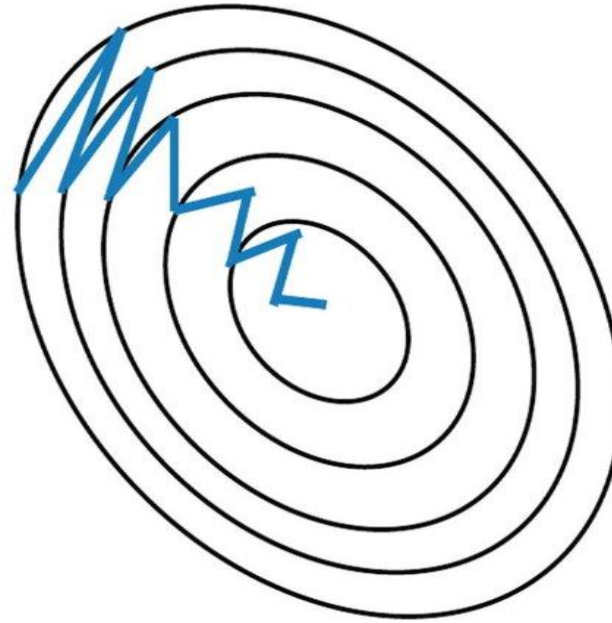
- Momentum-based optimization is a powerful technique that enhances the basic gradient descent algorithm by incorporating **past gradient information**.
- This results in faster convergence and improved stability, particularly in challenging optimization landscapes.



Momentum-Based Optimization



Stochastic Gradient
Descent **without**
Momentum



Stochastic Gradient
Descent **with**
Momentum

Momentum-Based Optimization

- Momentum-based optimization is a technique used in gradient descent algorithms to accelerate convergence, particularly in scenarios with noisy gradients or when dealing with ravines in the loss landscape.
- In a ravine, the loss function may drop sharply in one direction while being relatively flat in the perpendicular direction. This creates a "valley" effect
- The basic idea is to incorporate a "**momentum**" term that **accumulates past gradients**, allowing the optimizer to gain speed in relevant directions and dampen oscillations in others.

Momentum-Based Optimization

- **how a ball rolls down a hilly surface.**
- Driven by gravity, the ball eventually settles into a minimum on the surface, but for some reason, it doesn't suffer from the wild fluctuations and divergences that happen during gradient descent.
- there are two major components that determine how a ball rolls down an error surface.
- The first, is **SGD as the gradient, refer to as acceleration.**
- But acceleration does not determine the ball's movements.
- Instead, its motion is more directly determined **by its velocity.**
- Acceleration only indirectly changes ball's position by modifying its velocity.

Momentum-Based Optimization

- **Velocity-driven motion is desirable** because it counteracts the effects of a wildly fluctuating gradient (not stable gradient) by smoothing the ball's trajectory over its history.
- Velocity serves as a form of memory, and this allows us to more effectively **accumulate movement** in the **direction of the minimum** while **canceling out oscillating accelerations in orthogonal directions**.
- every update is computed by combining the update in the last iteration with the current gradient.

Momentum-Based Optimization

- Momentum introduces a velocity term that accumulates the **past gradients** to smooth out updates.
- This helps to navigate along the relevant directions and dampen oscillations.
- **Update Rule:**
- **Velocity Update**
- $v_t = \beta v_{t-1} + (1 - \beta) \nabla j(\theta)$
- Parameter Update
- $\theta = \theta - \alpha v_t$
- β is the momentum factor (usually between 0.5 and 0.9), α is the learning rate
- $J(\theta)$ is the loss function.
- It helps accelerate gradients in the right direction, leading to faster convergence.

Momentum-Based Optimization

- $j(\theta) = \theta^2$
- The gradient of this function is:
- $\nabla J(\theta) = 2\theta$

Parameters

- Initial parameter: $\theta = 10$
- Learning rate (α): 0.1
- Momentum factor (β): 0.9
- Initial velocity (v): 0

Momentum-Based Optimization

Iteration 1

1. Compute Gradient:

$$\nabla J(10) = 2 \times 10 = 20$$

2. Update Velocity:

$$v = \beta \cdot v + (1 - \beta) \cdot \nabla J(10) = 0.9 \cdot 0 + 0.1 \cdot 20 = 2$$

3. Update Parameter:

$$\theta = \theta - \alpha \cdot v = 10 - 0.1 \cdot 2 = 10 - 0.2 = 9.8$$

Momentum-Based Optimization

Iteration 2

1. Compute Gradient:

$$\nabla J(9.8) = 2 \times 9.8 = 19.6$$

2. Update Velocity:

$$v = 0.9 \cdot 2 + 0.1 \cdot 19.6 = 1.8 + 1.96 = 3.76$$

3. Update Parameter:

$$\theta = 9.8 - 0.1 \cdot 3.76 = 9.8 - 0.376 = 9.424$$

Momentum-Based Optimization

Iteration 3

1. Compute Gradient:

$$\nabla J(9.424) = 2 \times 9.424 = 18.848$$

2. Update Velocity:

$$v = 0.9 \cdot 3.76 + 0.1 \cdot 18.848 = 3.384 + 1.8848 = 5.2688$$

3. Update Parameter:

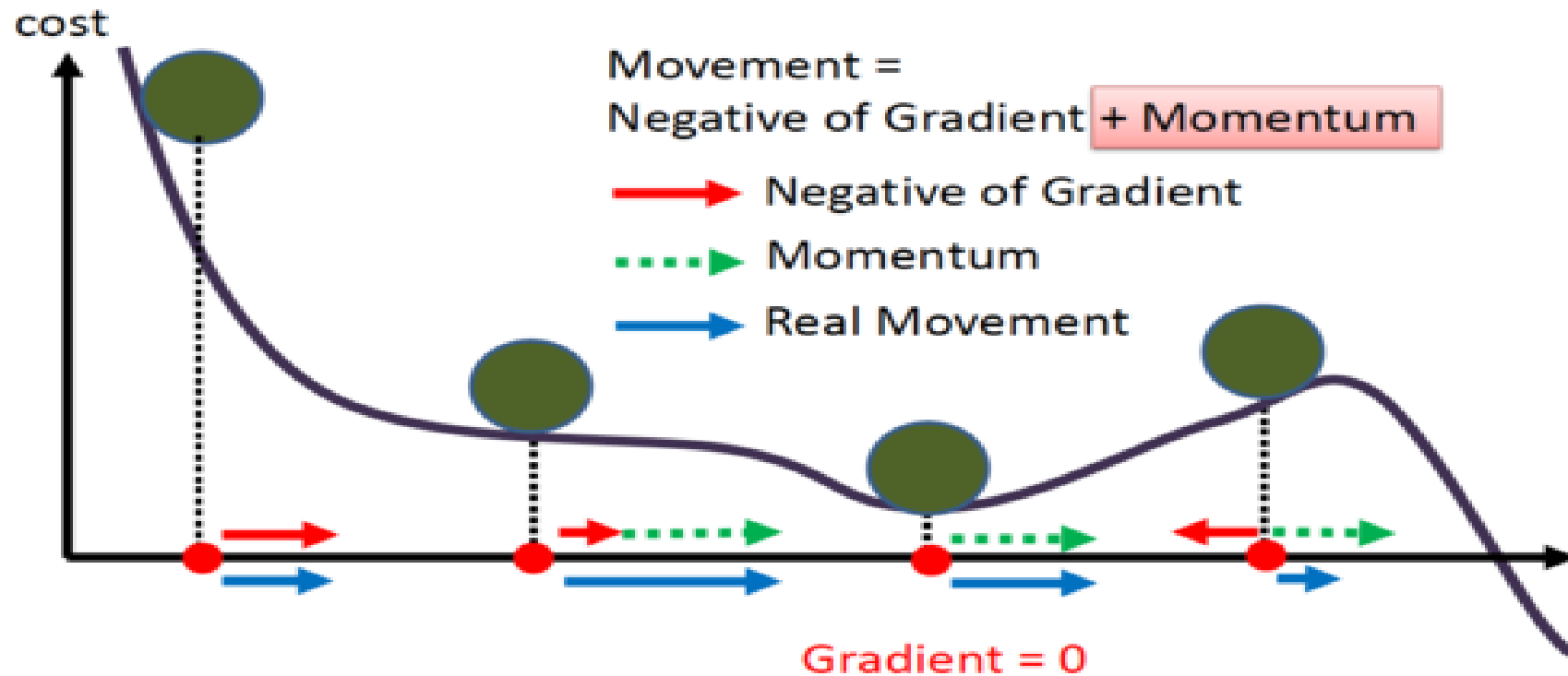
$$\theta = 9.424 - 0.1 \cdot 5.2688 = 9.424 - 0.52688 \approx 8.89712$$

- Iteration 1: $\theta \approx 9.8$
- Iteration 2: $\theta \approx 9.424$
- Iteration 3: $\theta \approx 8.89712$

Momentum-Based Optimization

- The momentum term accumulates past gradients, allowing the updates to incorporate information from previous steps.
- This can help navigate along the optimization landscape more effectively.
- The updates to θ decrease in size as it approaches the minimum, demonstrating how momentum can help smooth out the path toward convergence while avoiding oscillation
- Momentum-based gradient descent can significantly speed up convergence, particularly in high-dimensional spaces
- or when the loss surface has a lot of curvature.

Gradient descent with momentum

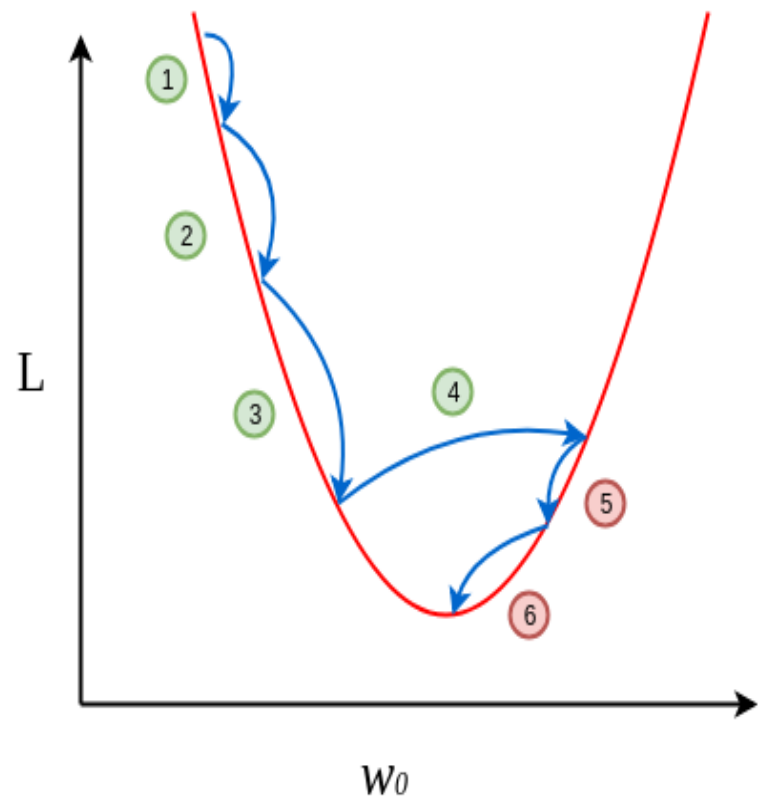


Gradient descent with momentum

- The intuition behind this is if we are repeatedly asked to go in a particular direction, we can take bigger steps towards that direction.
- The weighted average of all the previous gradients is added to our equation, and it acts as momentum to our step.
- As we start to descend, the momentum increases, and even at gentle slopes where the gradient is minimal, the actual movement is large due to the added momentum

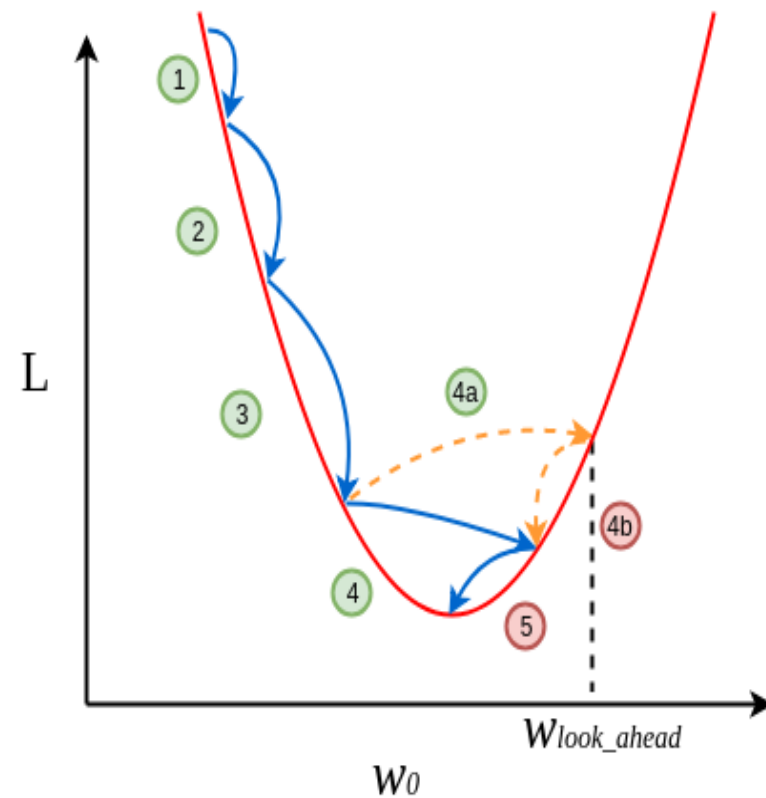
Gradient descent with momentum

- **But this added momentum causes a different type of problem.**
- We actually cross the minimum point and have to take a U-turn to get to the minimum point.
- Momentum-based gradient descent oscillates around the minimum point,
- and we have to take a lot of U-turns to reach the desired point.
- Despite these oscillations, momentum-based gradient descent is faster than conventional gradient descent.
- To reduce these oscillations, we can use **Nesterov Accelerated Gradient**



(a) Momentum-Based Gradient Descent

$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$



(b) Nesterov Accelerated Gradient Descent

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

- the updates of NAG are similar to that of the momentum-based gradient for the first three steps because the gradient at that point and the look-ahead point are positive.
- **But at step 4**, the gradient of the look-ahead point is negative.
- In NAG, the first partial update 4a will be used to go to the look-ahead point and then the gradient will be calculated at that point without updating the parameters.
- Since the gradient at step 4b is negative, the overall update will be smaller than the momentum-based gradient descent.

- We can see in the above example that the momentum-based gradient descent takes six steps to reach the minimum point, while NAG takes only five steps.
- This looking ahead helps NAG to converge to the minimum points in fewer steps and reduce the chances of overshooting.

- momentum-based gradient, the steps become larger and larger due to the accumulated momentum, and then we overshoot at the 4th step.
- We then have to take steps in the opposite direction to reach the minimum point.
- However, the update in NAG happens in two steps.
- First, a partial step to reach the look-ahead point,
- and then the final update.
- We calculate the gradient at the look-ahead point and then use it to calculate the final update
- . If the gradient at the look-ahead point is negative, our final update will be smaller than that of a regular momentum-based gradient.

Nesterov Accelerated Gradient Descent (NAG)

- NAG improves upon the standard momentum method by providing a more accurate estimate of the future position of the parameters, which can lead to faster convergence.
- The key idea behind NAG is to look ahead of the current position of the parameters in the direction of the momentum.
- This approach can provide a more accurate gradient update because it anticipates where the parameters will be in the next step,
- rather than just considering the current position

Nesterov Accelerated Gradient Descent (NAG)

- Here's a step-by-step explanation of how NAG operates:
- **Compute the Lookahead Position:**
 - Before computing the gradient, move the parameters in the direction of the current momentum. This is called the "lookahead" step.
- **Compute the Gradient:**
 - Calculate the gradient of the loss function at this lookahead position.
- **Update the Momentum:**
 - Update the momentum using the newly computed gradient.
- **Update the Parameters:**
 - Finally, update the parameters using the updated momentum.

Nesterov Accelerated Gradient Descent (NAG)

- Let θ be the parameters of the model, and let v be the momentum term. Here's how you can update your parameters using NAG:
- 1. Initialize:
 - θ_o (Initial Parameter)
 - $v_o = 0$ (initial momentum)
 - Learning rate η
 - Momentum coefficient μ
- 2. **Update Rule:**
 - For each iteration t :
 - **Compute the lookahead position:**

$$\tilde{\theta}_t = \theta_{t-1} + \mu \cdot v_{t-1}$$

Nesterov Accelerated Gradient Descent (NAG)

Compute the lookahead position:

$$\tilde{\theta}_t = \theta_{t-1} + \mu \cdot v_{t-1}$$

- Compute the gradient at the lookahead position:

$$g_t = \nabla_{\tilde{\theta}_t} L(\tilde{\theta}_t)$$

where L is the loss function.

- Update the momentum:

$$v_t = \mu \cdot v_{t-1} - \eta \cdot g_t$$

- Update the parameters:

$$\theta_t = \theta_{t-1} + v_t$$

Nesterov Accelerated Gradient Descent (NAG)

Function

Consider the function:

$$J(x, y) = x^2 + y^2$$

Initial Setup

- Initial Point: $(x, y) = (1, 1)$
- Learning Rate (α): 0.1
- Momentum Term (μ): 0.9

Nesterov Accelerated Gradient Descent (NAG)

Iteration 1

1. Lookahead Position:

$$\theta_{\text{lookahead}} = (1, 1) - 0.9 \cdot 0 = (1, 1)$$

2. Compute Gradient:

$$g_t = \nabla J(1, 1) = (2 \cdot 1, 2 \cdot 1) = (2, 2)$$

3. Update Momentum:

$$v = 0.9 \cdot 0 - 0.1 \cdot (2, 2) = (0, 0) - (0.2, 0.2) = (-0.2, -0.2)$$

4. Update Parameters:

$$\theta = (1, 1) + (-0.2, -0.2) = (0.8, 0.8)$$

Nesterov Accelerated Gradient Descent (NAG)

Iteration 2

1. Lookahead Position:

$$\theta_{\text{lookahead}} = (0.8, 0.8) - 0.9 \cdot (-0.2, -0.2) = (0.8 + 0.18, 0.8 + 0.18) = (0.98, 0.98)$$

2. Compute Gradient:

$$g_t = \nabla J(0.98, 0.98) = (2 \cdot 0.98, 2 \cdot 0.98) = (1.96, 1.96)$$

3. Update Momentum:

$$v = 0.9 \cdot (-0.2, -0.2) - 0.1 \cdot (1.96, 1.96) = (-0.18, -0.18) - (0.196, 0.196) = (-0.376, -0.376)$$

4. Update Parameters:

$$\theta = (0.8, 0.8) + (-0.376, -0.376) = (0.424, 0.424)$$

Nesterov Accelerated Gradient Descent (NAG)

- Iteration 1 Result:
- (0.8 , 0.8)
- Iteration 2 Result:
- (0.424 , 0.424)
- Nesterov Accelerated Gradient combines the benefits of momentum with a lookahead approach, leading to more efficient optimization.
- Its ability to predict the gradient direction based on anticipated future positions makes it a powerful method for training machine learning models, particularly in deep learning

Nesterov Accelerated Gradient Descent (NAG)

- **Advantages of NAG**
- **Faster Convergence:**
 - NAG often converges faster than standard gradient descent or vanilla momentum methods because it provides a better estimate of where the parameters will be in the next step.
- **Improved Stability:**
 - By incorporating a lookahead, NAG can reduce oscillations and provide a smoother convergence path, especially in scenarios where the standard momentum method might struggle.
- **More Accurate Updates:**
 - The lookahead step helps in making more informed parameter updates, which can be particularly beneficial in complex, high-dimensional optimization problems.

Nesterov Accelerated Gradient Descent (NAG)

- **When to Use NAG**
- Nesterov Accelerated Gradient Descent is useful in various scenarios:
- **Large-scale Machine Learning:** It's particularly effective for large-scale problems where faster convergence can save significant computational resources.
- **Deep Learning:** In deep neural networks, where training involves many layers and parameters, NAG can help in speeding up convergence and achieving better performance.
- **Non-Convex Optimization:** While not a cure-all for non-convex problems, NAG can still provide better performance than basic gradient descent in many cases.

AdaGrad (Adaptive Gradient Algorithm)

- **AdaGrad (Adaptive Gradient Algorithm)** is an optimization algorithm designed to adaptively **adjust the learning rate** for each parameter based on the historical gradients.
- This allows AdaGrad **to handle sparse data and gradients more effectively** than standard gradient descent.
- AdaGrad adjusts the learning rate for each parameter θ_i based on the historical squared gradients.
- The core idea is **to scale the learning rate** inversely proportional to the square root of the sum of past squared gradients for each parameter.
- This means that parameters **with large gradients will have smaller learning rates**,
- while those with smaller gradients will have larger learning rates

AdaGrad (Adaptive Gradient Algorithm)

1. Initialize:

- Parameters θ (initial values)
- Gradient accumulator G (initialized to zeros for each parameter)
- Learning rate η (often a small value, such as 0.01)

2. Update Rule:

For each parameter θ_i and each iteration t :

- **Compute the Gradient:**

$$g_{i,t} = \nabla_{\theta_i} L(\theta_t)$$

where L is the loss function.

AdaGrad (Adaptive Gradient Algorithm)

- Update the Accumulator:

$$G_{i,t} = G_{i,t-1} + g_{i,t}^2$$

Here, $G_{i,t}$ accumulates the squared gradients over time.

- Update the Parameters:

$$\theta_{i,t} = \theta_{i,t-1} - \frac{\eta}{\sqrt{G_{i,t} + \epsilon}} g_{i,t}$$

where ϵ is a small constant (e.g., $1e-8$) added for numerical stability to avoid division by zero.

Advantages of AdaGrad

- **Adaptive Learning Rates:**
 - AdaGrad automatically adapts the learning rates for each parameter based on their historical gradient information, making it suitable for problems with sparse features (e.g., text data).
- **Effective for Sparse Data:**
 - Since parameters with infrequent updates receive larger learning rates, AdaGrad can be particularly effective for sparse datasets, where some features might only be seen a few times.
- **Reduces Manual Tuning:**
 - The adaptive nature of AdaGrad reduces the need for manual tuning of the learning rate, which can simplify the optimization process.

AdaGrad (Adaptive Gradient Algorithm)

- **Disadvantages of AdaGrad**

- **Learning Rate Decay:**

- One major drawback is that the learning rate accumulates over time, leading to a progressive reduction in learning rates.
- This can cause the algorithm to make very small updates in later stages of training, potentially causing slow convergence or premature stopping.

- **No Momentum:**

- AdaGrad does not incorporate momentum, which can be beneficial for escaping local minima and accelerating convergence in certain scenarios.

- **Accumulation of Squared Gradients:**

- The accumulation of squared gradients can lead to very small learning rates in some cases, making it less effective for problems requiring large or consistent updates.

RMSProp—Exponentially Weighted Moving Average of Gradients

- RMSProp (**Root Mean Square Propagation**) is an adaptive learning rate optimization algorithm designed to address some of the shortcomings of AdaGrad,
- particularly the issue of **rapidly diminishing learning rates**.
- RMSProp maintains a moving average of the squared gradients and uses this to adjust the learning rate for each parameter, which helps to stabilize and accelerate training.
- While AdaGrad works well for simple convex functions,
- it isn't designed to navigate the complex error surfaces of deep networks.
- Flat regions may **force AdaGrad to decrease the learning rate before it reaches a minimum**.
- The conclusion is that simply using a naive accumulation of gradients isn't sufficient

RMSProp

- RMSProp modifies the learning rate of each parameter based on the recent magnitude of its gradients.
- Instead of accumulating all past squared gradients as in AdaGrad, RMSProp uses **an exponentially decaying average**,
- which helps in preventing the learning rate from becoming too small.

1. Initialize:

- Parameters θ (initial values)
- Accumulator $E[g^2]$ (initialized to zeros for each parameter)
- Learning rate η (a fixed value, such as 0.001)
- Decay factor ρ (usually between 0.9 and 0.99)
- Small constant ϵ (for numerical stability, e.g., $1e-8$)

RMSProp

2. Update Rule:

For each parameter θ_i and each iteration t :

- **Compute the Gradient:**

$$g_{i,t} = \nabla_{\theta_i} L(\theta_t)$$

where L is the loss function.

- **Update the Accumulator:**

$$E[g^2]_{i,t} = \rho \cdot E[g^2]_{i,t-1} + (1 - \rho) \cdot g_{i,t}^2$$

Here, $E[g^2]_{i,t}$ represents the exponentially weighted moving average of the squared gradients.

- **Update the Parameters:**

$$\theta_{i,t} = \theta_{i,t-1} - \frac{\eta}{\sqrt{E[g^2]_{i,t} + \epsilon}} g_{i,t}$$

The learning rate is scaled by the inverse of the square root of the moving average of squared gradients.

RMSProp -Advantages

- **Stabilized Learning Rates:**

- By using an exponentially decaying average of squared gradients, RMSProp prevents the learning rate from diminishing too quickly, allowing for more stable and consistent updates.

- **Effective in Practice:**

- like training deep neural networks.

- **Handles Sparse Gradients:**

- Like AdaGrad, RMSProp can handle sparse gradients effectively, which is useful in scenarios with high-dimensional data.

- parameters associated with frequently occurring features can have a smaller learning rate, while those associated with sparse features can maintain a larger learning rate.

- This adaptability helps ensure that all parameters are updated effectively, even when gradients are sparse.

RMSProp -Advantages

- **Disadvantages of RMSProp**

- **No Momentum:**

- RMSProp does not include momentum, which can be beneficial for accelerating convergence and escaping local minima.
- However, this limitation can be addressed with algorithms like Adam, which combines RMSProp with momentum.

- **Choice of Hyperparameter**

- The performance of RMSProp can be sensitive to the choice of hyperparameters like the decay factor ρ and the learning rate η .
- Fine-tuning these parameters might be necessary for optimal performance.

Adam—Combining Momentum and RMSProp

- Adam (short for Adaptive Moment Estimation)
- Adam as a variant combination of RMSProp and momentum.
- It adapts learning rates based on both the first and second moments of the gradients, leading to faster convergence and improved performance across a variety of machine learning tasks.
- Its robustness and effectiveness make it a go-to choice for many deep learning applications.

Adam—Combining Momentum and RMSProp

- **Initialization:**
- Parameters θ (initial values)
- Gradient moments m and v (initialized to zeros for each parameter)
- Learning rate η (a fixed value, such as 0.001)
- Exponential decay rates β_1 and β_2 (typically set to 0.9 and 0.999)
- Small constant ϵ (for numerical stability, e.g., $1e-8$)

Adam—Combining Momentum and RMSProp

- **Update Rule:**
- For each parameter θ_i and each iteration t :
- **Compute the Gradient:**
- $g_{i,t} = \nabla_{\theta_i} L(\theta_t)$, Where L is the loss function.

- Update the First Moment Estimate (Momentum):

$$m_{i,t} = \beta_1 \cdot m_{i,t-1} + (1 - \beta_1) \cdot g_{i,t}$$

Here, $m_{i,t}$ is an exponentially weighted moving average of the gradients.

- Update the Second Moment Estimate (RMSProp Component):

$$v_{i,t} = \beta_2 \cdot v_{i,t-1} + (1 - \beta_2) \cdot g_{i,t}^2$$

Here, $v_{i,t}$ is an exponentially weighted moving average of the squared gradients.

Adam—Combining Momentum and RMSProp

- **Bias Correction:** The estimates $m_{i,t}$ and $v_{i,t}$ are biased towards zero, especially at the beginning of training. To correct for this bias:

$$\hat{m}_{i,t} = \frac{m_{i,t}}{1 - \beta_1^t}$$

$$\hat{v}_{i,t} = \frac{v_{i,t}}{1 - \beta_2^t}$$

- **Update the Parameters:**

$$\theta_{i,t} = \theta_{i,t-1} - \frac{\eta \cdot \hat{m}_{i,t}}{\sqrt{\hat{v}_{i,t} + \epsilon}}$$

Adam—Advantages

- **Adaptive Learning Rates:**
 - Adam adjusts the learning rate for each parameter individually based on the historical gradient information, making it effective for a wide range of optimization problems.
- **Combines Momentum and RMSProp:**
 - By incorporating both momentum (to accelerate gradients) and RMSProp (to adapt learning rates), Adam benefits from the strengths of both approaches, leading to faster convergence and improved performance.
- **Bias Correction:**
 - The bias correction step helps to counteract the initial bias towards zero, providing more accurate updates in the early stages of training.
- **Robust to Hyperparameters:**
 - Adam is generally more robust to hyperparameter settings compared to other optimizers, making it easier to use with default settings.

Adam

- **Disadvantages of Adam**
- **Parameter Sensitivity:**
 - While Adam is robust, it can still be sensitive to the choice of hyperparameters, especially the learning rate. Fine-tuning may be required for optimal performance.
- **Computational Overhead:**
 - Adam requires additional memory to store the moment estimates and can be slightly more computationally intensive compared to simpler algorithms like standard SGD.

Eigen Values

- **We want “eigenvectors” x that don’t change direction when you multiply by A .**

A good model comes from the powers $A, A^2, A^3, \dots$ of a matrix. Suppose you need the hundredth power A^{100} . Its columns are very close to the *eigenvector* $(.6, .4)$:

$$A, A^2, A^3 = \begin{bmatrix} .8 & .3 \\ .2 & .7 \end{bmatrix} \quad \begin{bmatrix} .70 & .45 \\ .30 & .55 \end{bmatrix} \quad \begin{bmatrix} .650 & .525 \\ .350 & .475 \end{bmatrix} \quad A^{100} \approx \begin{bmatrix} .6000 & .6000 \\ .4000 & .4000 \end{bmatrix}$$

Eigen Values

- Almost all vectors change direction, when they are multiplied by A . ***Certain exceptional vectors x are in the same direction as Ax .***
- ***Those are the “eigenvectors”.***
- **The basic equation is $Ax = \lambda x$. The number λ is an eigenvalue of A .**
- We may find $\lambda = 2$ or $1/2$ or -1 or 1 .
- The eigenvalue λ could be zero!
- Then $Ax = 0x$ means that this eigenvector x is in the nullspace.
- If A is the identity matrix, every vector has $Ax = x$. All vectors are eigenvectors of I .
- All eigenvalues “lambda” are $\lambda = 1$.

Example 1 The matrix A has two eigenvalues $\lambda = 1$ and $\lambda = 1/2$. Look at $\det(A - \lambda I)$:

$$A = \begin{bmatrix} .8 & .3 \\ .2 & .7 \end{bmatrix} \quad \det \begin{bmatrix} .8 - \lambda & .3 \\ .2 & .7 - \lambda \end{bmatrix} = \lambda^2 - \frac{3}{2}\lambda + \frac{1}{2} = (\lambda - 1) \left(\lambda - \frac{1}{2} \right).$$

eigenvectors $\mathbf{x}_1$ and $\mathbf{x}_2$ are in the nullspaces of $A - I$ and $A - \frac{1}{2}I$.

$(A - I)\mathbf{x}_1 = 0$ is $A\mathbf{x}_1 = \mathbf{x}_1$ and the first eigenvector is $(.6, .4)$.

$(A - \frac{1}{2}I)\mathbf{x}_2 = 0$ is $A\mathbf{x}_2 = \frac{1}{2}\mathbf{x}_2$ and the second eigenvector is $(1, -1)$:

$$\mathbf{x}_1 = \begin{bmatrix} .6 \\ .4 \end{bmatrix} \quad \text{and} \quad A\mathbf{x}_1 = \begin{bmatrix} .8 & .3 \\ .2 & .7 \end{bmatrix} \begin{bmatrix} .6 \\ .4 \end{bmatrix} = \mathbf{x}_1 \quad (A\mathbf{x} = \mathbf{x} \text{ means that } \lambda_1 = 1)$$

$$\mathbf{x}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad \text{and} \quad A\mathbf{x}_2 = \begin{bmatrix} .8 & .3 \\ .2 & .7 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} .5 \\ -.5 \end{bmatrix} \quad (\text{this is } \frac{1}{2} \mathbf{x}_2 \text{ so } \lambda_2 = \frac{1}{2}).$$

If $\mathbf{x}_1$ is multiplied again by A , we still get $\mathbf{x}_1$. Every power of A will give $A^n \mathbf{x}_1 = \mathbf{x}_1$.
Multiplying $\mathbf{x}_2$ by A gave $\frac{1}{2}\mathbf{x}_2$, and if we multiply again we get $(\frac{1}{2})^2$ times $\mathbf{x}_2$.

When A is squared, the eigenvectors stay the same. The eigenvalues are squared.

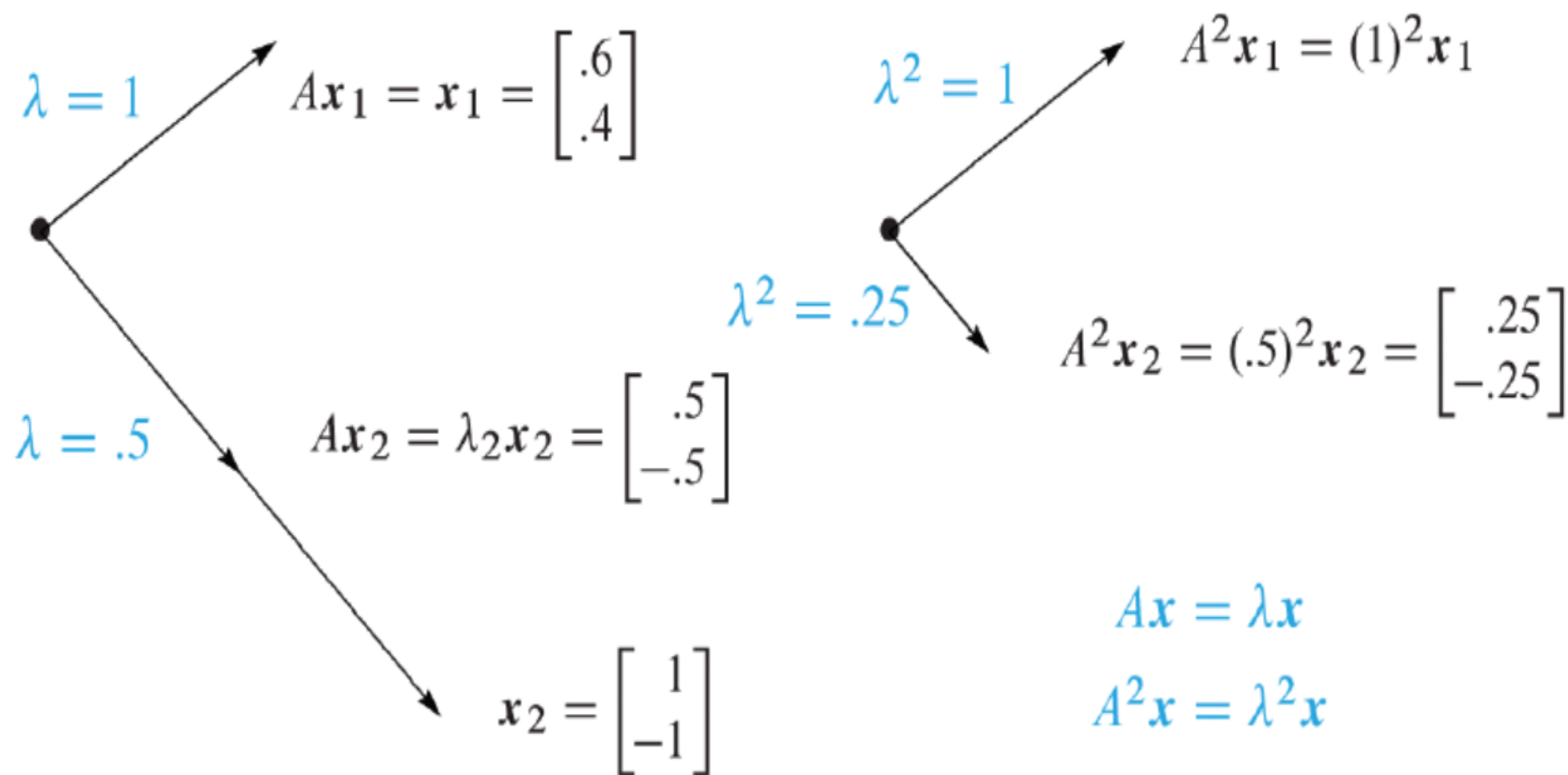


Figure 6.1: The eigenvectors keep their directions. $A^2\mathbf{x} = \lambda^2\mathbf{x}$ with $\lambda^2 = 1^2$ and $(.5)^2$.

Dimensionality Reduction

- In high dimensions the center of the space is devoid of points, with most of the points being scattered along the surface of the space or in the corners.
- As a consequence high-dimensional data can cause problems for data mining and analysis,
- although in some cases high-dimensionality can help, for example, for nonlinear classification.
- it is important to check whether the dimensionality can be reduced while preserving the essential properties of the full data matrix.
- This can aid data visualization as well as data mining

Let the data **D** consist of n points over d attributes, it is an $n \times d$ matrix,

$$\mathbf{D} = \begin{pmatrix} & \mathbf{X}_1 & \mathbf{X}_2 & \cdots & \mathbf{X}_d \\ \mathbf{x}_1 & x_{11} & x_{12} & \cdots & x_{1d} \\ \mathbf{x}_2 & x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_n & x_{n1} & x_{n2} & \cdots & x_{nd} \end{pmatrix}$$

- $x_i = (x_{i1}, x_{i2}, x_{i3}, \dots, x_{id})^t$ is a vector in the d-dimensional vector space spanned by the d standard basis vectors $e_1, e_2, \dots, e_d$, where e_i corresponds to the ith attribute X_i .
- basis vectors are pairwise orthogonal, $e_i^t \cdot e_j = 0$
- and have unit length $\|e_i\| = 1$

- given any other set of d orthonormal vectors $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_d$,
with $\mathbf{u}_i^t \cdot \mathbf{u}_j = 0$, $\|\mathbf{u}_i\| = 1$,
- we can re-express each point $\mathbf{x}$ as the linear combination,
- $\mathbf{x} = a_1\mathbf{u}_1 + a_2\mathbf{u}_2 + \dots + a_d\mathbf{u}_d$
- $\mathbf{a} = (a_1, a_2, \dots, a_d)^t$ represents the coordinates of $\mathbf{x}$ in the new basis.
- above linear combination can also be expressed as a matrix multiplication:
- $\mathbf{x} = \mathbf{U}\mathbf{a}$
- where $\mathbf{U}$ is $d \times d$ matrix, whose i th column comprises the i th basis vector $\mathbf{u}_i$

$$\mathbf{U} = \begin{pmatrix} | & | & \cdots & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_d \\ | & | & & | \end{pmatrix}$$

- The matrix $\mathbf{U}$ is an *orthogonal* matrix, whose columns, the basis vectors, are *orthonormal*, that is, they are pairwise orthogonal and have unit length

$$\mathbf{u}_i^T \mathbf{u}_j = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Because $\mathbf{U}$ is orthogonal, this means that its inverse equals its transpose:

$$\mathbf{U}^{-1} = \mathbf{U}^T$$

which implies that $\mathbf{U}^T \mathbf{U} = \mathbf{I}$, where $\mathbf{I}$ is the $d \times d$ identity matrix.

- $\mathbf{x} = \mathbf{U}\mathbf{a}$
- Multiplying Eq. on both sides by $\mathbf{U}^T$ yields the expression for computing **the coordinates of $\mathbf{x}$ in the new basis**

$$\mathbf{U}^T \mathbf{x} = \mathbf{U}^T \mathbf{U} \mathbf{a}$$

$$\mathbf{a} = \mathbf{U}^T \mathbf{x}$$

Example

the centered Iris dataset, with $n = 150$ points, in the $d = 3$ dimensional space comprising the sepal length (X_1), sepal width (X_2), and petal length (X_3) attributes. space is spanned by the standard basis vectors.

$$\mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

$$\mathbf{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

$$\mathbf{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

the same points in the space comprising the new basis vectors

$$\mathbf{u}_1 = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix}$$

$$\mathbf{u}_2 = \begin{pmatrix} -0.639 \\ -0.742 \\ 0.200 \end{pmatrix}$$

$$\mathbf{u}_3 = \begin{pmatrix} -0.663 \\ 0.664 \\ 0.346 \end{pmatrix}$$

Example

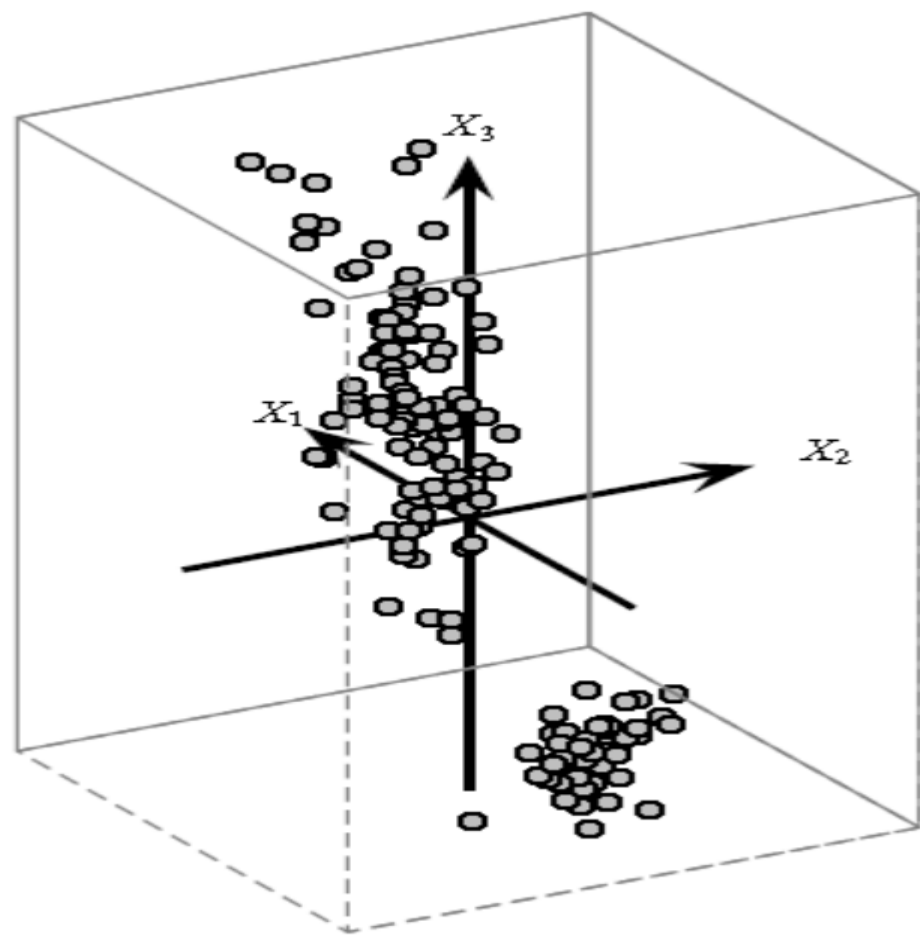
the new coordinates of the centered point $\mathbf{x} = (-0.343, -0.754, 0.241)^T$ can be computed as

$$\mathbf{a} = \mathbf{U}^T \mathbf{x} = \begin{pmatrix} -0.390 & 0.089 & -0.916 \\ -0.639 & -0.742 & 0.200 \\ -0.663 & 0.664 & 0.346 \end{pmatrix} \begin{pmatrix} -0.343 \\ -0.754 \\ 0.241 \end{pmatrix} = \begin{pmatrix} -0.154 \\ 0.828 \\ -0.190 \end{pmatrix}$$

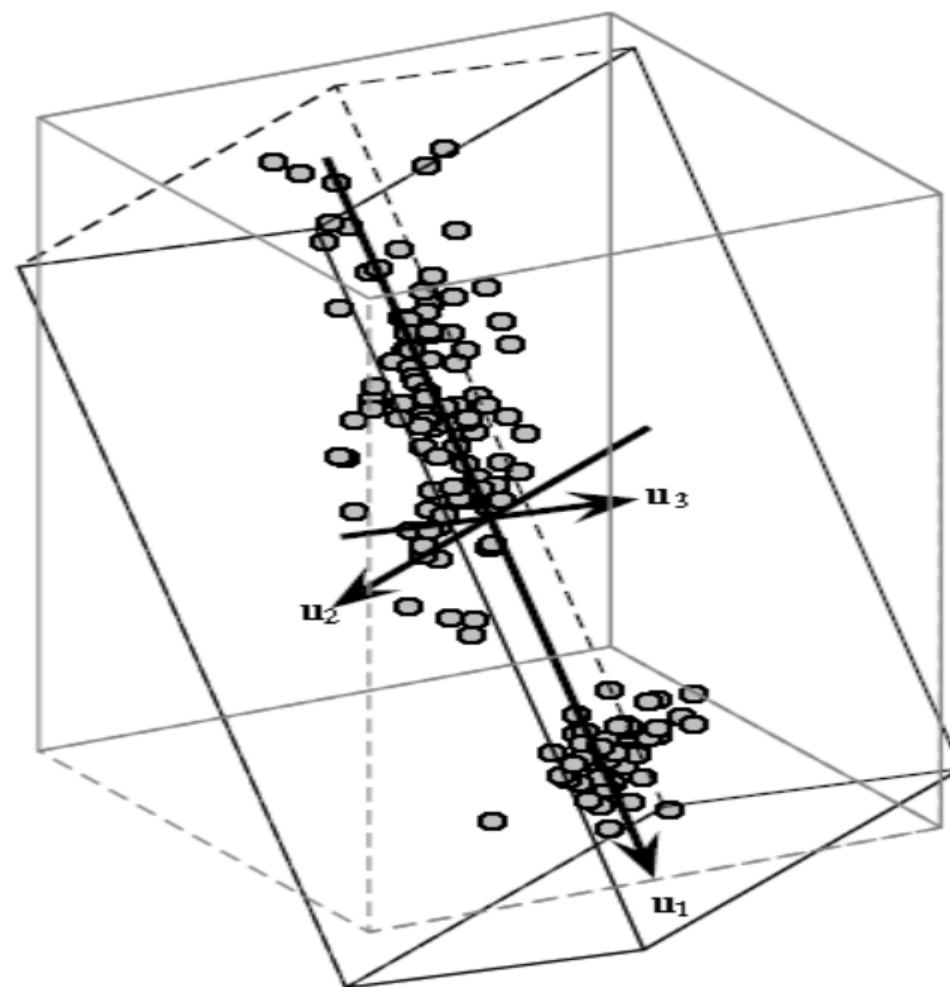
One can verify that $\mathbf{x}$ can be written as the linear combination

$$\mathbf{x} = -0.154\mathbf{u}_1 + 0.828\mathbf{u}_2 - 0.190\mathbf{u}_3$$

Example



(a) Original Basis



(b) Optimal Basis

Example

- finding the optimal r -dimensional representation of $\mathbf{D}$, with $r \ll d$.
- given a point $\mathbf{x}$, and assuming basis vectors have been sorted in decreasing order of importance, we can truncate its linear expansion to just r **terms**,

$$\mathbf{x}' = a_1 \mathbf{u}_1 + a_2 \mathbf{u}_2 + \dots + a_r \mathbf{u}_r = \sum_{i=1}^r a_i \mathbf{u}_i$$

- Here $\mathbf{x}'$ is the projection of $\mathbf{x}$ onto the first r basis vectors,
- which can be written in matrix notation as follows:

$$\mathbf{x}' = \begin{pmatrix} | & | & & | \\ \mathbf{u}_1 & \mathbf{u}_2 & \dots & \mathbf{u}_r \\ | & | & & | \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_r \end{pmatrix} = \mathbf{U}_r \mathbf{a}_r$$

- where $\mathbf{U}_r$ is the matrix comprising the first r basis vectors,
- and $\mathbf{a}_r$ is vector comprising the first r coordinates.

- Further, because $a_r = \mathbf{U}_r^T \mathbf{x}$

- projection of $\mathbf{x}$ onto $\mathbf{x}' = \mathbf{U}_r \mathbf{U}_r^T \mathbf{x} = \mathbf{P}_r \mathbf{x}$ be compactly written as

- where $\mathbf{P}_r$ is the *orthogonal projection matrix* for the subspace spanned by the first r basis vectors

$$\mathbf{P}_r = \mathbf{U}_r \mathbf{U}_r^T = \sum_{i=1}^r \mathbf{u}_i \mathbf{u}_i^T$$

projection of $\mathbf{x}$ onto the remaining dimensions comprises *error vector*

$$\boldsymbol{\epsilon} = \sum_{i=r+1}^d a_i \mathbf{u}_i = \mathbf{x} - \mathbf{x}'$$

- It is worth noting that that $\mathbf{x}'$ and ϵ are orthogonal vectors:

$$\mathbf{x}'^T \boldsymbol{\epsilon} = \sum_{i=1}^r \sum_{j=r+1}^d a_i a_j \mathbf{u}_i^T \mathbf{u}_j = 0$$

- The subspace spanned by the first r basis vectors
- $S_r = \text{span}(\mathbf{u}_1, \dots, \mathbf{u}_r)$
- and the subspace spanned by the remaining basis vectors
- $S_{d-r} = \text{span}(\mathbf{u}_{r+1}, \dots, \mathbf{u}_d)$ are *orthogonal subspaces*

Example

- centered point $\mathbf{x} = (-0.343, -0.754, 0.241)^T$ by using only the first basis vector $\mathbf{u}_1 = (-0.390, 0.089, -0.916)^T$, we have

$$\mathbf{x}' = a_1 \mathbf{u}_1 = -0.154 \mathbf{u}_1 = \begin{pmatrix} 0.060 \\ -0.014 \\ 0.141 \end{pmatrix}$$

- projection of $\mathbf{x}$ on $\mathbf{u}_1$ obtained directly from the projection matrix

$$\begin{aligned} \mathbf{P}_1 &= \mathbf{u}_1 \mathbf{u}_1^T = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix} \begin{pmatrix} -0.390 & 0.089 & -0.916 \end{pmatrix} \\ &= \begin{pmatrix} 0.152 & -0.035 & 0.357 \\ -0.035 & 0.008 & -0.082 \\ 0.357 & -0.082 & 0.839 \end{pmatrix} \end{aligned}$$

That is

$$\mathbf{x}' = \mathbf{P}_1 \mathbf{x} = \begin{pmatrix} 0.060 \\ -0.014 \\ 0.141 \end{pmatrix}$$

The error vector is given as

$$\boldsymbol{\epsilon} = a_2 \mathbf{u}_2 + a_3 \mathbf{u}_3 = \mathbf{x} - \mathbf{x}' = \begin{pmatrix} -0.40 \\ -0.74 \\ 0.10 \end{pmatrix}$$

One can verify that $\mathbf{x}'$ and $\boldsymbol{\epsilon}$ are orthogonal, i.e.,

$$\mathbf{x}'^T \boldsymbol{\epsilon} = (0.060 \quad -0.014 \quad 0.141) \begin{pmatrix} -0.40 \\ -0.74 \\ 0.10 \end{pmatrix} = 0$$

PRINCIPAL COMPONENT ANALYSIS

- **seeks a r -dimensional basis that best captures the variance** in data.
- The direction with the largest projected variance is called the **first principal component**.
- **The orthogonal direction** that captures the **second largest projected variance** is called the second principal component, and so on.
- **the direction that maximizes the variance is also the one that minimizes the mean squared error**

Best Line Approximation

- We will start with $r = 1$, that is, the one-dimensional subspace
- or line $\mathbf{u}$ that best approximates $\mathbf{D}$ in terms of the variance of projected points.
- This will lead to the general PCA technique for the best $1 \leq r \leq d$ dimensional basis for $\mathbf{D}$.
- we assume that $\mathbf{u}$ has magnitude $\|\mathbf{u}\|^2 = \mathbf{u}^t \mathbf{u} = 1$
- otherwise it is possible to keep on increasing the projected variance by simply increasing the magnitude of $\mathbf{u}$.
- We also assume that the data has been centered so that it has mean $\mu = \mathbf{0}$.

Best Line Approximation

- The projection of $\mathbf{x}_i$ on the vector $\mathbf{u}$ is given as

$$\mathbf{x}'_i = \left(\frac{\mathbf{u}^T \mathbf{x}_i}{\mathbf{u}^T \mathbf{u}} \right) \mathbf{u} = (\mathbf{u}^T \mathbf{x}_i) \mathbf{u} = a_i \mathbf{u}$$

where the scalar

$$a_i = \mathbf{u}^T \mathbf{x}_i$$

- gives the coordinate of x'_i along $\mathbf{u}$.
- Note that because the mean point is $\mu = \mathbf{0}$, its coordinate along $\mathbf{u}$ is $\mu_u = 0$

- We have to choose the direction $\mathbf{u}$ such that the variance of the projected points is maximized.
- The projected variance along $\mathbf{u}$ is given as

$$\begin{aligned}\sigma_{\mathbf{u}}^2 &= \frac{1}{n} \sum_{i=1}^n (a_i - \mu_{\mathbf{u}})^2 \\ &= \frac{1}{n} \sum_{i=1}^n (\mathbf{u}^T \mathbf{x}_i)^2 \\ &= \frac{1}{n} \sum_{i=1}^n \mathbf{u}^T (\mathbf{x}_i \mathbf{x}_i^T) \mathbf{u} \\ &= \mathbf{u}^T \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T \right) \mathbf{u} \\ &= \mathbf{u}^T \mathbf{\Sigma} \mathbf{u}\end{aligned}$$

where $\mathbf{\Sigma}$ is the covariance matrix for the centered data $\mathbf{D}$.

- **To maximize the projected variance**, we have to solve a constrained optimization problem, namely to maximize σ_u^2 subject to the constraint that $\mathbf{u}^t \mathbf{u} = 1$.
- This can be solved by introducing a **Lagrangian multiplier α** for the constraint, to obtain the unconstrained maximization problem

$$\max_{\mathbf{u}} J(\mathbf{u}) = \mathbf{u}^T \mathbf{\Sigma} \mathbf{u} - \alpha (\mathbf{u}^T \mathbf{u} - 1)$$

Setting the derivative of $J(\mathbf{u})$ with respect to $\mathbf{u}$ to the zero vector, we obtain

$$\frac{\partial}{\partial \mathbf{u}} J(\mathbf{u}) = \mathbf{0}$$

$$\frac{\partial}{\partial \mathbf{u}} (\mathbf{u}^T \mathbf{\Sigma} \mathbf{u} - \alpha (\mathbf{u}^T \mathbf{u} - 1)) = \mathbf{0}$$

$$2\mathbf{\Sigma} \mathbf{u} - 2\alpha \mathbf{u} = \mathbf{0}$$

$$\mathbf{\Sigma} \mathbf{u} = \alpha \mathbf{u}$$

- This implies that α is an eigenvalue of the covariance matrix Σ , with the associated eigenvector $\mathbf{u}$.
- Further, taking the dot product with $\mathbf{u}$ on both sides

$$\mathbf{u}^T \Sigma \mathbf{u} = \mathbf{u}^T \alpha \mathbf{u}$$

$$\sigma_{\mathbf{u}}^2 = \alpha \mathbf{u}^T \mathbf{u}$$

$$\text{or } \sigma_{\mathbf{u}}^2 = \alpha$$

- To maximize the projected variance $\sigma_{\mathbf{u}}^2$, we should thus choose the largest eigenvalue of Σ .
- the dominant eigenvector $\mathbf{u}_1$ specifies the direction of most variance, also called the *first principal component*, that is, $\mathbf{u} = \mathbf{u}_1$.
- the largest eigenvalue λ_1 specifies the projected variance, that is, σ^2
- $\mathbf{u} = \alpha = \lambda_1$.

Minimum Squared Error Approach

- direction that maximizes the projected variance is also the one that minimizes the average squared error.
- assume that the dataset **D** has been centered by subtracting the mean from each point

$$\begin{aligned}MSE(\mathbf{u}) &= \frac{1}{n} \sum_{i=1}^n \|\boldsymbol{\epsilon}_i\|^2 \\&= \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{x}'_i\|^2 \\&= \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \mathbf{x}'_i)^T (\mathbf{x}_i - \mathbf{x}'_i) \\&= \frac{1}{n} \sum_{i=1}^n \left(\|\mathbf{x}_i\|^2 - 2\mathbf{x}_i^T \mathbf{x}'_i + (\mathbf{x}'_i)^T \mathbf{x}'_i \right)\end{aligned}$$

Minimum Squared Error Approach

Noting that $\mathbf{x}'_i = (\mathbf{u}^T \mathbf{x}_i) \mathbf{u}$, we have

$$\begin{aligned} &= \frac{1}{n} \sum_{i=1}^n \left(\|\mathbf{x}_i\|^2 - 2\mathbf{x}_i^T (\mathbf{u}^T \mathbf{x}_i) \mathbf{u} + ((\mathbf{u}^T \mathbf{x}_i) \mathbf{u})^T (\mathbf{u}^T \mathbf{x}_i) \mathbf{u} \right) \\ &= \frac{1}{n} \sum_{i=1}^n \left(\|\mathbf{x}_i\|^2 - 2(\mathbf{u}^T \mathbf{x}_i)(\mathbf{x}_i^T \mathbf{u}) + (\mathbf{u}^T \mathbf{x}_i)(\mathbf{x}_i^T \mathbf{u}) \mathbf{u}^T \mathbf{u} \right) \\ &= \frac{1}{n} \sum_{i=1}^n \left(\|\mathbf{x}_i\|^2 - (\mathbf{u}^T \mathbf{x}_i)(\mathbf{x}_i^T \mathbf{u}) \right) \\ &= \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i\|^2 - \frac{1}{n} \sum_{i=1}^n \mathbf{u}^T (\mathbf{x}_i \mathbf{x}_i^T) \mathbf{u} \\ &= \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i\|^2 - \mathbf{u}^T \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T \right) \mathbf{u} \\ &= \sum_{i=1}^n \frac{\|\mathbf{x}_i\|^2}{n} - \mathbf{u}^T \mathbf{\Sigma} \mathbf{u} \end{aligned}$$

Minimum Squared Error Approach

- the total variance of the centered data (i.e., with $\mu = \mathbf{0}$)

$$var(\mathbf{D}) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{0}\|^2 = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i\|^2$$

- $$var(\mathbf{D}) = tr(\mathbf{\Sigma}) = \sum_{i=1}^d \sigma_i^2$$

$$MSE(\mathbf{u}) = var(\mathbf{D}) - \mathbf{u}^T \mathbf{\Sigma} \mathbf{u} = \sum_{i=1}^d \sigma_i^2 - \mathbf{u}^T \mathbf{\Sigma} \mathbf{u}$$

Minimum Squared Error Approach

$$MSE(\mathbf{u}_1) = var(\mathbf{D}) - \mathbf{u}_1^T \mathbf{\Sigma} \mathbf{u}_1 = var(\mathbf{D}) - \mathbf{u}_1^T \lambda_1 \mathbf{u}_1 = var(\mathbf{D}) - \lambda_1$$

- Thus, the principal component $\mathbf{u}_1$, which is the direction that maximizes the projected variance,
- is also the direction that minimizes the mean squared error.
-

Example

$$\mathbf{\Sigma} = \begin{pmatrix} 0.681 & -0.039 & 1.265 \\ -0.039 & 0.187 & -0.320 \\ 1.265 & -0.320 & 3.092 \end{pmatrix}$$

The variance values σ_i^2 for each of the original dimensions are given along the main diagonal of $\mathbf{\Sigma}$. For example, $\sigma_1^2 = 0.681$, $\sigma_2^2 = 0.187$, and $\sigma_3^2 = 3.092$. The largest eigenvalue of $\mathbf{\Sigma}$ is $\lambda_1 = 3.662$, and the corresponding dominant eigenvector is $\mathbf{u}_1 = (-0.390, 0.089, -0.916)^T$. The unit vector $\mathbf{u}_1$ thus maximizes the projected variance, which is given as $J(\mathbf{u}_1) = \alpha = \lambda_1 = 3.662$. Figure 7.2 plots the principal component $\mathbf{u}_1$. It also shows the error vectors ϵ_i , as thin gray line segments.

The total variance of the data is given as

$$var(\mathbf{D}) = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}\|^2 = \frac{1}{150} \cdot 594.04 = 3.96$$

Example

We can also directly obtain the total variance as the trace of the covariance matrix:

$$\text{var}(\mathbf{D}) = \text{tr}(\mathbf{\Sigma}) = \sigma_1^2 + \sigma_2^2 + \sigma_3^2 = 0.681 + 0.187 + 3.092 = 3.96$$

Thus, using Eq. (7.16), the minimum value of the mean squared error is given as

$$MSE(\mathbf{u}_1) = \text{var}(\mathbf{D}) - \lambda_1 = 3.96 - 3.662 = 0.298$$

Best 2-dimensional Approximation

- assume that $\mathbf{D}$ has already been centered, so that $\mu = \mathbf{0}$.
- We already computed the direction with the most variance, $\mathbf{u}_1$, which is eigenvector corresponding to the largest eigenvalue λ_1 of Σ .
- We now want to find another direction $\mathbf{v}$, which also maximizes the projected variance, but is orthogonal to $\mathbf{u}_1$.

projected variance along $\mathbf{v}$ is given as

$$\sigma_{\mathbf{v}}^2 = \mathbf{v}^T \Sigma \mathbf{v}$$

We further require that $\mathbf{v}$ be a unit vector orthogonal to $\mathbf{u}_1$, that is,

$$\mathbf{v}^T \mathbf{u}_1 = 0$$

$$\mathbf{v}^T \mathbf{v} = 1$$

Best 2-dimensional Approximation

- The optimization condition then becomes

$$\max_{\mathbf{v}} J(\mathbf{v}) = \mathbf{v}^T \mathbf{\Sigma} \mathbf{v} - \alpha (\mathbf{v}^T \mathbf{v} - 1) - \beta (\mathbf{v}^T \mathbf{u}_1 - 0)$$

- Taking the derivative of $J(\mathbf{v})$ with respect to $\mathbf{v}$, and setting it to the zero vector, gives

$$2\mathbf{\Sigma} \mathbf{v} - 2\alpha \mathbf{v} - \beta \mathbf{u}_1 = \mathbf{0}$$

If we multiply on the left by $\mathbf{u}_1^T$ we get

$$2\mathbf{u}_1^T \mathbf{\Sigma} \mathbf{v} - 2\alpha \mathbf{u}_1^T \mathbf{v} - \beta \mathbf{u}_1^T \mathbf{u}_1 = 0$$

$$2\mathbf{v}^T \mathbf{\Sigma} \mathbf{u}_1 - \beta = 0, \text{ which implies that}$$

$$\beta = 2\mathbf{v}^T \mathbf{\Sigma} \mathbf{u}_1 = 2\lambda_1 \mathbf{v}^T \mathbf{u}_1 = 0$$

Best 2-dimensional Approximation

- In the derivation above we used the fact that

$$\mathbf{u}_1^T \Sigma \mathbf{v} = \mathbf{v}^T \Sigma \mathbf{u}_1$$

- and that $\mathbf{v}$ is orthogonal to $\mathbf{u}_1$. Plugging $\beta = 0$

$$2\Sigma\mathbf{v} - 2\alpha\mathbf{v} = \mathbf{0}$$

$$\Sigma\mathbf{v} = \alpha\mathbf{v}$$

- This means that $\mathbf{v}$ is another eigenvector of Σ .
- we have $\sigma_u^2 \mathbf{v} = \alpha \mathbf{v}$.
- To maximize the variance along $\mathbf{v}$, we should choose $\alpha = \lambda_2$,
- second largest eigenvalue of Σ ,
- with the *second principal component* being given by the corresponding eigenvector, $\mathbf{v} = \mathbf{u}_2$

Total Projected Variance

- Let $\mathbf{U}_2$ be the matrix whose columns correspond to the two principal components,

- $\mathbf{a}_i = \mathbf{U}_2^t \mathbf{x}_i$

$$\mathbf{U}_2 = \begin{pmatrix} | & | \\ \mathbf{u}_1 & \mathbf{u}_2 \\ | & | \end{pmatrix}$$

- Assume that each point $\mathbf{x}_i \in \mathbb{R}^d$ in $\mathbf{D}$ has been projected to obtain its coordinates
- $\mathbf{a}_i \in \mathbb{R}^2$, yielding the new dataset $\mathbf{A}$.
- $\mathbf{D}$ is assumed to be centered, with $\mu = \mathbf{0}$, the coordinates of the projected mean are also zero because

- $\mathbf{U}_2^t \mu = \mathbf{U}_2^t \mathbf{0} = \mathbf{0}$

Total Projected Variance

$$\begin{aligned} \text{var}(\mathbf{A}) &= \frac{1}{n} \sum_{i=1}^n \|\mathbf{a}_i - \mathbf{0}\|^2 \\ &= \frac{1}{n} \sum_{i=1}^n (\mathbf{U}_2^T \mathbf{x}_i)^T (\mathbf{U}_2^T \mathbf{x}_i) \\ &= \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i^T (\mathbf{U}_2 \mathbf{U}_2^T) \mathbf{x}_i \\ &= \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i^T \mathbf{P}_2 \mathbf{x}_i \end{aligned}$$

where $\mathbf{P}_2$ is the orthogonal projection matrix

$$\mathbf{P}_2 = \mathbf{U}_2 \mathbf{U}_2^T = \mathbf{u}_1 \mathbf{u}_1^T + \mathbf{u}_2 \mathbf{u}_2^T$$

Total Projected Variance

$$\begin{aligned} \text{var}(\mathbf{A}) &= \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i^T \mathbf{P}_2 \mathbf{x}_i \\ &= \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i^T (\mathbf{u}_1 \mathbf{u}_1^T + \mathbf{u}_2 \mathbf{u}_2^T) \mathbf{x}_i \\ &= \frac{1}{n} \sum_{i=1}^n (\mathbf{u}_1^T \mathbf{x}_i)(\mathbf{x}_i^T \mathbf{u}_1) + \frac{1}{n} \sum_{i=1}^n (\mathbf{u}_2^T \mathbf{x}_i)(\mathbf{x}_i^T \mathbf{u}_2) \\ &= \mathbf{u}_1^T \mathbf{\Sigma} \mathbf{u}_1 + \mathbf{u}_2^T \mathbf{\Sigma} \mathbf{u}_2 \end{aligned}$$

Because $\mathbf{u}_1$ and $\mathbf{u}_2$ are eigenvectors of $\mathbf{\Sigma}$, we have $\mathbf{\Sigma} \mathbf{u}_1 = \lambda_1 \mathbf{u}_1$ and $\mathbf{\Sigma} \mathbf{u}_2 = \lambda_2 \mathbf{u}_2$, so that

$$\text{var}(\mathbf{A}) = \mathbf{u}_1^T \mathbf{\Sigma} \mathbf{u}_1 + \mathbf{u}_2^T \mathbf{\Sigma} \mathbf{u}_2 = \mathbf{u}_1^T \lambda_1 \mathbf{u}_1 + \mathbf{u}_2^T \lambda_2 \mathbf{u}_2 = \lambda_1 + \lambda_2$$

Thus, the sum of the eigenvalues is the total variance of the projected points, and the first two principal components maximize this variance.

Mean Squared Error

$$\begin{aligned}MSE &= \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{x}'_i\|^2 \\&= \frac{1}{n} \sum_{i=1}^n \left(\|\mathbf{x}_i\|^2 - 2\mathbf{x}_i^T \mathbf{x}'_i + (\mathbf{x}'_i)^T \mathbf{x}'_i \right), \\&= var(\mathbf{D}) + \frac{1}{n} \sum_{i=1}^n \left(-2\mathbf{x}_i^T \mathbf{P}_2 \mathbf{x}_i + (\mathbf{P}_2 \mathbf{x}_i)^T \mathbf{P}_2 \mathbf{x}_i \right),\end{aligned}$$

$$\mathbf{x}'_i = \mathbf{P}_2 \mathbf{x}_i$$

$$\begin{aligned}&= var(\mathbf{D}) - \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i^T \mathbf{P}_2 \mathbf{x}_i) \\&= var(\mathbf{D}) - var(\mathbf{A})\end{aligned}$$

$$MSE = var(\mathbf{D}) - \lambda_1 - \lambda_2$$

Example

- For the Iris dataset, the two largest eigenvalues are $\lambda_1 = 3.662$, and $\lambda_2 = 0.239$, with the corresponding eigenvectors

$$\mathbf{u}_1 = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix}$$

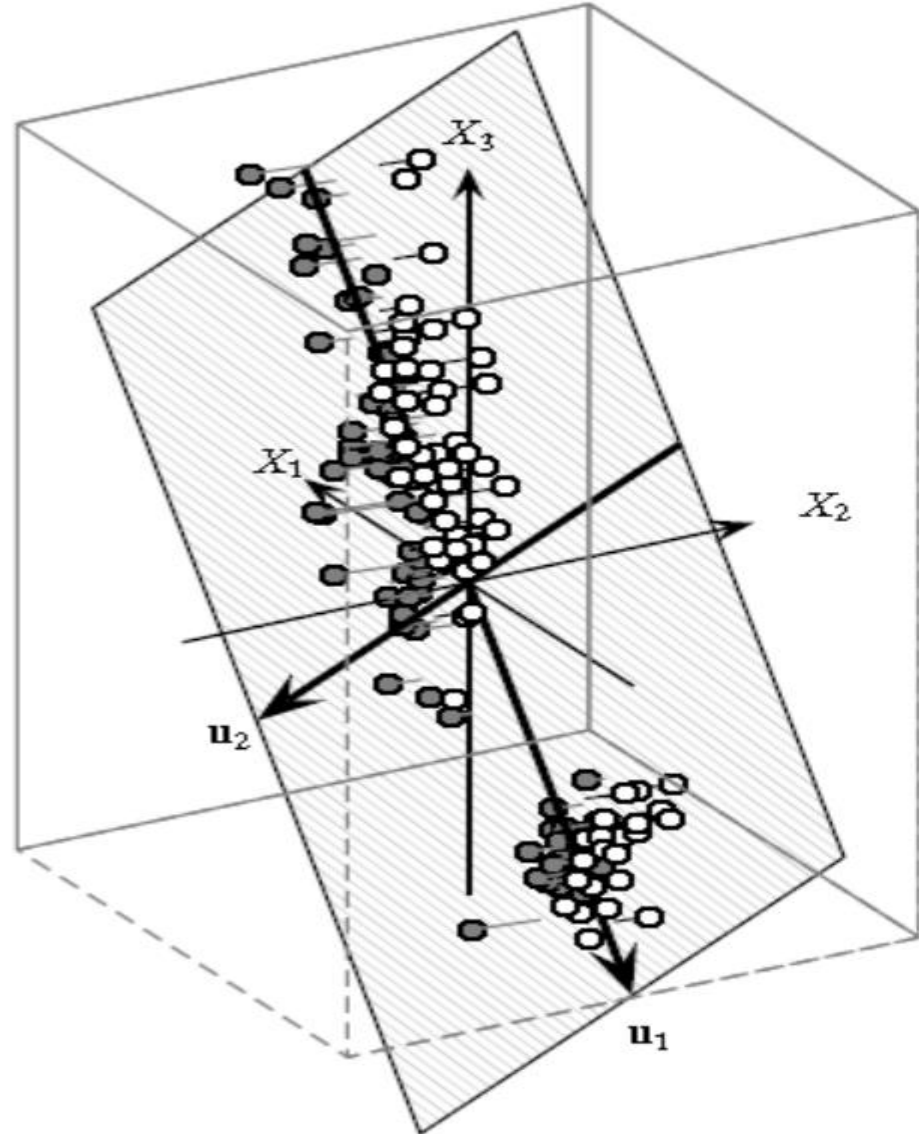
$$\mathbf{u}_2 = \begin{pmatrix} -0.639 \\ -0.742 \\ 0.200 \end{pmatrix}$$

The projection matrix is given as

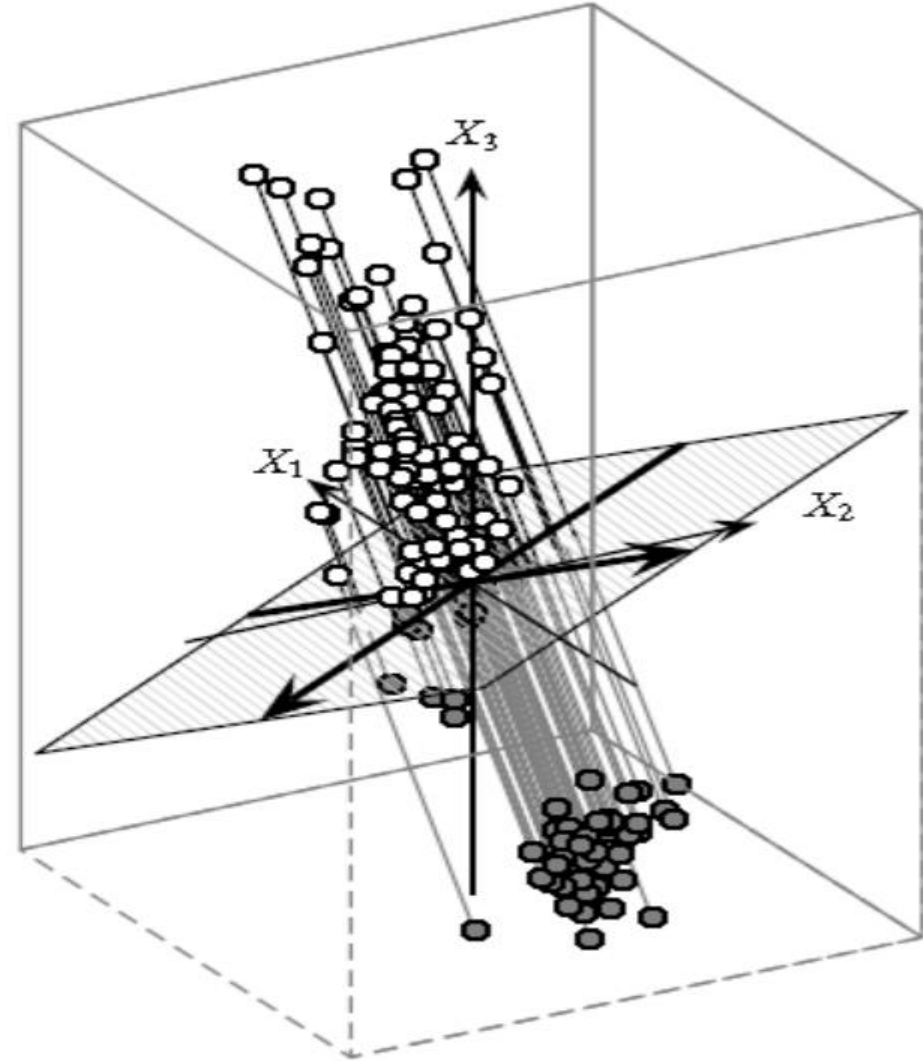
$$\begin{aligned} \mathbf{P}_2 &= \mathbf{U}_2 \mathbf{U}_2^T = \begin{pmatrix} | & | \\ \mathbf{u}_1 & \mathbf{u}_2 \\ | & | \end{pmatrix} \begin{pmatrix} \text{---} \mathbf{u}_1^T \text{---} \\ \text{---} \mathbf{u}_2^T \text{---} \end{pmatrix} = \mathbf{u}_1 \mathbf{u}_1^T + \mathbf{u}_2 \mathbf{u}_2^T \\ &= \begin{pmatrix} 0.152 & -0.035 & 0.357 \\ -0.035 & 0.008 & -0.082 \\ 0.357 & -0.082 & 0.839 \end{pmatrix} + \begin{pmatrix} 0.408 & 0.474 & -0.128 \\ 0.474 & 0.551 & -0.148 \\ -0.128 & -0.148 & 0.04 \end{pmatrix} \\ &= \begin{pmatrix} 0.560 & 0.439 & 0.229 \\ 0.439 & 0.558 & -0.230 \\ 0.229 & -0.230 & 0.879 \end{pmatrix} \end{aligned}$$

Example

- Thus, each point $\mathbf{x}_i$ can be approximated by its projection onto the first two principal components $\mathbf{x}'_i = \mathbf{P}_2 \mathbf{x}_i$
- The total variance captured by the subspace is given as
- $\lambda_1 + \lambda_2 = 3.662 + 0.239 = 3.901$.
- The mean squared error is given as
- $MSE = \text{var}(\mathbf{D}) - \lambda_1 - \lambda_2 = 3.96 - 3.662 - 0.239 = 0.059$



(a) Optimal basis



(b) Nonoptimal basis

Principal Component Analysis

PCA ($\mathbf{D}, \alpha$):

- 1 $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$ // compute mean
- 2 $\mathbf{Z} = \mathbf{D} - \mathbf{1} \cdot \boldsymbol{\mu}^T$ // center the data
- 3 $\boldsymbol{\Sigma} = \frac{1}{n} (\mathbf{Z}^T \mathbf{Z})$ // compute covariance matrix
- 4 $(\lambda_1, \lambda_2, \dots, \lambda_d) = \text{eigenvalues}(\boldsymbol{\Sigma})$ // compute eigenvalues
- 5 $\mathbf{U} = (\mathbf{u}_1 \quad \mathbf{u}_2 \quad \dots \quad \mathbf{u}_d) = \text{eigenvectors}(\boldsymbol{\Sigma})$ // compute eigenvectors
- 6 $f(r) = \frac{\sum_{i=1}^r \lambda_i}{\sum_{i=1}^d \lambda_i}$, for all $r = 1, 2, \dots, d$ // fraction of total variance
- 7 Choose smallest r so that $f(r) \geq \alpha$ // choose dimensionality
- 8 $\mathbf{U}_r = (\mathbf{u}_1 \quad \mathbf{u}_2 \quad \dots \quad \mathbf{u}_r)$ // reduced basis
- 9 $\mathbf{A} = \{\mathbf{a}_i \mid \mathbf{a}_i = \mathbf{U}_r^T \mathbf{x}_i, \text{ for } i = 1, \dots, n\}$ // reduced dimensionality data

Example

$$\Sigma = \begin{pmatrix} 0.681 & -0.039 & 1.265 \\ -0.039 & 0.187 & -0.320 \\ 1.265 & -0.32 & 3.092 \end{pmatrix}$$

The eigenvalues and eigenvectors of Σ are given as

$$\lambda_1 = 3.662$$

$$\lambda_2 = 0.239$$

$$\lambda_3 = 0.059$$

$$\mathbf{u}_1 = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix}$$

$$\mathbf{u}_2 = \begin{pmatrix} -0.639 \\ -0.742 \\ 0.200 \end{pmatrix}$$

$$\mathbf{u}_3 = \begin{pmatrix} -0.663 \\ 0.664 \\ 0.346 \end{pmatrix}$$

The total variance is therefore $\lambda_1 + \lambda_2 + \lambda_3 = 3.662 + 0.239 + 0.059 = 3.96$.

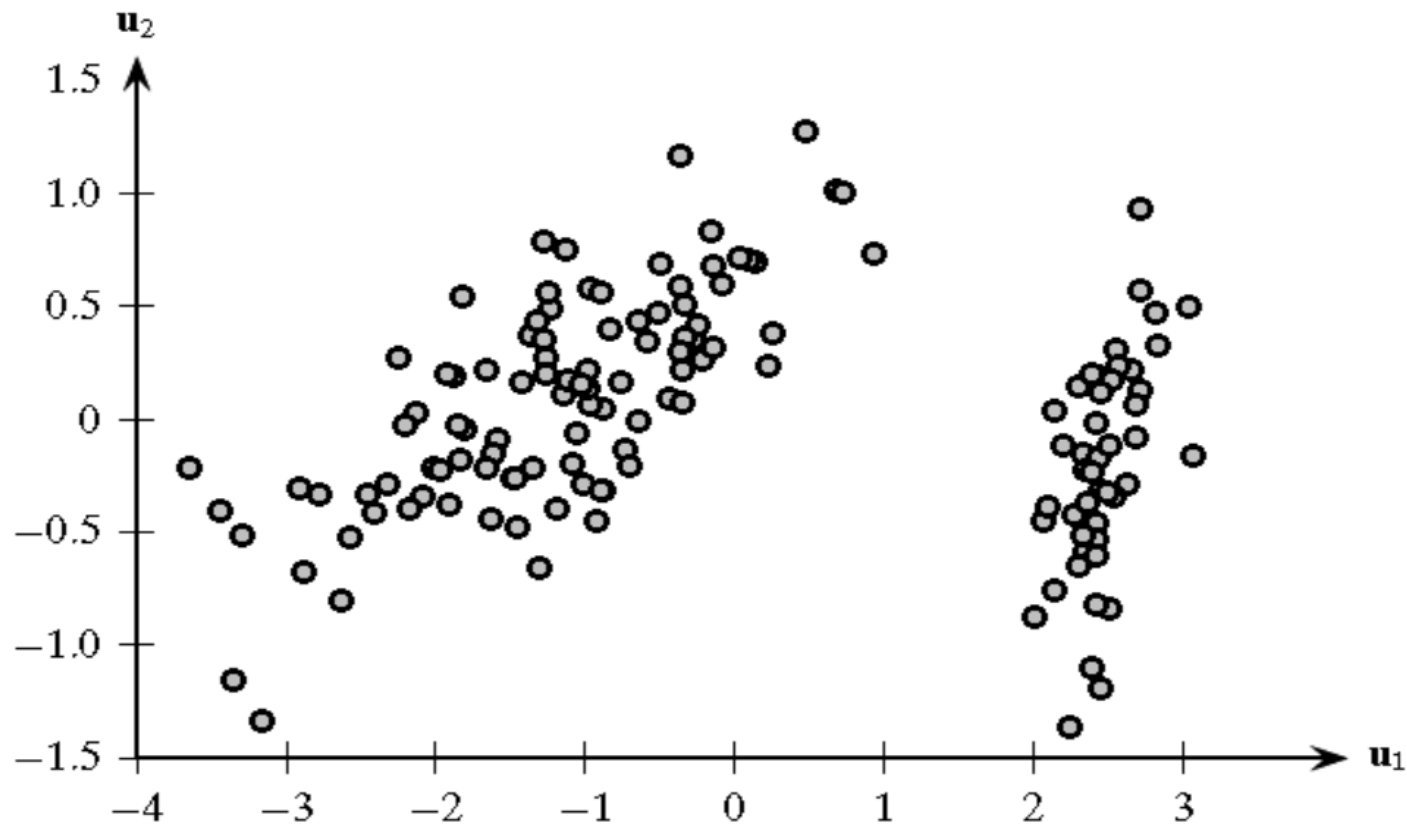
Example

To find a lower dimensional approximation, let $\alpha = 0.95$. The fraction of total variance for different values of r is given as

r	1	2	3
$f(r)$	0.925	0.985	1.0

For example, for $r = 1$, the fraction of total variance is given as $f(1) = \frac{3.662}{3.96} = 0.925$. Thus, we need at least $r = 2$ dimensions to capture 95% of the total variance.

Example



Reduced dimensionality dataset: Iris principal components.

SINGULAR VALUE DECOMPOSITION

- PCA yields the following decomposition of the covariance matrix:

$$\mathbf{\Sigma} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$$

- where the covariance matrix has been factorized into the orthogonal matrix $\mathbf{U}$ containing its eigenvectors,
- and a diagonal matrix $\mathbf{\Lambda}$ containing its eigenvalues (sorted in decreasing)
- **SVD generalizes the above factorization for any matrix.**
- for an $n \times d$ data matrix $\mathbf{D}$ with, SVD factorizes $\mathbf{D}$ as follows:

$$\mathbf{D} = \mathbf{L}\mathbf{\Lambda}\mathbf{R}^T$$

SINGULAR VALUE DECOMPOSITION

$$\mathbf{D} = \mathbf{L}\mathbf{\Delta}\mathbf{R}^T$$

- where $\mathbf{L}$ is a orthogonal $n \times n$ matrix, $\mathbf{R}$ is an orthogonal $d \times d$ matrix,
- $\mathbf{\Delta}$ is an $n \times d$ “diagonal” matrix.
- The columns of $\mathbf{L}$ are called the *left singular vectors*, and the
- columns of $\mathbf{R}$ (or rows of $\mathbf{R}^t$) are called the *right singular vectors*.
- The matrix $\mathbf{\Delta}$ is defined as.
- where $i = 1, \dots, n$
- and $j = 1, \dots, d$

$$\Delta(i, j) = \begin{cases} \delta_i & \text{If } i = j \\ 0 & \text{If } i \neq j \end{cases}$$

SINGULAR VALUE DECOMPOSITION

- The entries $\Delta(i, i) = \delta_i$ along the main diagonal of Δ are called the *singular values* of $\mathbf{D}$,
- and they are all non-negative.
- If the rank of $\mathbf{D}$ is $r \leq \min(n, d)$, then there will be only r nonzero singular values, which we assume are ordered as follows:
- $\delta_1 \geq \delta_2 \geq \dots \geq \delta_r > 0$
- One can discard those left and right singular vectors that correspond to zero singular values, to obtain the *reduced SVD* as

$$\mathbf{D} = \mathbf{L}_r \mathbf{\Delta}_r \mathbf{R}_r^T$$

SINGULAR VALUE DECOMPOSITION

$$\mathbf{D} = \mathbf{L}_r \mathbf{\Delta}_r \mathbf{R}_r^T$$

- where $\mathbf{L}_r$ is the $n \times r$ matrix of the left singular vectors,
- $\mathbf{R}_r$ is the $d \times r$ matrix of the right singular vectors,
- and $\mathbf{\Delta}_r$ is the $r \times r$ diagonal matrix containing the positive singular values.

reduced SVD leads directly to *spectral decomposition* of $\mathbf{D}$, given as

$$\begin{aligned}
 &= \begin{pmatrix} | & | & \cdots & | \\ \mathbf{l}_1 & \mathbf{l}_2 & \cdots & \mathbf{l}_r \\ | & | & \cdots & | \end{pmatrix} \begin{pmatrix} \delta_1 & 0 & \cdots & 0 \\ 0 & \delta_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \delta_r \end{pmatrix} \begin{pmatrix} \text{---} & \mathbf{r}_1^T & \text{---} \\ \text{---} & \mathbf{r}_2^T & \text{---} \\ \text{---} & \vdots & \text{---} \\ \text{---} & \mathbf{r}_r^T & \text{---} \end{pmatrix} \\
 &= \delta_1 \mathbf{l}_1 \mathbf{r}_1^T + \delta_2 \mathbf{l}_2 \mathbf{r}_2^T + \cdots + \delta_r \mathbf{l}_r \mathbf{r}_r^T \\
 &= \sum_{i=1}^r \delta_i \mathbf{l}_i \mathbf{r}_i^T
 \end{aligned}$$

SINGULAR VALUE DECOMPOSITION

- spectral decomposition represents $\mathbf{D}$ as a sum of rank one matrices of form

$$\delta_i \mathbf{l}_i \mathbf{r}_i^T.$$

- . By selecting the q largest singular values $\delta_1, \delta_2, \dots, \delta_q$ and the corresponding left and right singular vectors,
- we obtain the best rank q approximation to the original matrix $\mathbf{D}$.
- That is, if $\mathbf{D}_q$ is the matrix defined as

$$\mathbf{D}_q = \sum_{i=1}^q \delta_i \mathbf{l}_i \mathbf{r}_i^T$$

SVD

then it can be shown that $\mathbf{D}_q$ is the rank q matrix that minimizes the expression

$$\|\mathbf{D} - \mathbf{D}_q\|_F$$

where $\|\mathbf{A}\|_F$ is called the *Frobenius Norm* of the $n \times d$ matrix $\mathbf{A}$, defined as

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^d \mathbf{A}(i, j)^2}$$

Geometry of SVD

- In general, any $n \times d$ matrix $\mathbf{D}$ represents a *linear transformation*,
- $\mathbf{D}: \mathbb{R}^d \rightarrow \mathbb{R}^n$, from the space of d -dimensional vectors to the space of n -dimensional vectors
- because for any $\mathbf{x} \in \mathbb{R}^d$ there exists $\mathbf{y} \in \mathbb{R}^n$ such that
- $\mathbf{D}\mathbf{x} = \mathbf{y}$
- The set of all vectors $\mathbf{y} \in \mathbb{R}^n$ such that $\mathbf{D}\mathbf{x} = \mathbf{y}$ over all possible $\mathbf{x} \in \mathbb{R}^d$ is called the *column space* of $\mathbf{D}$,
- and the set of all vectors $\mathbf{x} \in \mathbb{R}^d$, such that $\mathbf{D}^t \mathbf{y} = \mathbf{x}$ over all $\mathbf{y} \in \mathbb{R}^n$,
- is called the *row space* of $\mathbf{D}$, which is equivalent to the column space of $\mathbf{D}^t$.

Geometry of SVD

- Also note that
- the set of all vectors $\mathbf{x} \in \mathbb{R}^d$, such that $\mathbf{D}\mathbf{x} = \mathbf{0}$ is called the *null space* of $\mathbf{D}$, and finally,
- the set of all vectors $\mathbf{y} \in \mathbb{R}^n$, such that $\mathbf{D}^t \mathbf{y} = \mathbf{0}$ is called the *left null space* of $\mathbf{D}$.

Geometry of SVD

- If $\mathbf{D}$ has rank r , it means that it has only r independent columns, and also only r independent rows.
- Thus, the r **left singular vectors** $\mathbf{l}_1, \mathbf{l}_2, \dots, \mathbf{l}_r$ corresponding to the r nonzero singular values of $\mathbf{D}$ in Eq. represent a basis for the **column space of $\mathbf{D}$** .
- The remaining $n-r$ left singular vectors $\mathbf{l}_{r+1}, \dots, \mathbf{l}_n$ represent a basis for the **left null space of $\mathbf{D}$** .
- **For the row space**, the r right singular vectors $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_r$ corresponding to the r non-zero singular values, represent a basis for the **row space of $\mathbf{D}$** ,
- and the remaining $d-r$ right singular vectors $\mathbf{r}_j$ ($j = r+1, \dots, d$), represent a basis for the null space of $\mathbf{D}$

Geometry of SVD

- Consider the reduced SVD expression,
- Right multiplying both sides of the equation by $\mathbf{R}_r$ and noting that $\mathbf{R}_r^T \mathbf{R}_r = \mathbf{I}_r$
- $\mathbf{R}_r^T \mathbf{R}_r = \mathbf{I}_r$, where $\mathbf{I}_r$ is the $r \times r$ identity matrix,
- we have

$$\mathbf{D} \mathbf{R}_r = \mathbf{L}_r \mathbf{\Delta}_r \mathbf{R}_r^T \mathbf{R}_r$$

$$\mathbf{D} \mathbf{R}_r = \mathbf{L}_r \mathbf{\Delta}_r$$

$$\mathbf{D} \mathbf{R}_r = \mathbf{L}_r \begin{pmatrix} \delta_1 & 0 & \cdots & 0 \\ 0 & \delta_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \delta_r \end{pmatrix}$$

$$\mathbf{D} \begin{pmatrix} | & | & \cdots & | \\ \mathbf{r}_1 & \mathbf{r}_2 & \cdots & \mathbf{r}_r \\ | & | & \cdots & | \end{pmatrix} = \begin{pmatrix} | & | & \cdots & | \\ \delta_1 \mathbf{l}_1 & \delta_2 \mathbf{l}_2 & \cdots & \delta_r \mathbf{l}_r \\ | & | & \cdots & | \end{pmatrix}$$

Geometry of SVD

- From the above, we conclude that

$$\mathbf{D}\mathbf{r}_i = \delta_i \mathbf{l}_i \quad \text{for all } i = 1, \dots, r$$

- SVD is a special factorization of the matrix $\mathbf{D}$,
- such that any basis **vector $\mathbf{r}_i$ for the row space** is mapped to the corresponding basis vector **$\mathbf{l}_i$ in the column space**,
- scaled by the **singular value δ_i** .
- SVD as a mapping from an orthonormal basis $(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_r)$ in $\mathbb{R}^d$ (the row space) to an orthonormal basis $(\mathbf{l}_1, \mathbf{l}_2, \dots, \mathbf{l}_r)$ in $\mathbb{R}^n$ (the column space), with the corresponding axes scaled according to the singular values $\delta_1, \delta_2, \dots, \delta_r$.

Connection between SVD and PCA

$$\mathbf{D} = \mathbf{L}\mathbf{\Delta}\mathbf{R}^T,$$

$$\begin{aligned}\mathbf{D}^T\mathbf{D} &= (\mathbf{L}\mathbf{\Delta}\mathbf{R}^T)^T (\mathbf{L}\mathbf{\Delta}\mathbf{R}^T) \\ &= \mathbf{R}\mathbf{\Delta}^T\mathbf{L}^T\mathbf{L}\mathbf{\Delta}\mathbf{R}^T\end{aligned}$$

$$\begin{aligned}&= \mathbf{R}(\mathbf{\Delta}^T\mathbf{\Delta})\mathbf{R}^T \\ &= \mathbf{R}\mathbf{\Delta}_d^2\mathbf{R}^T\end{aligned}$$

where $\mathbf{\Delta}_d^2$ is the $d \times d$ diagonal matrix defined as $\mathbf{\Delta}_d^2(i, i) = \delta_i^2$, for $i = 1, \dots, d$. Only $r \leq \min(d, n)$ of these eigenvalues are positive, whereas the rest are all zeros.

Connection between SVD and PCA

Because the covariance matrix of centered $\mathbf{D}$ is given as $\mathbf{\Sigma} = \frac{1}{n}\mathbf{D}^T\mathbf{D}$, it can be decomposed as $\mathbf{\Sigma} = \mathbf{\bar{U}}\mathbf{\Lambda}\mathbf{U}^T$ via PCA

$$\begin{aligned}\mathbf{D}^T\mathbf{D} &= n\mathbf{\Sigma} \\ &= n\mathbf{U}\mathbf{\Lambda}\mathbf{U}^T \\ &= \mathbf{U}(n\mathbf{\Lambda})\mathbf{U}^T\end{aligned}$$

we conclude that the right singular vectors $\mathbf{R}$ are the same as the eigenvectors of $\mathbf{\Sigma}$
corresponding singular values of $\mathbf{D}$ are related to the eigenvalues of $\mathbf{\Sigma}$ by expression

Connection between SVD and PCA

$$n\lambda_i = \delta_i^2$$

$$\text{or, } \lambda_i = \frac{\delta_i^2}{n}, \text{ for } i = 1, \dots, d$$

Let us now consider the matrix $\mathbf{D}\mathbf{D}^T$. We have

$$\begin{aligned}\mathbf{D}\mathbf{D}^T &= (\mathbf{L}\mathbf{\Delta}\mathbf{R}^T)(\mathbf{L}\mathbf{\Delta}\mathbf{R}^T)^T \\ &= \mathbf{L}\mathbf{\Delta}\mathbf{R}^T\mathbf{R}\mathbf{\Delta}^T\mathbf{L}^T \\ &= \mathbf{L}(\mathbf{\Delta}\mathbf{\Delta}^T)\mathbf{L}^T \\ &= \mathbf{L}\mathbf{\Delta}_n^2\mathbf{L}^T\end{aligned}$$

Connection between SVD and PCA

where $\mathbf{\Delta}_n^2$ is the $n \times n$ diagonal matrix given as $\mathbf{\Delta}_n^2(i, i) = \delta_i^2$, for $i = 1, \dots, n$. Only r of these singular values are positive, whereas the rest are all zeros. Thus, the left singular vectors in $\mathbf{L}$ are the eigenvectors of the matrix $n \times n$ matrix $\mathbf{D}\mathbf{D}^T$, and the corresponding eigenvalues are given as δ_i^2 .

Example

- Let us consider the $n \times d$ centered Iris datamatrix $\mathbf{D}$
- we computed the eigenvectors and eigenvalues of the covariance matrix Σ as follows:

$$\lambda_1 = 3.662$$

$$\mathbf{u}_1 = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix}$$

$$\lambda_2 = 0.239$$

$$\mathbf{u}_2 = \begin{pmatrix} -0.639 \\ -0.742 \\ 0.200 \end{pmatrix}$$

$$\lambda_3 = 0.059$$

$$\mathbf{u}_3 = \begin{pmatrix} -0.663 \\ 0.664 \\ 0.346 \end{pmatrix}$$

Example

- Computing the SVD of **D** yields the following nonzero singular values and the corresponding right singular vectors

$$\delta_1 = 23.437$$

$$\delta_2 = 5.992$$

$$\delta_3 = 2.974$$

$$\mathbf{r}_1 = \begin{pmatrix} -0.390 \\ 0.089 \\ -0.916 \end{pmatrix}$$

$$\mathbf{r}_2 = \begin{pmatrix} 0.639 \\ 0.742 \\ -0.200 \end{pmatrix}$$

$$\mathbf{r}_3 = \begin{pmatrix} -0.663 \\ 0.664 \\ 0.346 \end{pmatrix}$$

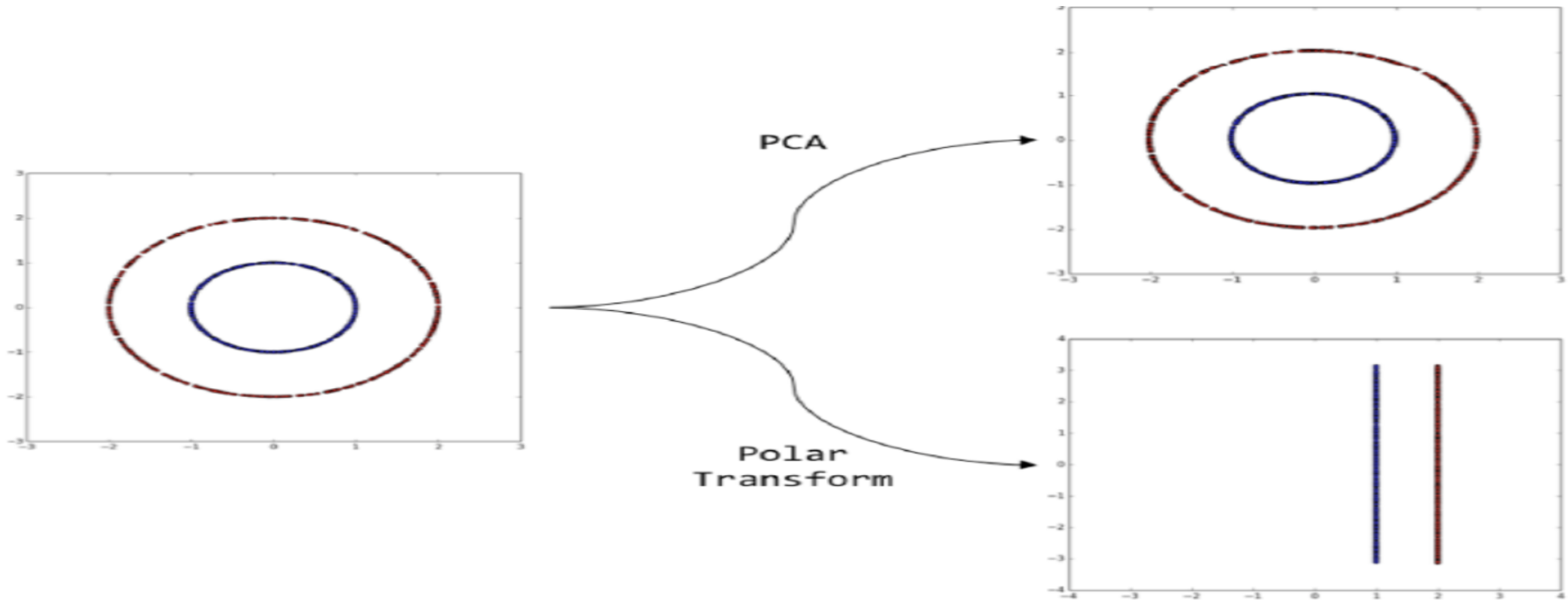
$$\lambda_1 = \frac{\delta_1^2}{n} = \frac{23.437^2}{150} = \frac{549.29}{150} = 3.662$$

Example

- Notice also that the right singular vectors are equivalent to the principal components or eigenvectors of Σ , up to **isomorphism**.
- That is, they may potentially be reversed in direction.
- **For the Iris dataset, we have $r_1 = u_1$, $r_2 = -u_2$, and $r_3 = u_3$.**
- Here the **second right singular vector is reversed** in sign when compared to the second principal component.

Problem with PCA

- While PCA has been used for decades for dimensionality reduction, it fails to capture important relationships that are piecewise linear or nonlinear.



Problem with PCA

- The example shows data points selected at random from two concentric circles.
- We hope that PCA will transform this dataset so that we can pick a single new axis that allows us to easily separate the red and blue dots. Unfortunately for us, there is no linear direction that contains more information here than another (we have equal variance in all directions)..

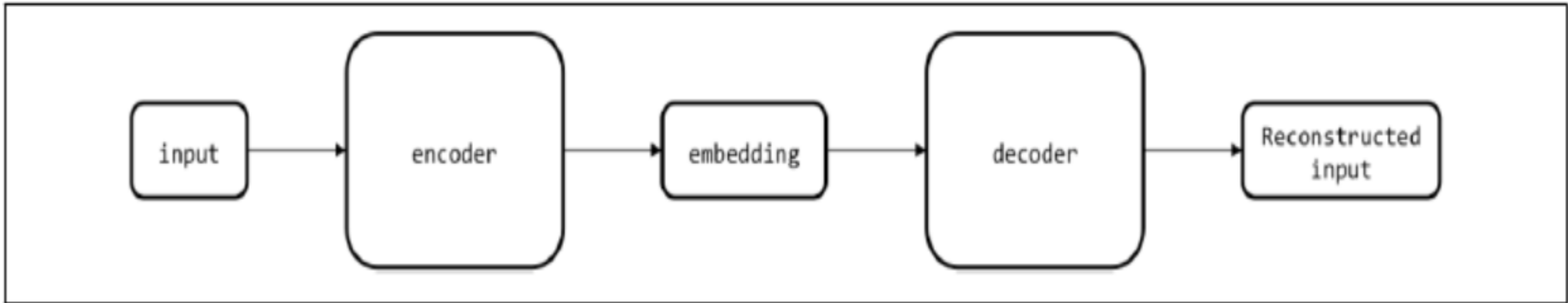
Motivating the Autoencoder Architecture

- In feed-forward networks, how each layer learned progressively more relevant representations of the input.
- output of the final convolutional layer and used that as a lower-dimensional representation of the input image.
- we want to generate these **low-dimensional representations in an unsupervised fashion**,

Motivating the Autoencoder Architecture

- We first take the input and compress it into a low-dimensional vector. This part of the network is called **the *encoder***
- because it is responsible for producing the low-dimensional embedding or *code*.
- The second part of the network, instead of mapping the embedding to an arbitrary label as we would in a feed-forward network,
tries to invert the computation of the first half of the network and reconstruct the original input. This piece is known as the ***decoder***

Motivating the Autoencoder Architecture



- *The autoencoder architecture attempts to construct a high-dimensional input into a low-dimensional embedding and then uses that low-dimensional embedding to reconstruct the input*

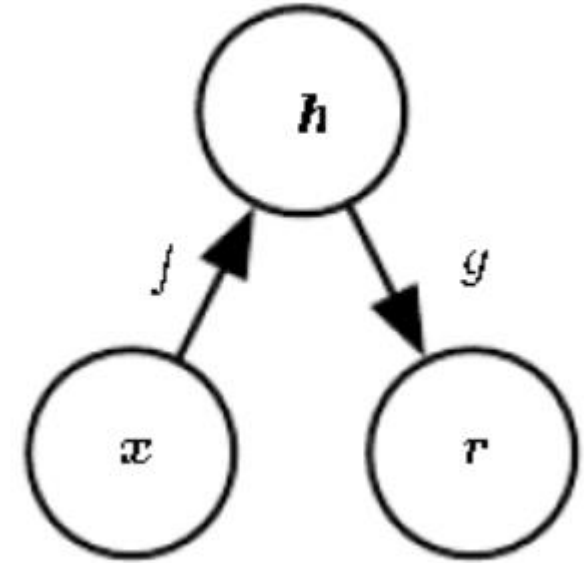
Autoencoder

- **An autoencoder is a neural network**
- **that is trained to attempt to copy its input to its output.**
- it has a hidden layer h that describes a code used to represent the input.
- The network may be viewed as consisting of two parts:
- An encoder function $h = f(x)$
- and a decoder that produces a reconstruction $r = g(h)$.
- If an autoencoder succeeds in simply learning to set $g(f(x)) = x$ everywhere, then it is not especially useful.
- **autoencoders are designed to be unable to learn to copy perfectly**

Autoencoder

Modern autoencoders have generalized the idea of an encoder and a decoder beyond deterministic functions to stochastic mappings

$p_{\text{encoder}}(h \mid x)$ and
 $p_{\text{decoder}}(x \mid h)$.



mapping an input x to an output (called reconstruction) r through an internal representation or code h .

The autoencoder has two components:

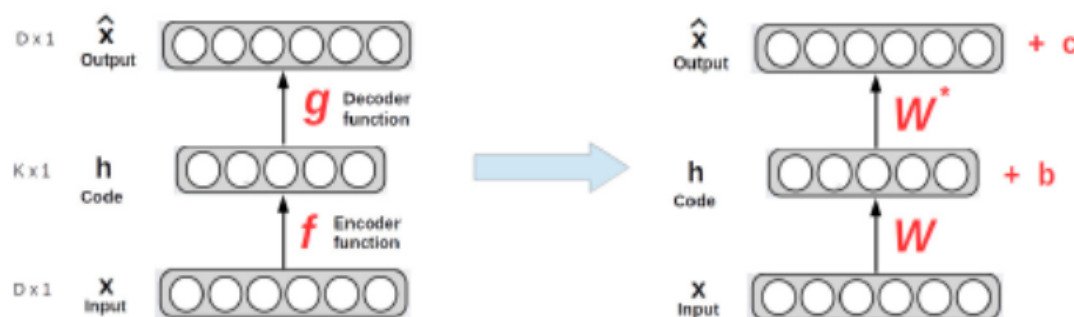
the encoder f (mapping x to h) and the decoder g (mapping h to r).

Simple autoencoder

- Let's assume a $D \times 1$ input $\mathbf{x} \in \mathbb{R}^D$, and a single hidden layer with $K \times 1$ code $\mathbf{h} \in \mathbb{R}^K$
- We can then define a simple **linear autoencoder** as

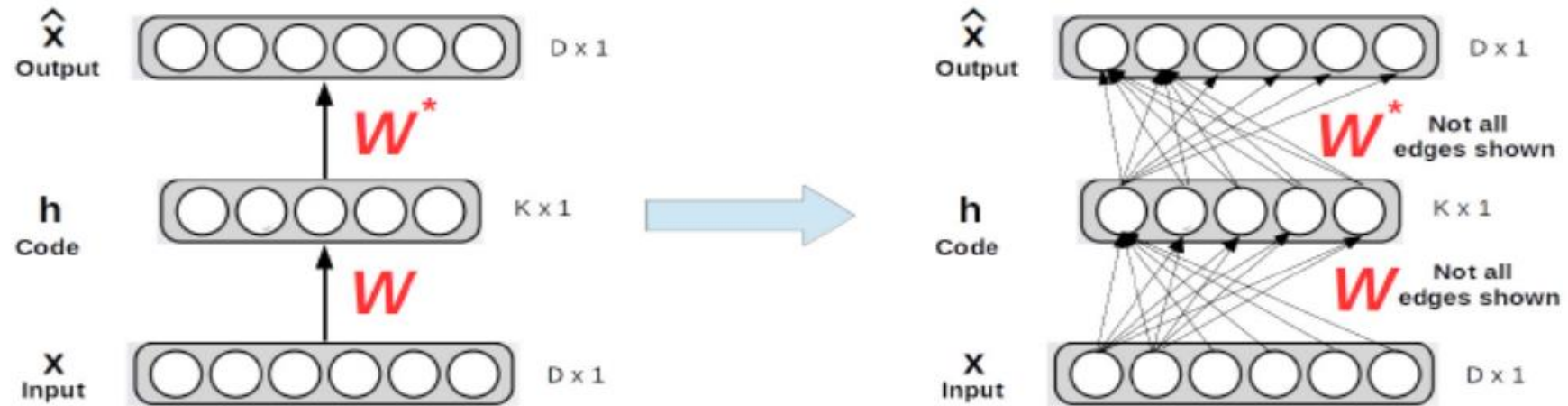
$$\begin{aligned}\mathbf{h} &= f(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b} \\ \hat{\mathbf{x}} &= g(\mathbf{h}) = \mathbf{W}^*\mathbf{h} + \mathbf{c}\end{aligned}$$

where f is defined by $\mathbf{W} \in \mathbb{R}^{K \times D}$ and $\mathbf{b} \in \mathbb{R}^{K \times 1}$, g is defined by $\mathbf{W}^* \in \mathbb{R}^{D \times K}$ and $\mathbf{c} \in \mathbb{R}^{D \times 1}$



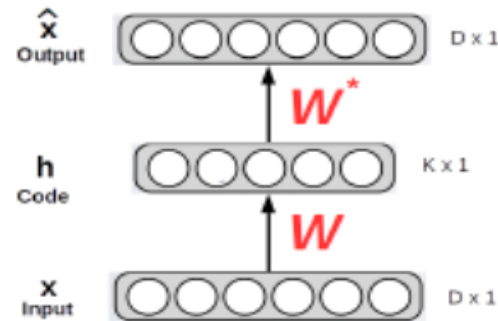
- Note: If we learn f, g to minimize the squared error $\|\hat{\mathbf{x}} - \mathbf{x}\|^2$ then the linear autoencoder with $\mathbf{W}^* = \mathbf{W}^T$ is optimal

Simple autoencoder



- $\mathbf{W}$: $K \times D$ matrix of weights of edges between input and hidden layer
 - W_{kd} is the weight of edge connecting input layer node d to hidden layer node k
- $\mathbf{W}^*$: $D \times K$ matrix of weights of edges between hidden and output layer
 - W_{dk}^* is the weight of edge connecting hidden layer node k to output layer node d
- If $\mathbf{W}^* = \mathbf{W}^\top$, the autoencoder architecture is said to have “**tied weights**”

Non Linear autoencoder



- The hidden nodes can also be **nonlinear transforms** of the inputs, e.g.,
 - Can define $\mathbf{h}$ as a linear transform of $\mathbf{x}$ followed by a nonlinearity (e.g., sigmoid, ReLU)

$$\mathbf{h} = \text{sigmoid}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

where the nonlinearity $\text{sigmoid}(z) = \frac{1}{1+\exp(-z)}$ squashes the real-valued z to lie between 0 and 1

- Most commonly used autoencoders use such nonlinear transforms
- Note: If inputs $\mathbf{x} \in \{0, 1\}^D$ are binary, it may be more appropriate to also define $\hat{\mathbf{x}}$ as

$$\hat{\mathbf{x}} = \text{sigmoid}(\mathbf{W}^*\mathbf{h} + \mathbf{c})$$

Training the autoencoder

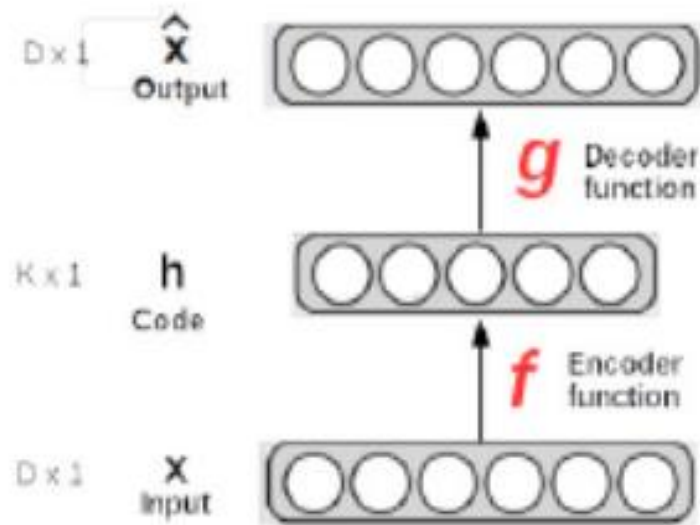
- To train the autoencoder, we need to define a **loss function** $\ell(\hat{\mathbf{x}}, \mathbf{x})$
- The loss function (a function of parameters $\mathbf{W}, \mathbf{b}, \mathbf{W}^*, \mathbf{c}$) can be defined using various ways
- In general, it is defined in terms of the difference between $\hat{\mathbf{x}}$ and $\mathbf{x}$ (**reconstruction error**)
- For a single input $\mathbf{x} = [x_1, \dots, x_D]$ and its reconstruction $\hat{\mathbf{x}} = [\hat{x}_1, \dots, \hat{x}_D]$

$$\ell(\hat{\mathbf{x}}, \mathbf{x}) = \sum_{d=1}^D (\hat{x}_d - x_d)^2 \quad (\text{**squared loss**; used if input are real-valued})$$

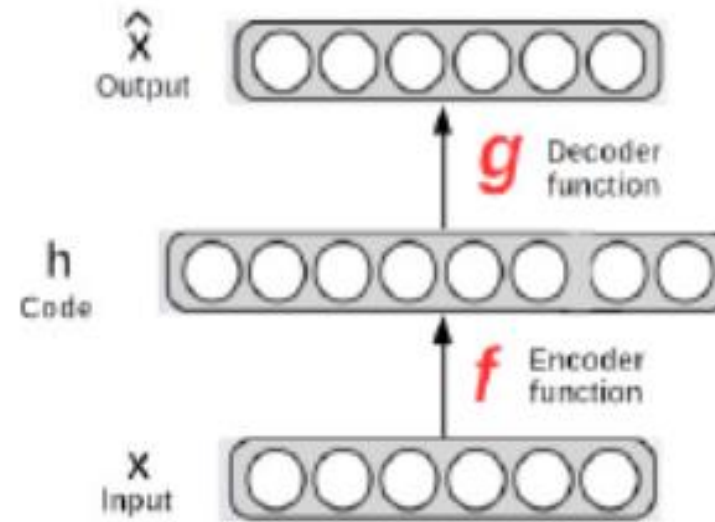
$$\ell(\hat{\mathbf{x}}, \mathbf{x}) = - \sum_{d=1}^D [x_d \log(\hat{x}_d) + (1 - x_d) \log(1 - \hat{x}_d)] \quad (\text{**cross-entropy loss**; used if input are binary})$$

- We find $(\mathbf{W}, \mathbf{b}, \mathbf{W}^*, \mathbf{c})$ by minimizing the reconstruction error (summed over all training data)
- This can be done using backpropagation

Undercomplete, Overcomplete, and Need for Regularization



Undercomplete
($K < D$)



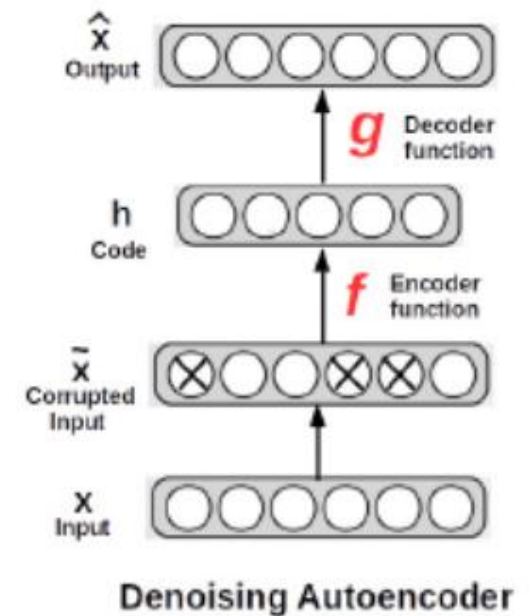
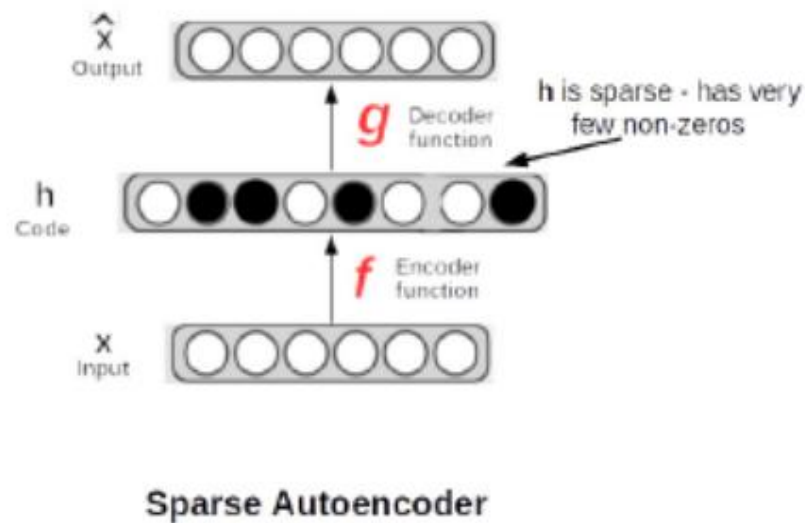
Overcomplete
($K \geq D$)

Undercomplete, Overcomplete, and Need for Regularization

- In both cases, it is important **to control the capacity of encoder and decoder**
- **Undercomplete:** Imagine $K = 1$ and very powerful f and g .
Can achieve very small reconstruction error but the **learned code will not capture any interesting properties in the data**
- **Overcomplete:** Imagine $K \gg D$ and trivial (identity) functions f and g .
Can achieve even zero reconstruction error
but the learned code will not capture any interesting properties in the data
- It is therefore **important to regularize the functions** as well as the **learned code**, and **not just focus on minimizing the reconstruction error**

Regularized Autoencoders

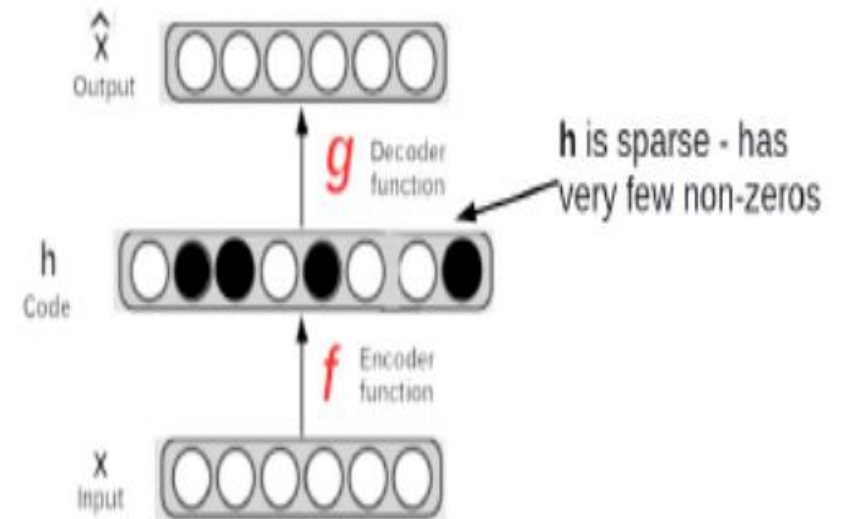
- Several ways to regularize the model, e.g.
- Make the learned code sparse (**Sparse Autoencoders**)
- Make the model robust against noisy/incomplete inputs (**Denoising Autoencoders**)



Make the model robust against small changes in the input (Contractive Autoencoders)

Sparse Autoencoders

- Make the **learned code sparse** (Sparse Autoencoders). Done by adding a sparsity penalty on h
- Loss Function: $l(\hat{x}, x) + \Omega(h)$
- Where $\Omega(h) = \sum_{k=1}^K |h_k|$ is the **l_1 norm of h**

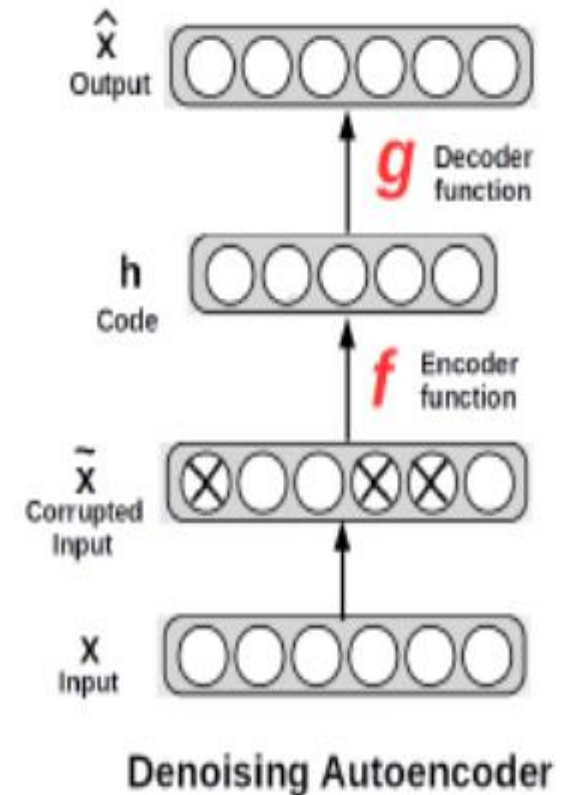


- Sparse autoencoder is learned by minimizing the above regularized loss function

Denoising Autoencoders

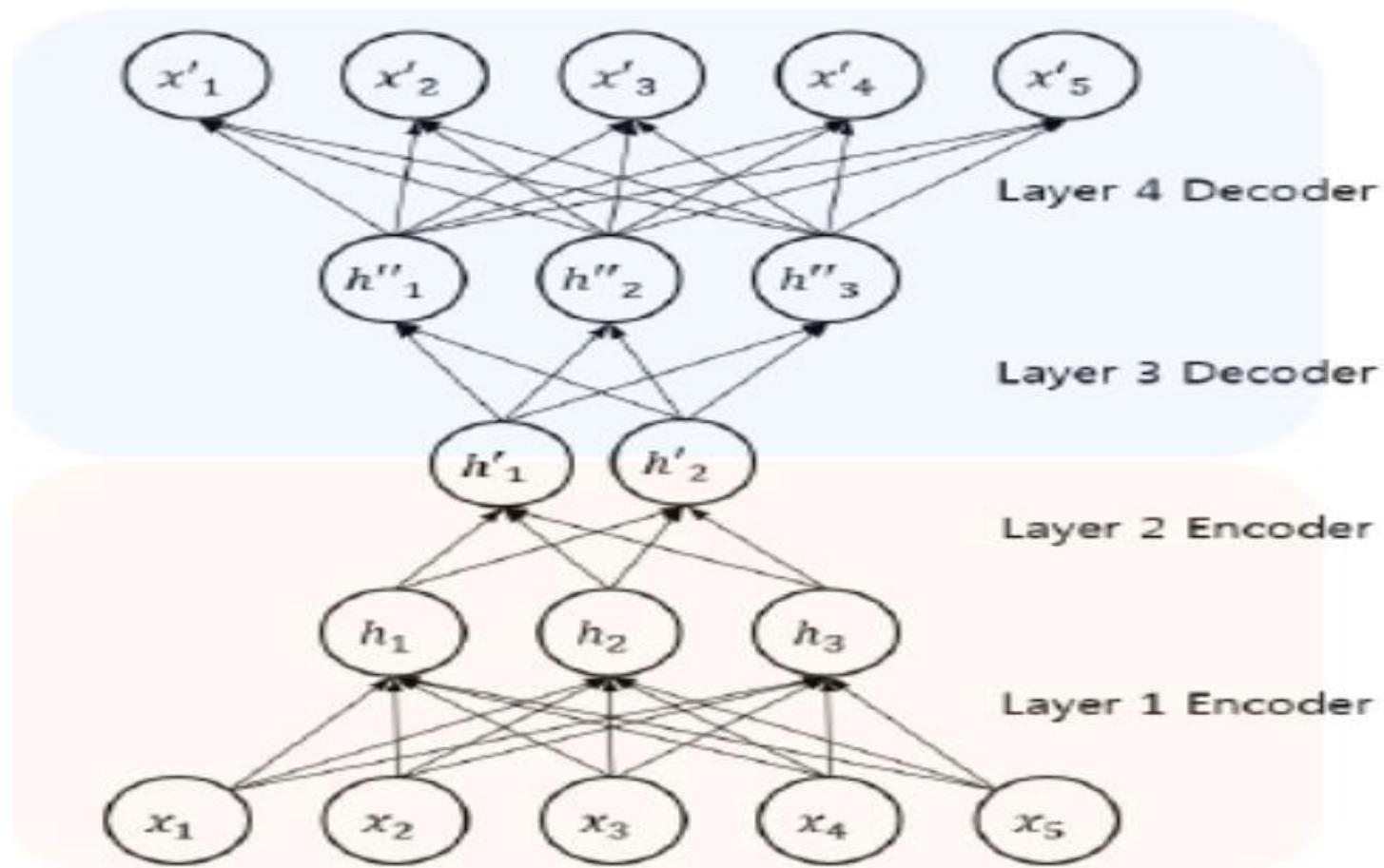
- First add some noise (e.g., Gaussian noise) to the original input x
- Let's denote $\tilde{x}$ as the corrupted version of x
- The encoder f operates on $\tilde{x}$ i.e., $h = f(\tilde{x})$
- However, we still want to reconstruction $\hat{x}$ to be close to the original uncorrupted input x
- Since the corruption is stochastic, we minimize the expected loss:

$$\mathbb{E}_{\tilde{x} \sim p(\tilde{x}|x)} [\ell(\hat{x}, \tilde{x})] + \Omega(h)$$



Deep/Stacked Autoencoders

- Most autoencoders can be extended to have more than one hidden layer



Stochastic Autoencoders

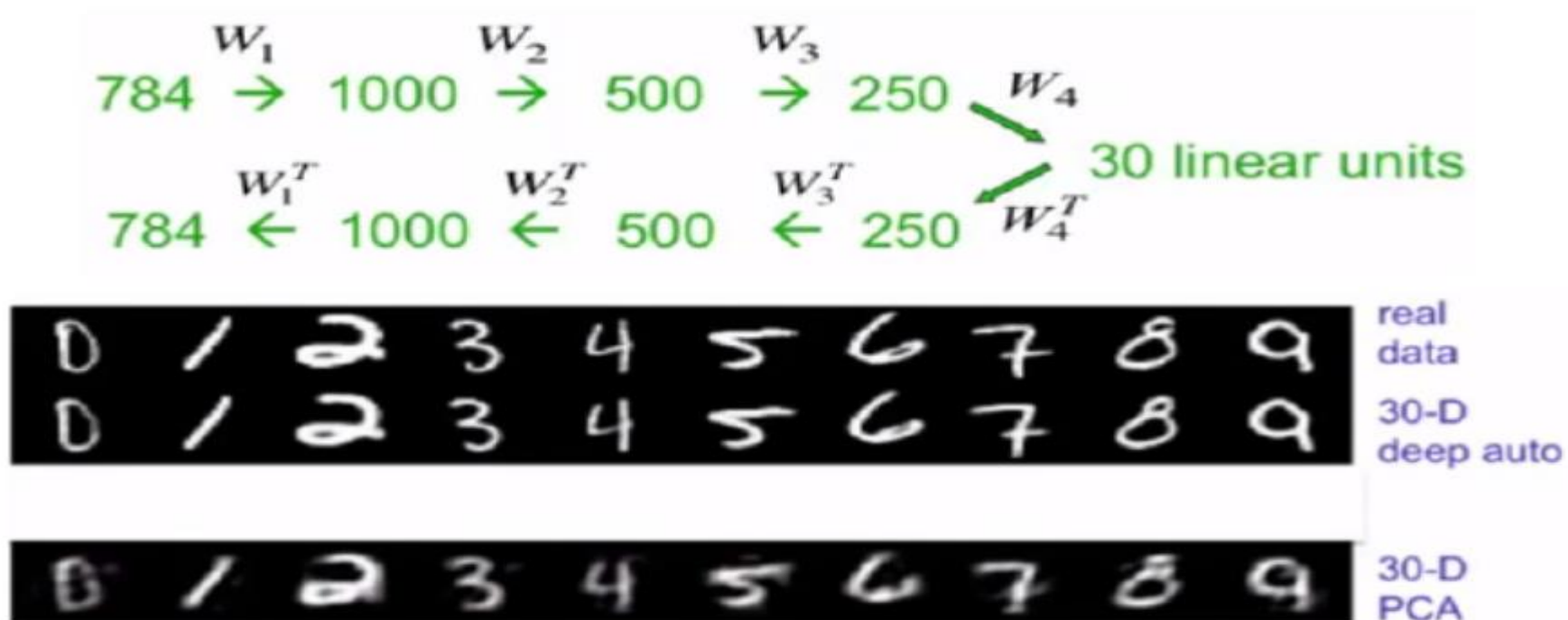
- Can also done the encoder and decoder functions using probability distributions.
- $p_{\text{encoder}}(h|x)$
- $p_{\text{decoder}}(x|h)$
- The choice of distributions depends on the type of data being modeled and of the encodings
- This gives a **probabilistic approach** for designing autoencoders
- **Negative log-likelihood** $-\log p_{\text{encoder}}(h|x)$ is equivalent to the **reconstruction error**.
- Can also use a **prior distribution** $p(h)$ on the encodings (**equivalent to regularizer**)
- Such ideas have been used to design generative models for autoencoders
- Variational Autoencoder (VAE) is a popular example of such a model

Applications of Autoencoders

- **autoencoders were used for dimensionality reduction or feature learning.**
- **Denoising and inpainting** (that fills in missing or damaged parts of an image or video using information from the surrounding area)
- **Pre-training of deep neural networks**
- **Recommender systems applications**

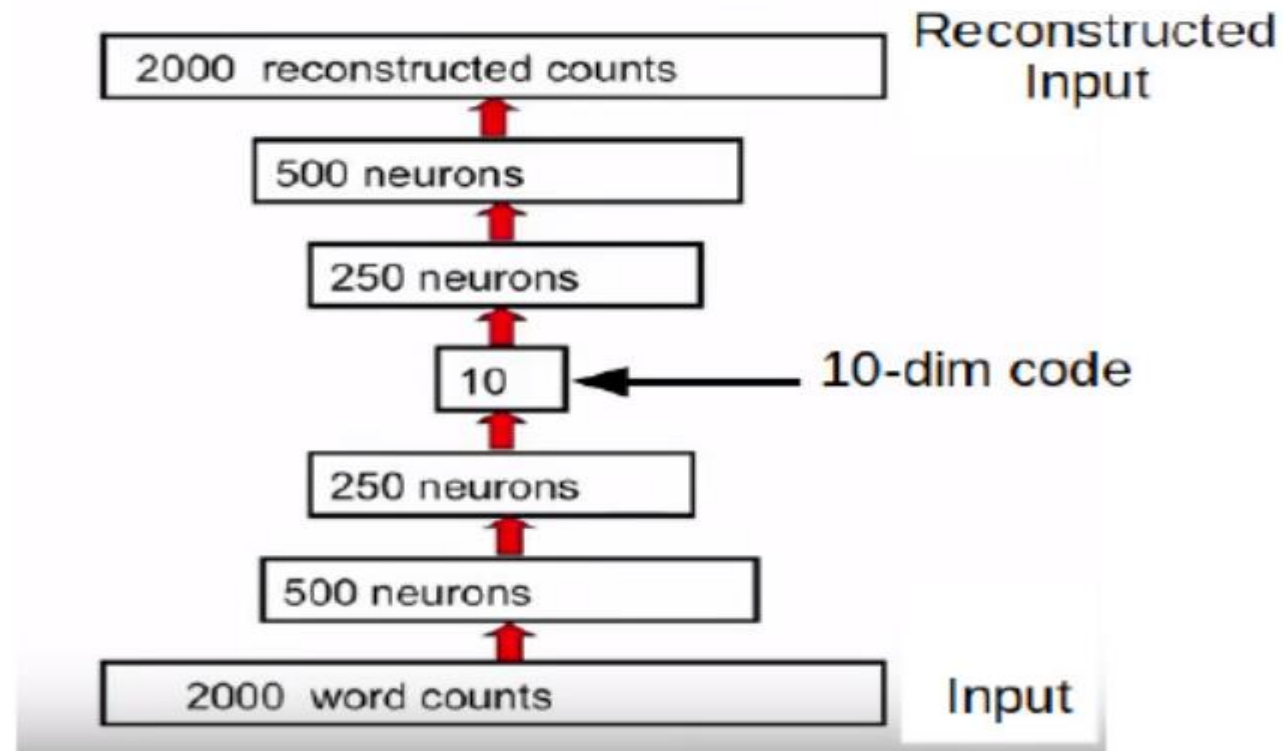
Feature learning and Dimensionality Reduction

- Example: A deep AE for low-dim feature learning for 784-dimensional MNIST images

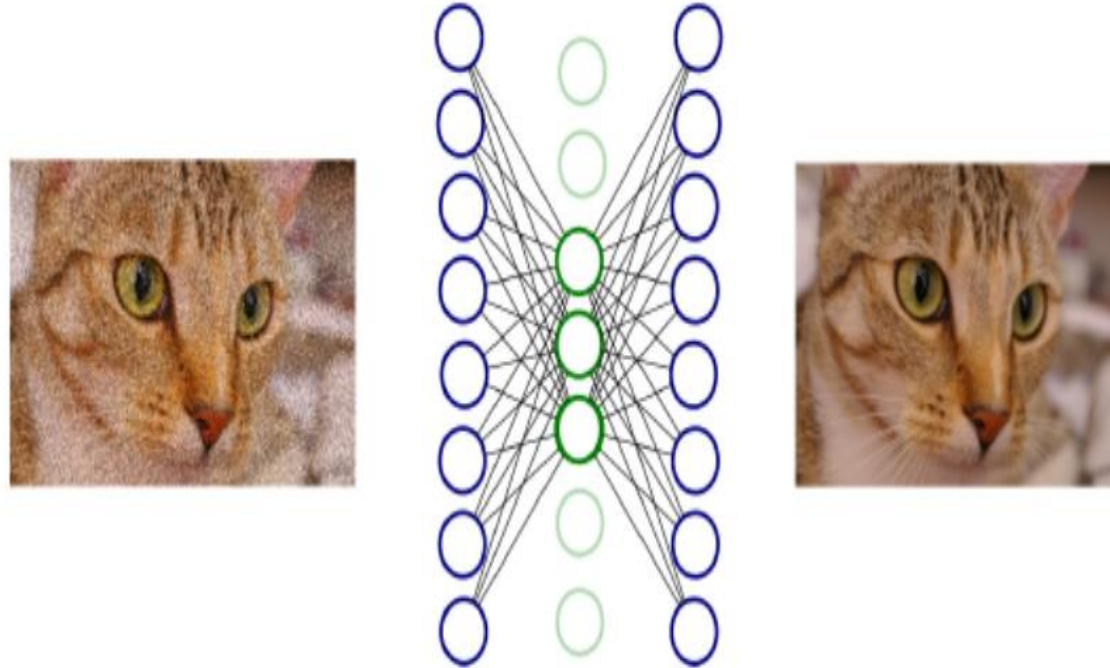


Feature learning and Dimensionality Reduction

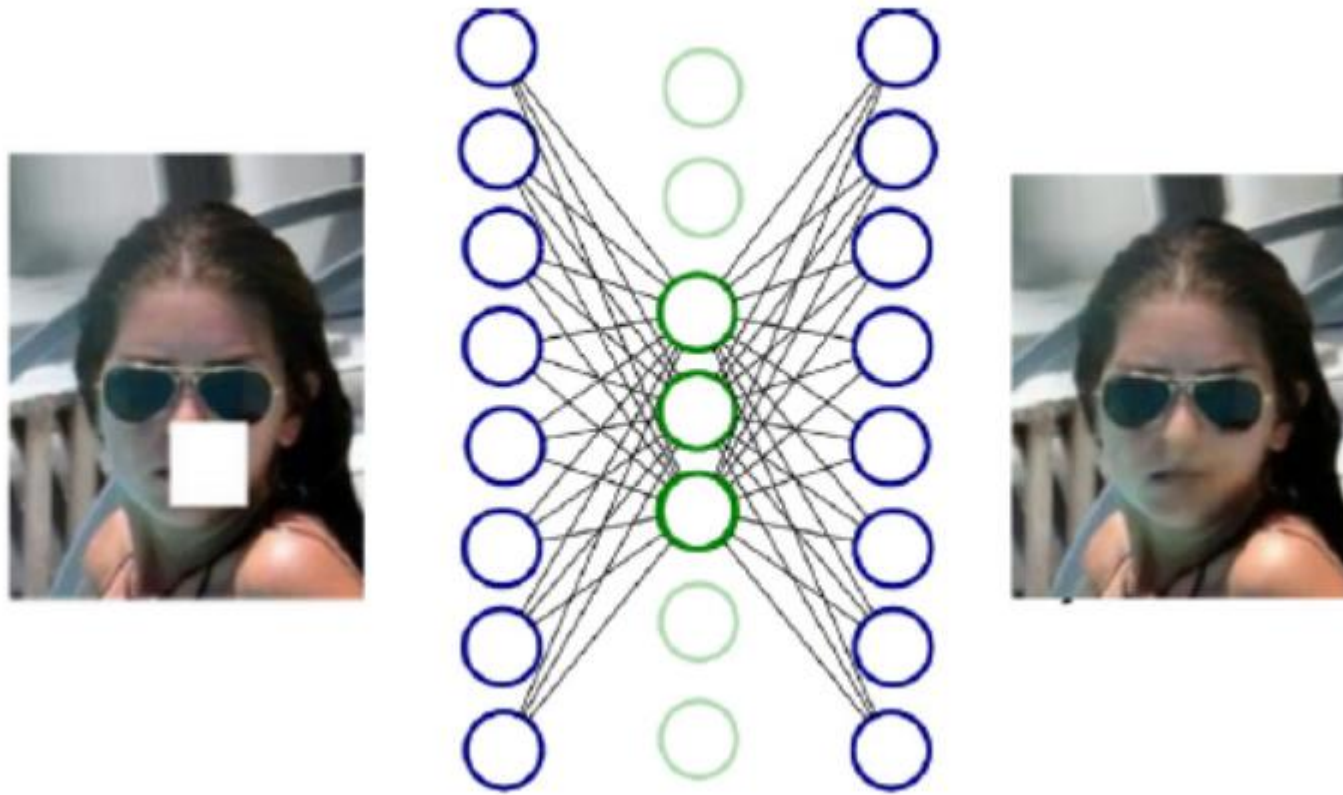
- Low-dim feature learning for 2000-dimensional bag-of-words documents



Denoising and Inpainting



Denoising and Inpainting



Undercomplete Autoencoders

- An autoencoder whose code dimension is less than the input dimension is called **undercomplete**.
- Learning an undercomplete representation **forces the autoencoder to capture the most salient features of the training data**.
- The learning process is described simply as minimizing a loss function
- $L(x, g(f(x)))$
- where L is a loss function penalizing $g(f(x))$ for being dissimilar from x , such as the mean squared error

Autoencoder

- When the decoder is linear and L is the mean squared error,
- an undercomplete autoencoder learns to span the same subspace as PCA.
- Autoencoders with **nonlinear encoder functions f** and **nonlinear decoder functions g** can thus learn **a more powerful nonlinear generalization of PCA**.

Regularized Autoencoders

- if the hidden code is allowed to have dimension equal to the input, and in the **overcomplete case** in which the hidden code has dimension greater than the input.

This means that the number of features in the latent representation exceeds the number of input features.

Having more neurons than **inputs allows the model to learn richer representations**, potentially capturing **complicated relationships** in the data.

- In these cases, even a linear encoder and linear decoder can learn to copy the input to the output **without learning anything useful about the data distribution**.

Regularized Autoencoders

- Rather than limiting the model capacity by keeping the encoder and decoder shallow and the code size small,
- **regularized autoencoders use a loss function** that encourages the model to have other properties besides the ability to copy its input to its output.
- **To prevent overfitting in overcomplete autoencoders, regularization techniques (like L1 or L2 regularization, dropout, or sparsity constraints) are often applied. This encourages the model to learn useful features**
- These other properties include sparsity of the representation, smallness of the derivative of the representation (relates to how sensitive the output (or reconstruction) of the model is to changes in the input.)
- , and robustness to noise or to missing inputs(models can effectively handle noisy input)

Regularized Autoencoders

- In the case of overcomplete autoencoders, regularization prevent the model from simply learning to output the training data without generalizing to unseen examples.
- L1 Regularization: **Encourages sparsity in the hidden layer, pushing many activations toward zero.**
- L2 Regularization: **Helps prevent large weights**, promoting smoother and more generalized solutions.
- **Sparsity Penalties:**
enforce that **only a small number of neurons are activated**,
making the representation more interpretable and robust.

Regularized Autoencoders

- When the derivatives of the representation (the activations in the hidden layers) are **small**,
- it means that small changes in the input will lead to small changes in the output
- A regularized autoencoder can be **nonlinear and overcomplete** but still learn something useful about the data distribution even if the model capacity is great enough to learn a trivial identity function.
- The identity function is a mapping where the output is the same as the input
- Most autoencoders are designed **to learn complex representations** of data **using nonlinear activation functions, such as ReLU, sigmoid, or tanh.**
- Nonlinear autoencoders typically consist of multiple layers, where the encoding and decoding processes use nonlinear transformations.

Regularized Autoencoders

- **How Autoencoders Achieve Robustness?**
- By compressing the data into a lower-dimensional representation, autoencoders can filter out noise, focusing on the underlying structure of the data rather than the noise itself.
- Sparse autoencoders enforce sparsity in the hidden layer, which encourages the model to focus on the most important features.
- This can help in ignoring noisy inputs that do not contribute to the essential structure of the data.

Regularized Autoencoders

- **How Autoencoders Achieve Robustness?**
- Techniques like **L2 regularization or dropout** can help **prevent overfitting to noisy data**,
- making the model more **generalizable to unseen examples**.
- A specific variant designed to enhance robustness is **denoising autoencoder**.
- During training, random noise is added to the input data, and the model is trained to reconstruct the original, noise-free input.
- This helps the autoencoder learn to ignore noise and capture meaningful features.

Sparse Autoencoders

- designed to learn efficient representations of data by enforcing sparsity in the hidden layer activations.
- This means that, for any given input, only a small number of neurons in the hidden layer will be activated, leading to a more compact representation.
- This is typically achieved using a **sparsity penalty in the loss function**, which can be implemented through various methods:
- **for example (L1 Regularization or KL Divergence or L2 Regularization)**

Sparse Autoencoders

- Sparse autoencoders are powerful tools **for learning efficient representations from high-dimensional data.**
- By enforcing sparsity in the hidden layer, they can extract meaningful features,
- improve robustness to noise, and prevent overfitting,
- making them suitable for a wide range of applications in machine learning and data analysis.

Sparse Autoencoders

- **Regularized maximum likelihood** is a powerful approach that combines the strengths of **maximum likelihood estimation with regularization techniques**.
- This combination enhances model robustness, improves generalization,
- and helps manage complexity in high-dimensional datasets,
- making it a fundamental tool in statistical modeling and machine learning.
- **regularized maximum likelihood corresponds to maximizing $p(\vartheta / x)$,**
which is equivalent to maximizing **$\log p(x / \vartheta) + \log p(\vartheta)$** .

Sparse Autoencoders

- Suppose we have a model with visible variables x and latent variables h , with an explicit joint distribution
- **$p_{\text{model}}(x, h) = p_{\text{model}}(h)p_{\text{model}}(x \mid h)$.**
- We refer to **$p_{\text{model}}(h)$ as the model's prior distribution** over the latent variables, representing the model's beliefs prior to seeing x .
- The log-likelihood can be decomposed as

$$\log p_{\text{model}}(\boldsymbol{x}) = \log \sum_h p_{\text{model}}(\boldsymbol{h}, \boldsymbol{x}).$$

Sparse Autoencoders

- we are maximizing

$$\log p_{\text{model}}(\mathbf{h}, \mathbf{x}) = \log p_{\text{model}}(\mathbf{h}) + \log p_{\text{model}}(\mathbf{x} \mid \mathbf{h}).$$

- The $\log p_{\text{model}}(\mathbf{h})$ term can be sparsity-inducing. For example, the **Laplace prior, corresponds** to an absolute value sparsity penalty

$$p_{\text{model}}(h_i) = \frac{\lambda}{2} e^{-\lambda |h_i|},$$

- Expressing the log-prior as an absolute value penalty, we obtain

$$\begin{aligned} \Omega(\mathbf{h}) &= \lambda \sum_i |h_i| \\ -\log p_{\text{model}}(\mathbf{h}) &= \sum_i \left(\lambda |h_i| - \log \frac{\lambda}{2} \right) = \Omega(\mathbf{h}) + \text{const} \end{aligned}$$

Sparse Autoencoders

- where the constant term depends only on λ and not h .
- We typically treat λ as a hyperparameter and discard the constant term since it does not affect the parameter learning.
- From this point of view of **sparsity as resulting from the effect of $p_{\text{model}}(h)$ on approximate maximum likelihood learning**,
- the sparsity penalty is not a regularization term at all.
- using a **Laplace prior** on the coefficients of a model encourages sparsity in the estimates, **meaning that many of the coefficients are pushed towards zero**.
- the Laplace prior is a powerful tool for achieving sparsity in high-dimensional statistical problems

Denoising Autoencoders

- Rather than adding a penalty Ω to the cost function, we can obtain an autoencoder that learns something useful by changing the **reconstruction error term of the cost function**.
- Traditionally, autoencoders minimize some function
- $L(x, g(f(x)))$
- where L is a loss function penalizing $g(f(x))$ for being dissimilar from x , such as the L^2 norm of their difference.
- This encourages $g \circ f$ to learn to be merely an identity function if they have the capacity to do so.

Denoising Autoencoders

- A denoising autoencoder or DAE instead minimizes
- $L(x, g(f(\bar{x})))$,
- where $\bar{x}$ is a copy of x that has been corrupted by some form of noise.
- Denoising autoencoders must therefore undo this corruption rather than simply copying their input.
- thus provide yet another example of how useful properties can emerge as a byproduct of minimizing reconstruction error

Regularizing by Penalizing Derivatives

- for regularizing an autoencoder is to use a penalty Ω as in sparse autoencoders,

$$L(\mathbf{x}, g(f(\mathbf{x}))) + \Omega(\mathbf{h}, \mathbf{x}),$$

but with a different form of Ω :

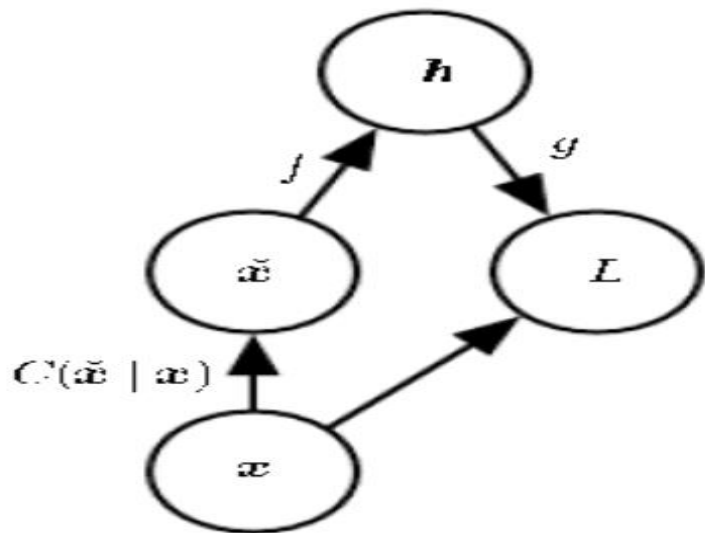
$$\Omega(\mathbf{h}, \mathbf{x}) = \lambda \sum_i \|\nabla_{\mathbf{x}} h_i\|^2.$$

Regularizing by Penalizing Derivatives

- This forces the model to learn a function that does not change much when x changes slightly.
 - Because this penalty is applied only at training examples,
 - it forces the autoencoder to learn features that capture information about the training distribution.
 - An autoencoder regularized in this way is called a **contractive autoencoder or CAE**.
 - This approach has theoretical connections to denoising autoencoders, manifold learning (Nonlinear dimensionality reduction,) and probabilistic modeling
- This helps in making learned features more invariant to small variations in input data, which is useful for tasks such as noise reduction and feature extraction.

Denoising Autoencoders (DAE)

- denoising autoencoder is an autoencoder that receives a corrupted data point as input and is trained to predict the original, uncorrupted data point as its output.
- corruption process $C(\bar{x} / x)$ which represents a conditional distribution over corrupted samples $\bar{x}$, given a data sample x .



The computational graph of the cost function for a denoising autoencoder, which is trained to reconstruct the clean data point x from its corrupted version $\bar{x}$.

This is accomplished by minimizing the

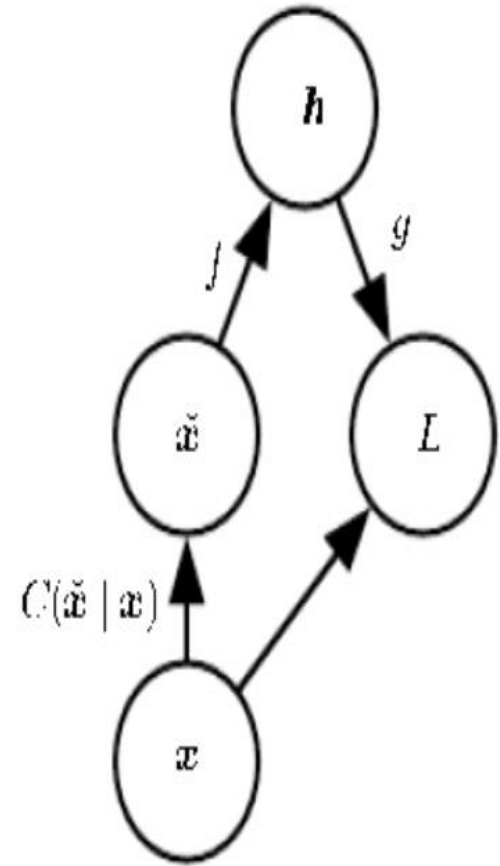
loss $L = -\log p_{\text{decoder}}(x / h = f(\tilde{x}))$,

where $\bar{x}$ is a corrupted version of the data example x , obtained through a given corruption process $C(\bar{x} / x)$.

distribution p_{decoder} is a factorial distribution whose mean parameters are emitted by a feedforward network g .

Denoising Autoencoders

- The autoencoder then learns a reconstruction distribution $p_{\text{reconstruct}}(x / \bar{x})$ estimated from training pairs $(x, \bar{x})$, as follows:
 - 1. Sample a training example x from the training data.
 - 2. Sample a corrupted version $\bar{x}$ from $C(\bar{x} / x = x)$.
 - 3. Use $(x, \bar{x})$ as a training example for estimating the autoencoder reconstruction distribution
- $p_{\text{reconstruct}}(x / \bar{x}) = p_{\text{decoder}}(x / h)$ with h the output of encoder $f(\tilde{x})$ and p_{decoder} typically defined by a decoder $g(h)$.

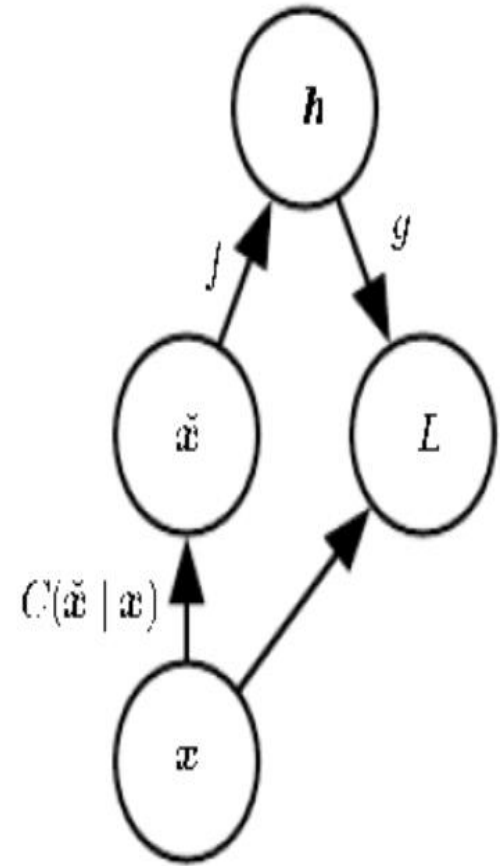


Denoising Autoencoders

- we can simply perform gradient-based approximate minimization (such as minibatch gradient descent) on the negative log-likelihood **$-\log p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h})$** .
- DAE as performing stochastic gradient descent on the following expectation:

$$- \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}(\mathbf{x})} \mathbb{E}_{\tilde{\mathbf{x}} \sim C(\tilde{\mathbf{x}} \mid \mathbf{x})} \log p_{\text{decoder}}(\mathbf{x} \mid \mathbf{h} = f(\tilde{\mathbf{x}}))$$

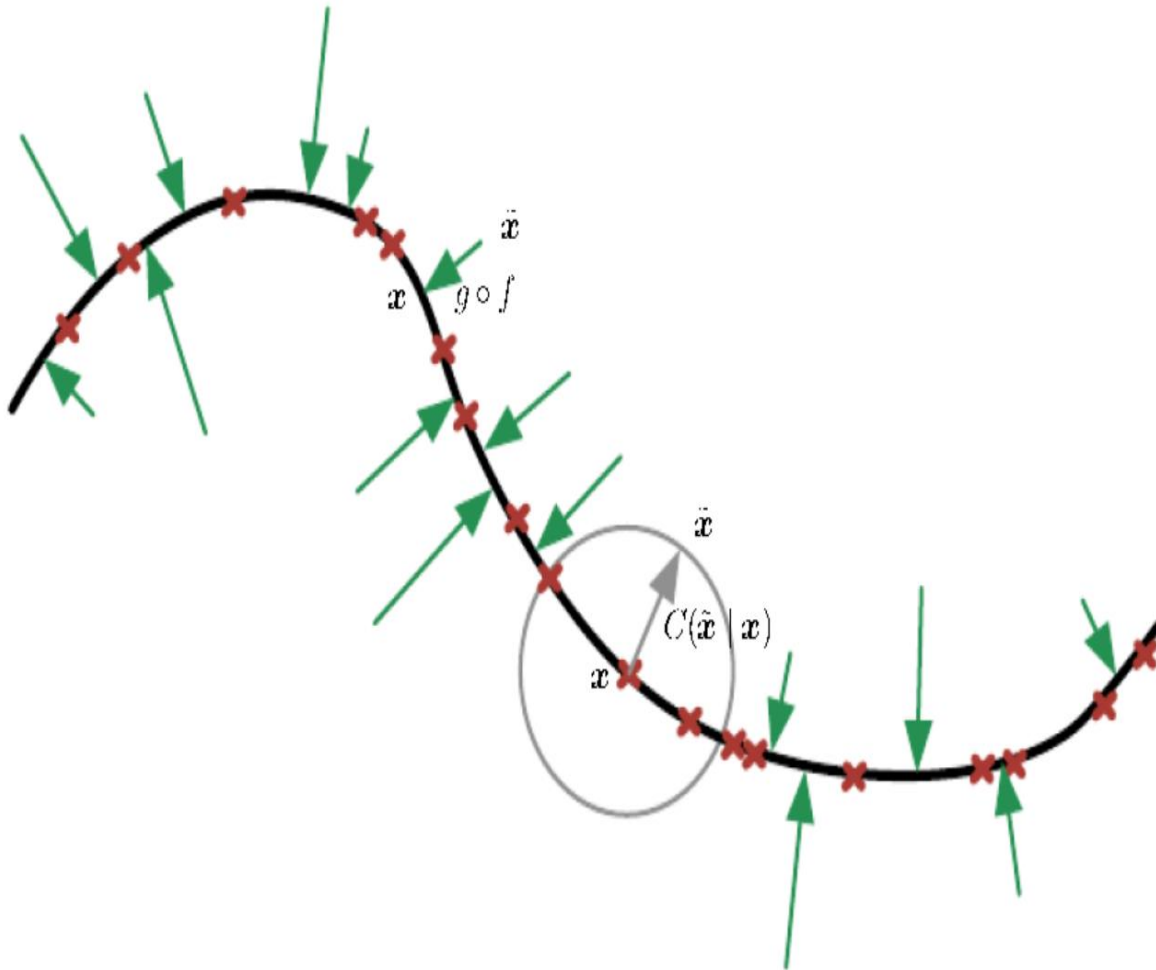
- where $\hat{p}_{\text{data}}(\mathbf{x})$ is the training distribution
- **representing underlying distribution of clean data.**



Denoising Autoencoder

- The aim is to learn this distribution through training on noisy examples,
- allowing the model to recover clean data effectively.
- Denoising Autoencoder, the loss function is
- **mean squared error or binary cross-entropy**
- between the original clean input x and the reconstructed output $\hat{x}$:

Denoising Autoencoders

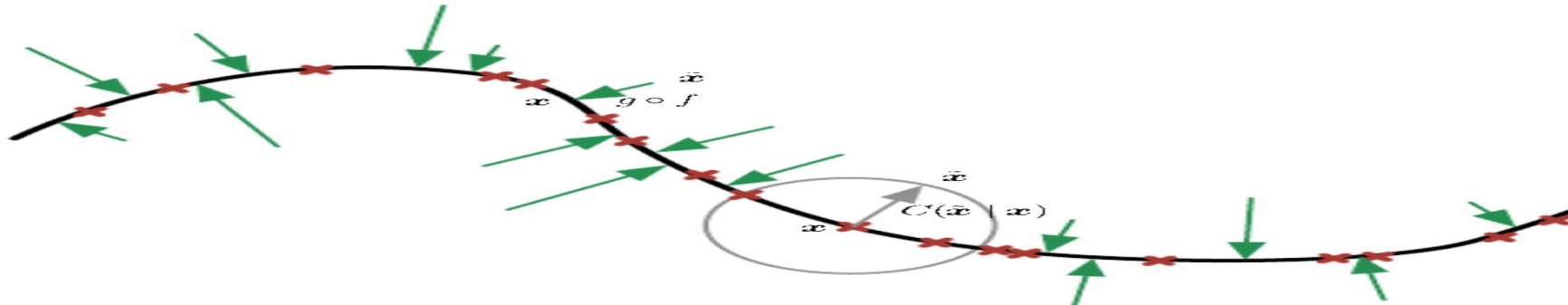


A denoising autoencoder is trained to map a corrupted data point $\tilde{x}$ back to the original data point x .

We illustrate training examples x as red crosses lying near a low-dimensional manifold illustrated with the bold black line.

We illustrate the corruption process $C(\tilde{x} | x)$ with a gray circle of equiprobable corruptions

Denoising Autoencoders



A gray arrow demonstrates how one training example is transformed into one sample from this corruption process.

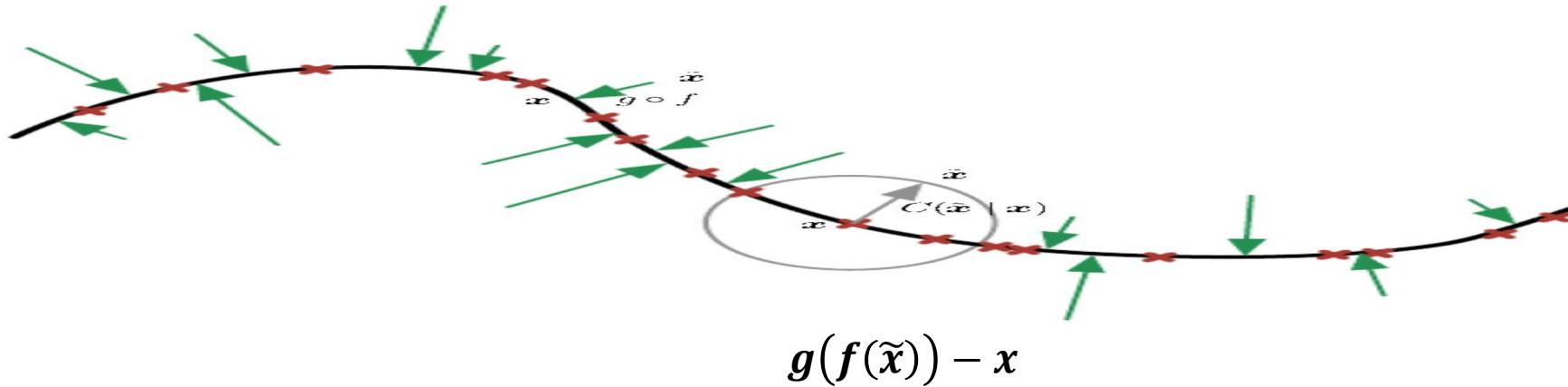
When the denoising autoencoder is trained to minimize the average of squared errors

$$\|g(f(\tilde{x})) - x\|^2$$

The reconstruction of $g(f(\tilde{x}))$ estimates $E_{x, \tilde{x} \sim p_{data}(x)} C(\tilde{x} | x) [X | \tilde{x}]$

$E_{x, \tilde{x} \sim p_{data}(x)}$ taking the expectation of x over the distribution of samples $p_{data}(x)$
corruption process $C(\tilde{x} | x)$

Denoising Autoencoders



points approximately towards the nearest point on the manifold ,
 since $g(f(\tilde{x}))$ estimates the center of mass of the clean points x
 The autoencoder thus learns a vector field $g(f(x)) - x$ indicated by the green arrows.

This vector field estimates the score $\nabla_x \log p_{\text{data}}(x)$ up to a multiplicative factor that is the average root mean square reconstruction error.

Estimating the Score

- the score is a particular gradient field:
- $\nabla_x \log p(x)$.
- regarding autoencoders, it is sufficient to understand that learning the gradient field of $\log p_{\text{data}}$ is one way to learn the structure of p_{data} itself.
- It can guide the optimization process,
- ensuring that the model learns to generate outputs that are more likely according to the true data distribution.
- A very important property of DAEs is that their training criterion (with conditionally Gaussian $p(x / h)$) makes the autoencoder learn a vector field $(g(f(x)) - x)$ that estimates the score of the data distribution

Estimating the Score

- training with the squared error criterion

$$||g(f(\tilde{x})) - x||^2$$

and corruption

$$C(\tilde{\mathbf{x}} = \tilde{x} | x) = \mathcal{N}(\tilde{x}; \mu = x, \Sigma = \sigma^2 I)$$

- with noise variance σ^2
- there is no guarantee that the reconstruction $g(f(x))$ minus the input x corresponds to the gradient of any function, let alone to the score .

Contractive Autoencoders (CAEs)

- **Contractive Autoencoders** (CAEs) are a type of neural network designed to learn robust representations of data by enforcing a regularization term
- encourages the model to be invariant to small changes in the input.
- useful for tasks where robustness to noise or slight variations is essential
- Like traditional autoencoders, CAEs consist of an encoder and a decoder.
- encoder compresses input data into a lower-dimensional representation,
- while the decoder reconstructs the input from this representation.

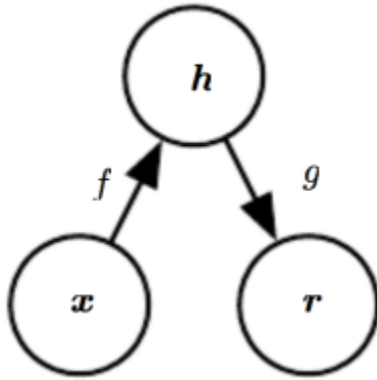
Contractive Autoencoders (CAEs)

- **Contractive Loss:**
- The main innovation in CAEs is the introduction of a contractive loss term in addition to the reconstruction loss.
- The loss function for a CAE can be expressed as
- $L(x, \hat{x}) = L_{reconstruction}(x, \hat{x}) + \lambda L_{contractive}(h)$
- Where $L_{reconstruction}(x, \hat{x})$ is typically the **mean squared error or binary cross-entropy** between the input x and the reconstructed output $\hat{x}$
- $L_{contractive}(h)$ encourages the hidden representation h to be robust to small perturbations in the input.
- λ is a hyperparameter that controls the trade-off between reconstruction accuracy and contractive regularization.

Contractive Autoencoders (CAEs)

- **Advantages**
- **Robust Representations:**
 - By enforcing stability, CAEs learn features that are less sensitive to noise and perturbations in the input data.
- **Improved Generalization:**
 - generalize better on unseen data due to the regularization effect.
- **Feature Extraction:**
 - effectively extract meaningful features from the data,
 - which can be useful for downstream tasks like classification or clustering.

- Contractive autoencoder has an explicit regularizer on $h=f(x)$,
- encouraging the derivatives of f to be as small as possible:



$$\Omega(\mathbf{h}) = \lambda \left\| \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right\|_F^2$$

- Where $L(f(\mathbf{x})) + \Omega(\mathbf{h})$
- Penalty $\Omega(\mathbf{h})$ is the squared Frobenius norm (sum of squared elements) of Jacobian matrix of partial derivatives associated with encoder function

- New loss function:

$$\underbrace{l(f(\mathbf{x}^{(t)}))}_{\text{autoencoder reconstruction}} + \lambda \underbrace{\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2}_{\text{Jacobian of encoder}}$$

- where, for binary observations:

$$l(f(\mathbf{x}^{(t)})) = -\sum_k \left(x_k^{(t)} \log(\hat{x}_k^{(t)}) + (1 - x_k^{(t)}) \log(1 - \hat{x}_k^{(t)}) \right)$$

$$\|\nabla_{\mathbf{x}^{(t)}} \mathbf{h}(\mathbf{x}^{(t)})\|_F^2 = \sum_j \sum_k \left(\frac{\partial h(\mathbf{x}^{(t)})_j}{\partial x_k^{(t)}} \right)^2$$

References

- Ian Goodfellow and YoshuaBengio and Aaron Courville,Deep Learning (2016) AnMIT Press book, [\(T2\)](http://www.deeplearningbook.org)
- . Skansi S., Introduction to Deep Learning - From Logical Calculus to Artificial Intelligence, 1st Edition, Springer International Publishing, 2018.(T2)
- Buduma N., Fundamentals of Deep Learning, 1st Edition, O Reilly Media, 2016.(R1)
- Piyush Rai, Autoencoders, Extensions, and Applications, IIT Kanpur
- **Online :**
<https://cedar.buffalo.edu/~srihari/CSE676/14.3%20Learning%20Manifolds.pdf>